

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 2 — Servicos, Analise e Banco de Dados

Coaching Services, Knowledge & Retrieval, Analysis Engines, Database
Schema, Training Pipeline

Autore: Renan Augusto Macena

Marzo 2026

Ultimate CS2 Coach — Parte 2: Servicos, Analise, Conhecimento, Controle, Progresso e Banco de Dados

Temas: Servicos de Coaching (pipeline 4-modos: COPER/Hibrido/RAG/Base, Ollama LLM), Motores de Coaching (11 motores de analise game-theoretic e estatistica), Conhecimento e Recuperacao (RAG + Experience Bank COPER), Motores de analise (belief model bayesiano, expectiminimax, momentum, papeis, deception index, entropia, engagement range, utility economy, win probability), Processamento e Feature Engineering (vetor canonico 25-dim, baselines profissionais), Modulo de Controle (lifecycle daemon, fila de ingestao, controle ML), Progresso e Tendencias (tracking longitudinal), Banco de Dados e Storage (schema SQLite WAL three-tier), Pipeline de Treinamento e Orquestracao, Funcoes de Perda.

Autor: Renan Augusto Macena

Este documento e a continuacao da **Parte 1B** — *Os Sentidos e o Especialista*, que cobre o modelo RAP Coach e todas as fontes de dados externas. A Parte 2 documenta os servicos de coaching que consomem essas saidas neurais e as transformam em conselhos acionaveis, junto com os motores de analise, o conhecimento tatico e a infraestrutura de storage e treinamento.

Indice

Parte 2 — Servicos, Analise e Banco de Dados (este documento)

5. **Subsistema 3 — Servicos de Coaching** ([backend/services/](#))
 - CoachingService (pipeline 4-modos)
 - OllamaWriter (refinamento LLM)
 - AnalysisOrchestrator
 - Server FastAPI (endpoints REST standalone)
 - Onboarding (fluxo usuario a 3 estagios) 5B. **Subsistema 3B — Motores de Coaching** ([backend/coaching/](#))
6. **Subsistema 4 — Conhecimento e Recuperacao** ([backend/knowledge/](#))
 - RAG Knowledge (retrieval por padroes de coaching)

- Experience Bank COPER (storage e retrieval de experiencias)

7. Subsistema 5 — Motores de analise (`backend/analysis/`)

- Belief Model bayesiano
- Game Tree (expectiminimax)
- Momentum, Role Classifier, Deception Index, Entropy Analysis
- Engagement Range, Utility Economy, Win Probability, Blind Spots

8. Subsistema 6 — Processamento e Feature Engineering (`backend/processing/`)

- Vectorizer (contrato canonico METADATA_DIM=25)
- TensorFactory (construcao tensor visao/mapa)
- Baselines profissionais e meta drift detection
- Heatmap, validacao

9. Subsistema 7 — Modulo de Controle (`backend/control/`)

10. Subsistema 8 — Progresso e Tendencias (`backend/progress/`)

11. Subsistema 9 — Banco de Dados e Storage (`backend/storage/`)

12. Pipeline de Treinamento e Orquestracao

13. Funcoes de Perda — Detalhe Implementativo

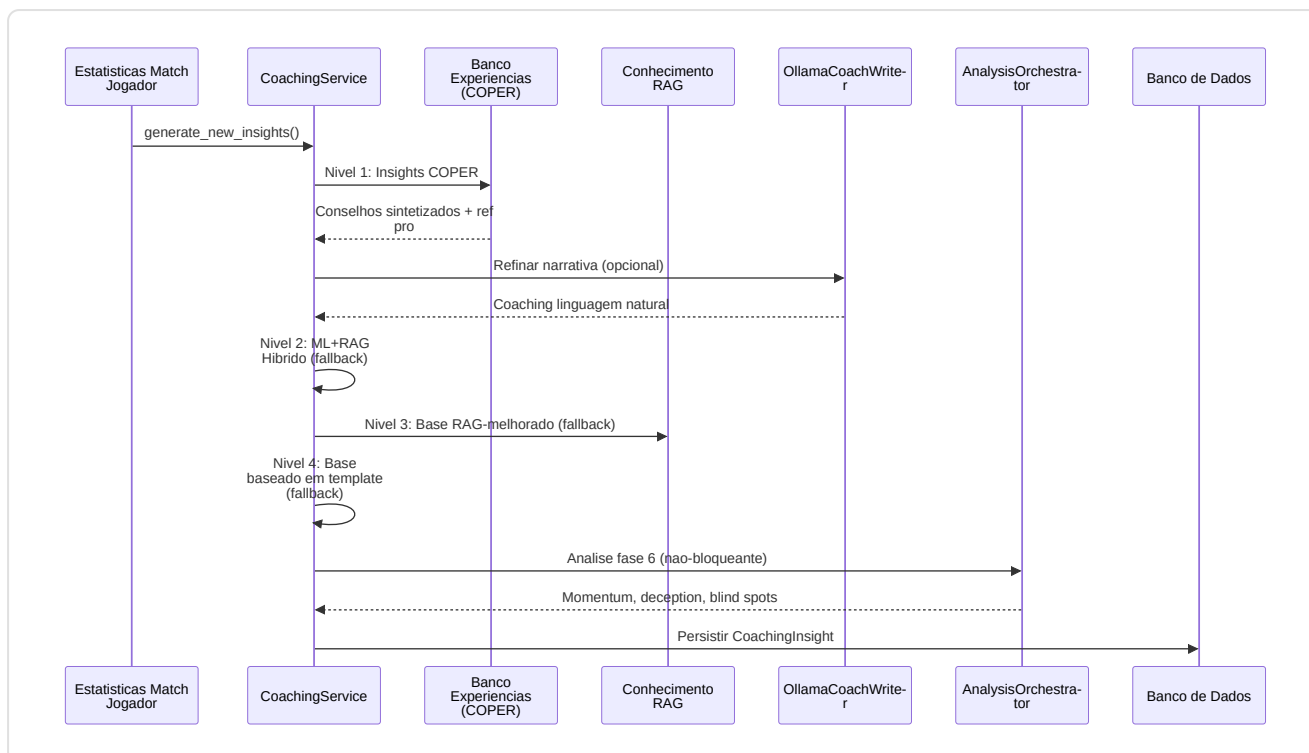
5. Subsistema 3 — Servicos de Coaching

Pasta no repo: `backend/services/` **Arquivos:** `coaching_service.py` , `ollama_writer.py` , `analysis_orchestrator.py`

Este subsistema e o **centro decisional**: pega tudo o que as redes neurais e os sistemas de conhecimento produziram e o sintetiza em verdadeiros conselhos de coaching para o jogador.

-CoachingService (`coaching_service.py`)

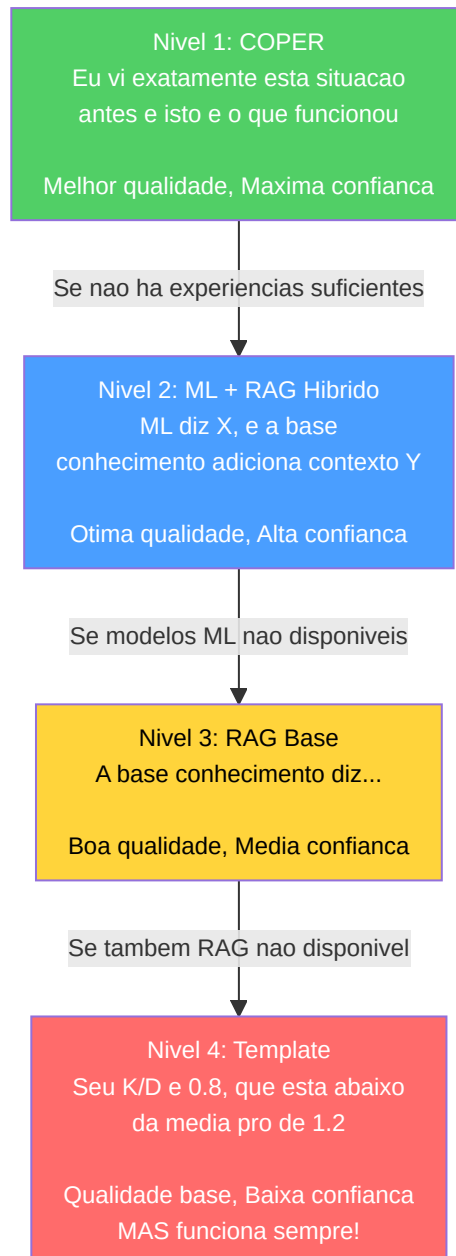
O **motor de sintese central** que implementa uma **cadeia de fallback de coaching de 4 niveis** com degradacao gradual:



Explicacao Diagrama: Siga as setas de cima para baixo: este e o "processo de pensamento" do treinador em ordem: (1) Chegam os dados da partida. (2) O treinador pergunta primeiro ao Experience Bank: "Ja vimos esta situacao? O que funcionou?". (3) Se a resposta parece robotica demais, a envia opcionalmente ao Ollama (um programador de inteligencia artificial local) para torna-la mais natural. (4) Se o Experience Bank nao dispoe de dados suficientes, recorre ao modo hibrido (previsoes ML + conhecimento RAG combinados). (5) Se tambem os modelos ML nao estao disponiveis, recorre so a RAG (buscando sugestoes pertinentes). (6) Se tambem RAG falha, usa simples modelos estatisticos ("Seu ratio K/D e 0,8, abaixo da media"). (7) Enquanto isso, em background, 7 motores de analise executam investigacoes especiais atraves de 5 pipelines (momentum, deception, entropia, estrategia+blind spots, distancia engajamento). (8) Tudo e salvo no banco de dados para referencia futura.

Cadeia de fallback de 4 niveis:

Nivel	Metodo	Confianca	Quando utilizado
1.COPER	<code>_generate_coper_insights()</code>	Maxima	Padrao — sintese baseada na experiencia
2.Hibrido	<code>_generate_hybrid_insights()</code>	Alta	Se COPER nao tem experiencia suficiente
3.RAG Base	<code>_enhance_with_rag()</code>	Media	Se os modelos ML nao estao disponiveis
4.Template	Modelo estatistico de base	Baixa	Ultimo recurso — retorna sempre <i>algo</i>



Design chave: Nunca retorna zero insights. A analise de Fase 6 e nao bloqueante (encapsulada em try-catch, registrada de forma nao fatal).

Correcao G-08 (Coaching Fallback): Anteriormente, o metodo `_generate_coper_insights()` nao recebia os parametros `deviations` e `rounds_played` do chamador, causando um fallback silencioso aos templates base. Apes a remediacao, `generate_new_insights()` agora passa explicitamente ambos os parametros (`deviations=deviations, rounds_played=rounds_played`) ao handler COPER, garantindo que o Nivel 1 COPER possa gerar insights contextuais baseados nos desvios estatisticos reais do jogador em relacao a baseline pro. Sem esta correcao, o sistema sempre degradaria ao Nivel 4 (template) mesmo quando os dados COPER estivessem disponiveis.

Enriquecimento da baseline temporal (Proposta 11): O servico de coaching agora integra comparacoes pro ponderadas no tempo atraves de dois novos metodos:

- `_get_temporal_baseline(map_name)` — recupera uma baseline pro ponderada com base em `TemporalBaselineDecay` (meia-vida = 90 dias) ao inves de medias estaticas
- `_baseline_context_note(deviations, map_name)` — gera uma descricao em linguagem natural ("Com base nos dados pro recentes ponderados pela recencia, seu ADR e inferior em 12% em relacao a meta media atual em de_mirage") para o enriquecimento COPER

Isto garante que os insights de coaching reflitam a **meta media atual** em vez de medias historicas obsoletas. Se os dados temporais nao sao suficientes (< 10 stat cards), o servico recorre a funcao legacy `get_pro_baseline()` de forma transparente.

Inferencia da fase do round (`_infer_round_phase`): Valor do equipamento -> classificacao da fase do round:

Valor do equipamento	Fase do round
< \$1.500	pistol
\$1.500 - \$2.999	eco
\$ 3.000 - \$ 3.999	force
>= \$ 4.000	full_buy

Classificacao do intervalo de saude (`_health_to_range`): Usado para o hashing do contexto COPER: "full" (>=80), "damaged" (40-79), "critical" (<40).

-OllamaCoachWriter (`ollama_writer.py`)

Transforma as informacoes de coaching estruturadas em linguagem natural atraves de LLM local (Ollama).

- **Singleton** via factory `get_ollama_writer()`
- **Funcionalidade sinalizada:** configuracao `USE_OLLAMA_COACHING` (padrao: False)
- **Degradacao graciosa:** retorna o texto original se Ollama nao esta disponivel
- **Prompt de sistema:** tom de especialista de coaching CS2, <100 palavras, acionavel, encorajador

-AnalysisOrchestrator (`analysis_orchestrator.py`)

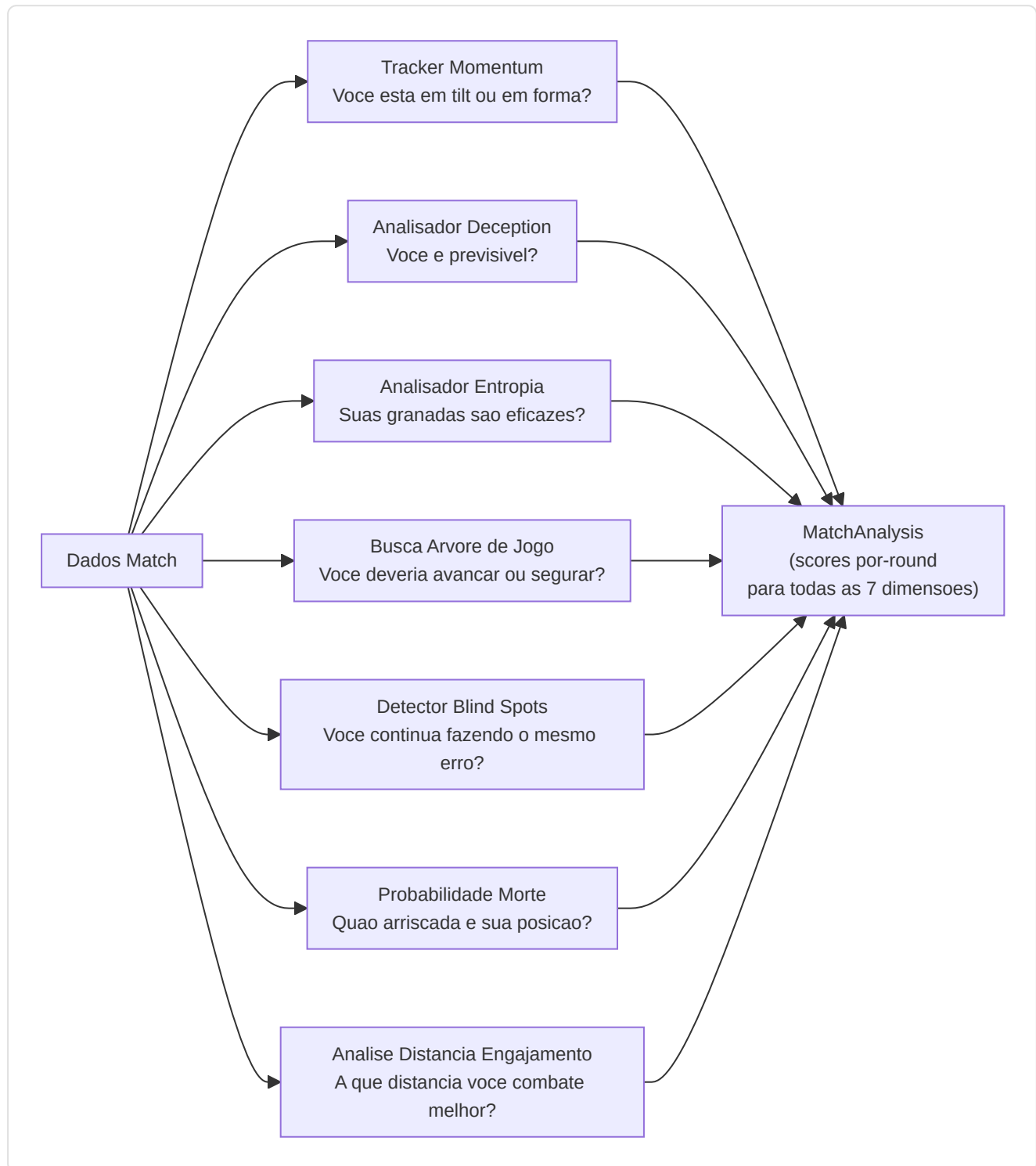
Sintetiza a analise avancada de Fase 6 instanciando 7 motores e orquestrando 5 pipelines de analise:

Input: Dados de tick da partida, eventos, estatisticas dos jogadores **Output:** `MatchAnalysis` com objetos `RoundAnalysis` por round contendo:

- `momentum_score` (tilt/sequencia vencedora)
- `deception_score` (sofisticacao tatica)
- `utility_entropy` (medicao da eficacia)

- `blind_spots` (lacunas estrategicas)
- `strategy_rec` (recomendacao da arvore de jogo)
- `engagement_range` (analise distancia de engajamento)

Motores instanciados: `belief_estimator`, `deception_analyzer`, `momentum_tracker`, `entropy_analyzer`, `game_tree`, `blind_spot_detector`, `engagement_analyzer` (7 motores). O `belief_estimator` e instanciado mas atualmente nao invocado diretamente como pipeline separado.



-Servicos Adicionais (Nao documentados anteriormente)

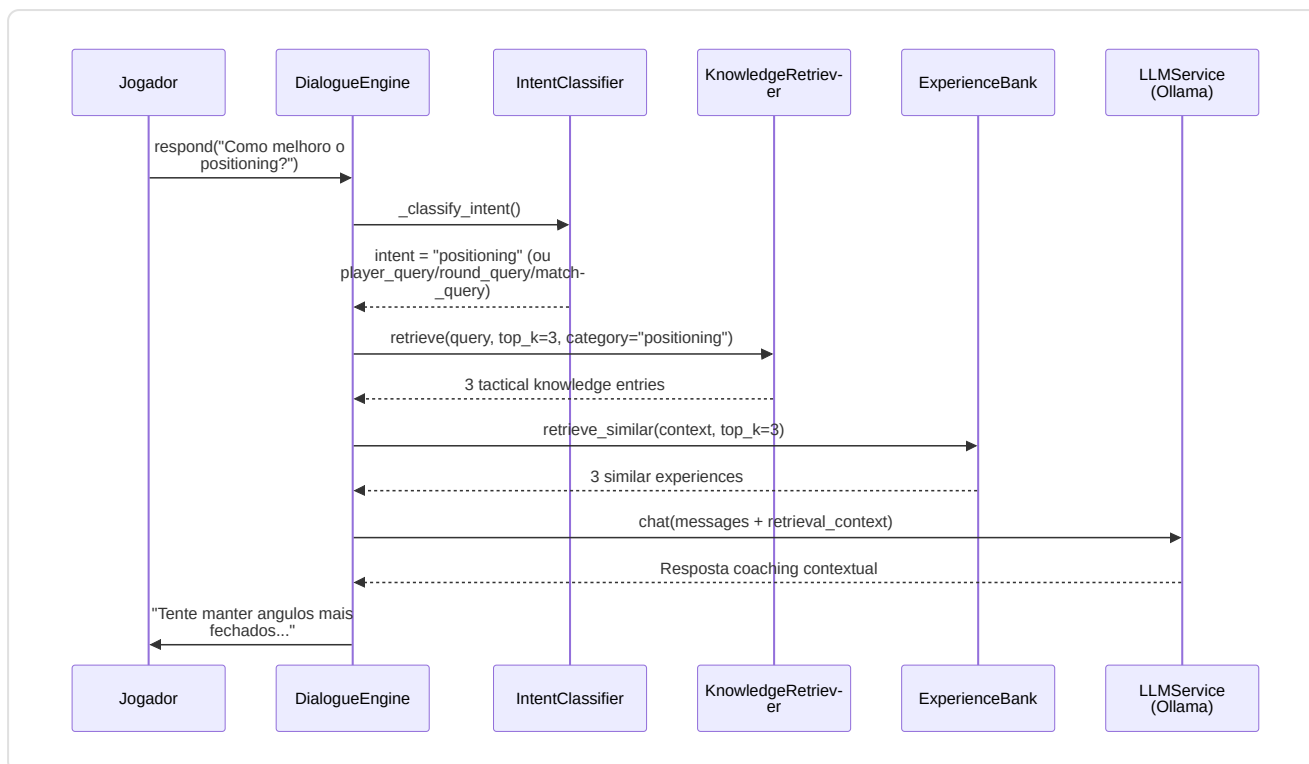
Alem dos tres servicos principais (CoachingService, OllamaCoachWriter, AnalysisOrchestrator), o diretorio `backend/services/` contem **7 servicos adicionais** que completam o ecossistema de coaching:

CoachingDialogueEngine (`coaching_dialogue.py`)

Motor de dialogo multi-turno com augmentacao RAG e Experience Bank. Evolui o single-shot OllamaCoachWriter em uma **sessao interativa** onde os jogadores podem fazer perguntas follow-up sobre seus desempenhos.

Componente	Detalhe
Classificacao intent	Keyword-based: 7 categorias (positioning, utility, economy, aim, player_query, round_query, match_query) + general fallback
Contexto sliding window	<code>MAX_CONTEXT_TURNS = 6</code> (12 mensagens: 6 user + 6 assistant)
Retrieval augmentation	RAG top-3 + Experience Bank top-3, injetados na mensagem do usuario
Player entity detection	Integracao com <code>PlayerLookupService</code> para detectar mencoes de jogadores profissionais na mensagem do usuario e injetar blocos "VERIFIED PLAYER DATA" no contexto LLM
Fallback offline	Template-based com RAG-only quando Ollama nao esta disponivel
Singleton	<code>get_dialogue_engine()</code> — factory module-level
Anti-hallucination (WR-78/79)	System prompt com regras: "use ONLY verified data", provenance markers ("pro data" vs "user data"), nunca fabricar narrativas taticas

Pipeline de resposta:



Gestao sessao: `start_session(player_name, demo_name)` -> carrega contexto do DB (ultimos 5 insights, area de foco primaria) -> gera mensagem de abertura via LLM -> `respond(user_message)` para turnos subsequentes -> `clear_session()` para reset.

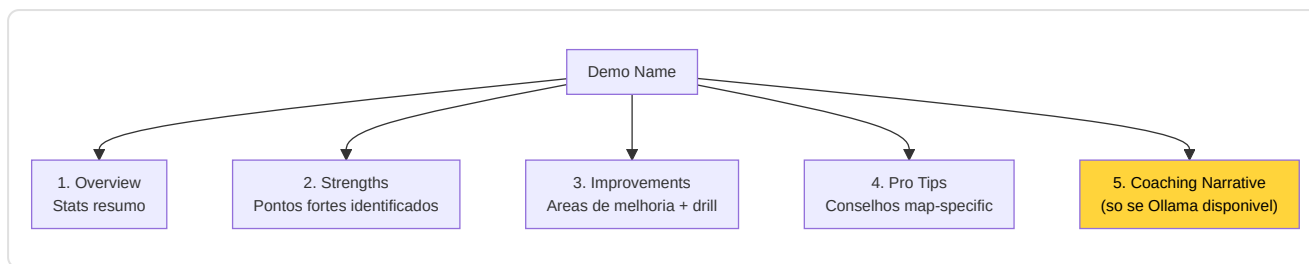
Seguranca do historico (F5-06): A mensagem do usuario e adicionada ao historico apenas *apos* obter uma resposta valida do LLM, evitando estados inconsistentes em caso de execucao.

LessonGenerator (`lesson_generator.py`)

Gerador de licoes educacionais estruturadas a partir da analise das demos:

Limiar	Constante	Valor	Uso
ADR forte	<code>_ADR_STRONG_THRESHOLD</code>	75.0	Identifica pontos fortes
ADR fraco	<code>_ADR_WEAK_THRESHOLD</code>	60.0	Identifica areas de melhoria
HS% forte	<code>_HS_STRONG_THRESHOLD</code>	0.40	Precisao acima da media
HS% fraco	<code>_HS_WEAK_THRESHOLD</code>	0.35	Precisao abaixo da media
Rating acima da media	<code>_RATING_ABOVE_AVG</code>	1.0	Desempenho positivo
KAST forte	<code>_KAST_STRONG_THRESHOLD</code>	0.70	Contribuicao consistente
Ratio mortes	<code>_DEATH_RATIO_WARNING</code>	1.5x	deaths > kills x 1.5 = warning

Estrutura licao gerada:



Pro Tips map-specific: Banco interno com conselhos para mirage, inferno, dust2, ancient, nuke. Fallback a conselhos gerais para mapas nao cobertos.

LLMService (`llm_service.py`)

Servico de integracao Ollama para inferencia LLM local:

Parametro	Valor	Descricao
<code>OLLAMA_URL</code>	<code>http://localhost:11434</code>	Endpoint Ollama (env: <code>OLLAMA_URL</code>)
<code>DEFAULT_MODEL</code>	<code>llama3.1:8b</code>	Modelo 8B general-purpose (env: <code>OLLAMA_MODEL</code>)
<code>_AVAILABILITY_TTL</code>	60s	Cache de disponibilidade
<code>temperature</code>	0.7	Criatividade das respostas
<code>top_p</code>	0.9	Nucleus sampling
<code>num_predict</code>	500	Limite comprimento resposta

APIs suportadas:

Metodo	Endpoint Ollama	Uso
<code>generate(prompt)</code>	<code>/api/generate</code>	Geracao single-shot
<code>chat(messages)</code>	<code>/api/chat</code>	Conversacao multi-turno
<code>generate_lesson(insights)</code>	<code>/api/generate</code>	Licoes de insights RAP
<code>explain_round_decision(round_data)</code>	<code>/api/generate</code>	Explicacao round singular
<code>generate_pro_tip(context)</code>	<code>/api/generate</code>	Tip contextual

Auto-discovery modelo: Se o modelo configurado nao esta disponivel, usa automaticamente o primeiro modelo instalado. Singleton via `get_llm_service()` .

VisualizationService (`visualization_service.py`)

Gera graficos radar Matplotlib para comparacao User vs Pro:

- `generate_performance_radar(user_stats, pro_stats, output_path)` -> arquivo PNG com radar overlay
- `plot_comparison_v2(p1_name, p2_name, p1_stats, p2_stats)` -> `io.BytesIO` buffer PNG
- Features comparadas: avg_kills, avg_adr, avg_hs, avg_kast, accuracy

- Renderizacao wrappada em try/except (F5-19) para gerenciar stats vazias ou backend matplotlib ausente

ProfileService (`profile_service.py`)

Orquestrador de sincronizacao perfis externos:

- `fetch_steam_stats(steam_id)` -> nickname, avatar, playtime CS2 (horas)
- `fetch_faceit_stats(nickname)` -> faceit_elo, faceit_level, faceit_id
- `sync_all_external_data(steam_id, faceit_name)` -> upsert em `PlayerProfile` no DB
- Chaves API carregadas de environment/keyring (F5-22), nunca hardcoded

AnalysisService (`analysis_service.py`)

Servico de coordenacao analise com drift detection:

- `analyze_latest_performance(player_name)` -> ultimas stats do DB
- `get_pro_comparison(player_name, pro_name)` -> comparacao side-by-side
- `check_for_drift(player_name)` -> `detect_feature_drift()` nas ultimas 100 partidas

TelemetryClient (`telemetry_client.py`)

Cliente async para envio de metricas a servidor central:

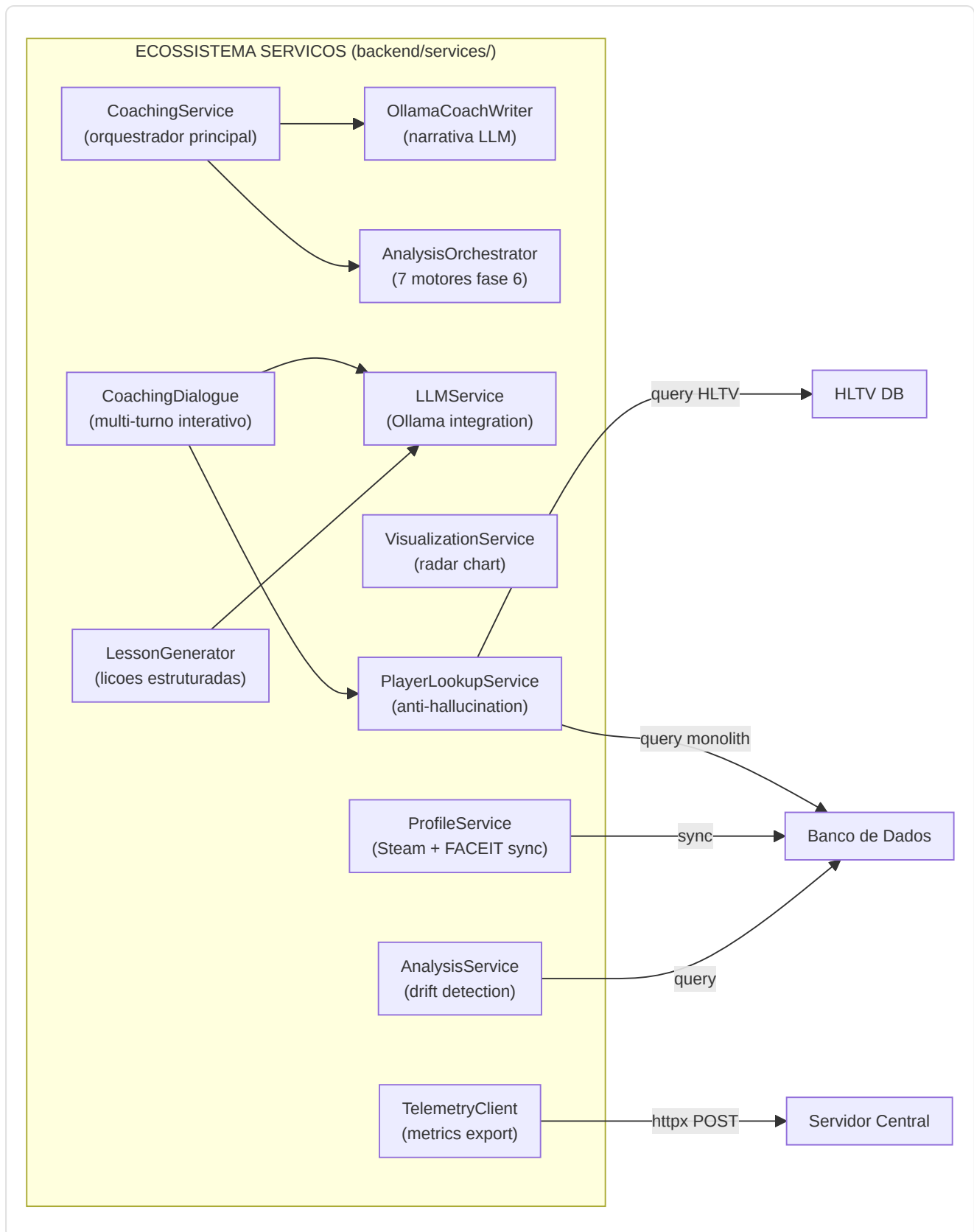
- Protocolo: `httpx.Client` -> `POST /api/ingest/telemetry`
- URL configuravel: `CS2_TELEMETRY_URL` (default: `http://127.0.0.1:8000`)
- Fallback graceful se httpx nao instalado (feature opcional)
- **Anti-fabrication compliant:** nenhum dado sintetico no self-test

PlayerLookupService (`player_lookup.py`)

Servico de busca jogadores profissionais que previne a alucinacao LLM injetando dados verificados no contexto do dialogo de coaching:

Componente	Detalhe
Matching em 3 niveis	Exato (case-insensitive) -> Fuzzy (SequenceMatcher >= 0.75) -> Pattern (regex nickname)
Cache nickname	TTL 60s, carrega todos os <code>ProPlayer.nickname</code> do DB HLTV na inicializacao
Stop-word filter	89 palavras comuns inglesas filtradas para evitar falsos positivos
Output	<code>ProPlayerProfile</code> dataclass: nickname, hltv_id, real_name, country, team, estatisticas HLTV (rating, KPR, ADR, KAST), performance de demos locais
Integracao	<code>CoachingDialogueEngine</code> -> <code>detect_player_mentions()</code> -> <code>lookup_player()</code> -> <code>format_player_context()</code> -> bloco "VERIFIED PLAYER DATA" injetado no contexto LLM
Singleton	<code>get_player_lookup_service()</code>

Pipeline anti-alucinacao:



-Server FastAPI (`server.py`)

Servidor REST standalone que expoe endpoints para telemetria, insights, health e controle remoto dos servicos. **Nao esta integrado no fluxo principal da aplicacao** — e um servico de utilidade separado, iniciavel independentemente.

Rate Limiting: Classe `RateLimiter` com janela deslizante — maximo 10 requisicoes por minuto por endereco IP. A estrutura interna `_requests` mapeia cada IP a uma lista de timestamps `float`, purgando os timestamps expirados a cada verificacao.

Lifecycle: O servidor utiliza o lifespan handler assincrono do FastAPI para inicializar o banco de dados na inicializacao via `init_database()`.

Modelos Pydantic:

Modelo	Tipo	Campos principais
<code>InsightRead</code>	Response	<code>id</code> , <code>player_name</code> , <code>title</code> , <code>message</code> , <code>focus_area</code> , <code>severity</code> , <code>created_at</code>
<code>MatchTelemetry</code>	Request	<code>player_id</code> , <code>match_id</code> , <code>stats</code> (dict), <code>timestamp</code> (float)

Tabela de Endpoints:

Metodo	Path	Rate Limited	Descricao
POST	<code>/api/ingest/telemetry</code>	Sim	Ingestao de dados de telemetria da partida. Escrita em background no disco via <code>_write_telemetry_to_disk()</code>
GET	<code>/api/insights</code>	Nao	Retorna insights de coaching para um jogador (query param: <code>player_name</code>)
GET	<code>/api/status</code>	Nao	Status da aplicacao: versao, uptime, estado do banco de dados
GET	<code>/api/console/health</code>	Nao	Health check para monitoramento de servicos
POST	<code>/api/training/start</code>	Nao	Inicia treinamento ML via Console backend
POST	<code>/api/training/stop</code>	Nao	Para treinamento ML
POST	<code>/api/training/pause</code>	Nao	Pausa treinamento ML
POST	<code>/api/training/resume</code>	Nao	Retoma treinamento ML da pausa
POST	<code>/api/services/hunter/start</code>	Nao	Inicia servico HLTV Hunter
POST	<code>/api/services/hunter/stop</code>	Nao	Para servico HLTV Hunter
GET	<code>/api/services/status</code>	Nao	Status de todos os servicos gerenciados

Seguranca telemetria: `_write_telemetry_to_disk(data)` sanitiza nomes de arquivo derivados de `player_id` e `match_id` para prevenir ataques de path traversal — caracteres perigosos sao removidos antes da construcao do path de destino.

-Onboarding (`new_user_flow.py`)

Gerencia a experiencia dos novos usuarios atraves de um processo de onboarding a **3 estagios progressivos** que guia o jogador da instalacao ao coaching completo.

Estagios de Onboarding:

Constante	Valor	Significado	Condicao
<code>AWAITING_FIRST_DEMO</code>	<code>"awaiting_first_demo"</code>	Nenhuma demo carregada	0 demos
<code>BUILDING_BASELINE</code>	<code>"building_baseline"</code>	Demos carregadas mas insuficientes para baseline estavel	1-2 demos
<code>COACH_READY</code>	<code>"coach_ready"</code>	Coaching completo ativo	3+ demos

Classe `UserOnboardingManager` :

- `MIN_INITIAL_DEMOS = 1` — minimo para iniciar o coaching basico
- `RECOMMENDED_DEMOS = 3` — minimo para baseline estatistica estavel
- `_CACHE_TTL_SECONDS = 60` — cache TTL para contagem de demos (evita queries repetidas durante refreshes rapidos da UI)

Metodos:

- `get_status(user_id) -> OnboardingStatus` — retorna o status completo de onboarding com contagem de demos, estagio, mensagens e flags `coach_ready` / `baseline_stable`
- `_count_user_demos(user_id)` — conta demos processadas (non-pro) com cache TTL-based. Filtro: `player_name == user_id AND is_pro == False`
- `invalidate_cache(user_id=None)` — invalida a cache apos upload de nova demo (para usuario especifico ou cache inteira)
- `_determine_stage(count)` — mapeia contagem de demos para estagio
- `_get_stage_message(stage, count)` — mensagem human-readable para cada estagio

Dataclass `OnboardingStatus` : `stage` , `demos_uploaded` , `demos_required` (1), `demos_recommended` (3), `coach_ready` , `baseline_stable` , `message` .

Singleton: `get_onboarding_manager()` — factory module-level (nao thread-safe com double-checked locking — aceitavel para este caso de uso com baixa contencao).

5B. Subsistema 3B — Motores de Coaching

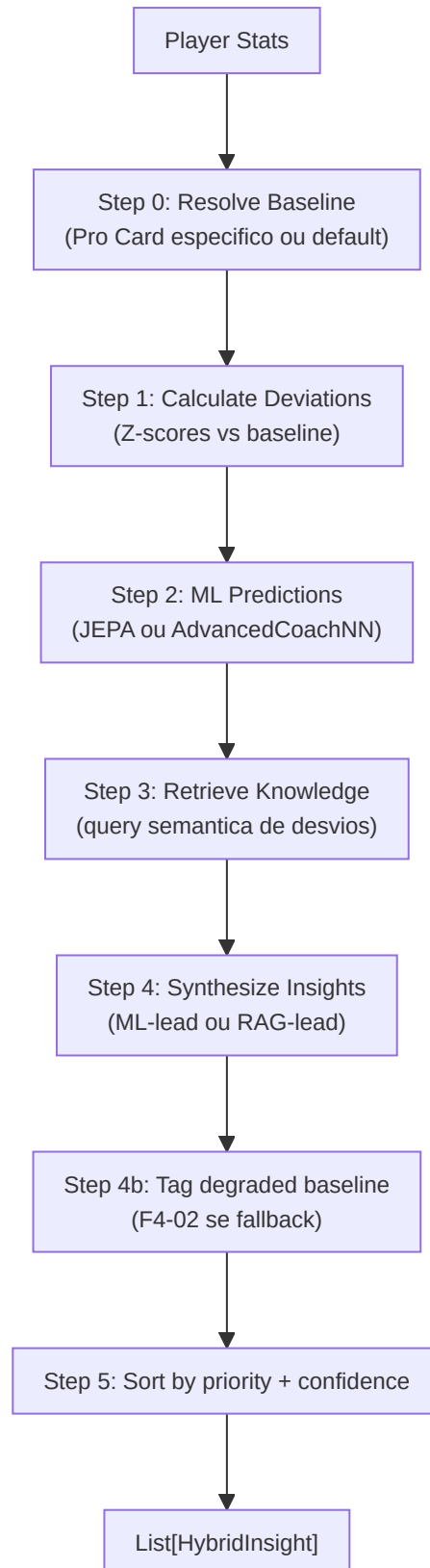
Pasta no repo: `backend/coaching/` **Arquivos:** 8 modulos

Este subsistema contem os **motores decisoriais de coaching** que transformam desvios estatisticos brutos em conselhos priorizados e contextualizados. Diferente dos Servicos (Secao 5) que orquestram e apresentam, os Motores de Coaching contem a **logica de raciocinio** do treinador.

-HybridCoachingEngine (`hybrid_engine.py`)

O **coracao decisional** do coaching: sintetiza predicoes ML com conhecimento RAG para gerar insights unificados e deduplicados.

Pipeline de 5 estagios:



Classes e dataclass:

Tipo	Nome	Descricao
Enum	InsightPriority	CRITICAL (Z >2.5, conf>0.8), HIGH (Z >2.0, conf>0.6), MEDIUM (Z >1.0, conf>0.4), LOW
dataclass	HybridInsight	title, message, priority, confidence, feature, ml_z_score, knowledge_refs, pro_examples, tick_range, demo_name

Formula de confianca:

```
base_confidence = |z_score|/3.0 x 0.6 + knowledge_effectiveness x 0.4
confidence = base_confidence x MetaDriftEngine.get_meta_confidence_adjustment()
```

Onde `knowledge_effectiveness = min(1.0, mean(usage_count) / 100)`.

Estrategia de sintese:

Condicao	Estrategia
Z > 2 (alta confianca ML)	Lead com ML, suporte com RAG
Z < 1 (baixa confianca ML)	Lead com RAG
1 <= Z <= 2	Abordagem balanceada
Nenhum desvio significativo	Insight knowledge-only (prioridade LOW)

TASK 2.7.1 — Reference Clip: Cada `HybridInsight` pode incluir `tick_range: (start, end)` e `demo_name` para permitir a UI pular diretamente para a evidencia no arquivo demo.

Fallback baseline (F4-02): Se `get_pro_baseline()` falha, usa valores estaticos hardcoded e marca todos os insights com `baseline_quality=degraded` na mensagem.

-CorrectionEngine (`correction_engine.py`)

Gera as **top-3 correcoes ponderadas** de desvios Z-score:

Constante	Valor	Uso
<code>CONFIDENCE_ROUNDS_CEILING</code>	300	Scaling confianca: <code>min(1.0, rounds / 300)</code>

Pesos de importancia (DEFAULT_IMPORTANCE):

Feature	Peso
avg_kast	1.5
avg_adr	1.5
accuracy	1.4
impact_rounds	1.3
avg_hs	1.2
econ_rating	1.1
positional_aggression_score	1.0

Pipeline: `deviations` -> aplica confidence scaling x `rounds_played` -> opcional

`apply_nn_refinement()` -> sort por `|weighted_z| x importance` -> retorna top 3.

Override usuário: Os pesos são sobrescrevedores via `get_setting("COACH_WEIGHT_OVERRIDES")`.

-ExplanationGenerator (`explainability.py`)

Traduz sinais latentes RL em **narrativas compreensíveis** organizadas pelos 5 eixos de competência (`SkillAxes`):

Eixo	Template Negativo	Template Ação
MECHANICS	"Your {feature} is {delta}% below professional standards..."	"Focus on crosshair height when clearing corners..."
POSITIONING	"Positioning at {location} was suboptimal..."	"Try holding a tighter angle at {location}..."
UTILITY	"Utility timing with {weapon} was suboptimal..."	"Wait for a clear sound cue before deploying..."
TIMING	"Engagement timing is lagging behind..."	"Coordinate with teammates to trade-frag..."
DECISION	"Decision efficiency is {delta}% lower..."	"In clutch scenarios, prioritize the round objective..."

Princípio "Silence is a Valid Action":

Limiar	Constante	Valor	Comportamento
Silêncio	<code>SILENCE_THRESHOLD</code>	0.2	$ \text{delta} < 0.2$ -> nenhum feedback (silêncio)
Severidade alta	<code>SEVERITY_HIGH_BOUNDARY</code>	1.5	$ \text{delta} > 1.5$ -> "High"
Severidade média	<code>SEVERITY_MEDIUM_BOUNDARY</code>	0.8	$ \text{delta} > 0.8$ -> "Medium"

Filtro de complexidade: Para `skill_level < 3` (iniciantes), as narrativas negativas são simplificadas a apenas a ação sugerida, evitando sobrecarga cognitiva.

-NNRefinement (`nn_refinement.py`)

Aplica ajustes de peso da rede neural as correcoes pre-calculadas:

```
refined_z = weighted_z x (1 + nn_adjustments["{feature}_weight"])
```

Modulo leve que escala as correcoes do `CorrectionEngine` usando pesos aprendidos do modelo ML, permitindo ao sistema neural influenciar a priorizacao dos conselhos.

-ProBridge (`pro_bridge.py`)

Layer de traducao que assimila Player Card HLTV no modelo cognitivo do coach:

Classe	Responsabilidade
<code>PlayerCardAssimilator</code>	Converte <code>ProPlayerStatCard</code> -> baseline coach-compatible
<code>get_pro_baseline_for_coach()</code>	Factory function direto

Constante legacy: `ESTIMATED_ROUNDS_PER_MATCH = 24.0` — presente no codigo mas **nao mais utilizada** para a conversao KPR/DPR apos a correcao P3-02.

Mapeamento metricas:

Metrica HLTV	Metrica Coach	Transformacao
<code>card.kpr</code>	<code>avg_kills</code>	direto (P3-02: NAO multiplicado por 24)
<code>card.dpr</code>	<code>avg_deaths</code>	direto (P3-02: NAO multiplicado por 24)
<code>card.adr</code>	<code>avg_adr</code>	direto
<code>card.kast</code>	<code>avg_kast</code>	V-2: normalizacao defensiva (<code>/100</code> se <code>kast > 1.0</code> , senao direto)
<code>card.impact</code>	<code>impact_rounds</code>	direto
<code>card.rating_2_0</code>	<code>rating</code>	direto
<code>detailed_stats.headshot_pct</code>	<code>avg_hs</code>	V-2: normalizacao defensiva (<code>/100</code> se <code>> 1.0</code> , default 0.45)
<code>detailed_stats.total_opening_kills</code>	<code>entry_rate</code>	<code>/100</code> (heuristica)
<code>detailed_stats.utility_damage_per_round</code>	<code>utility_damage</code>	direto (default 45.0)

Correcao P3-02 — Escala KPR/DPR: O codigo anterior multiplicava `kpr x 24` e `dpr x 24`, produzindo kills/deaths totais por partida (valores 15-20) ao inves de valores por-round (0.6-0.8). Isto tornava todos os z-scores de comparacao invalidos pois as estatisticas do usuario extraidas de `base_features.py` e `pro_baseline.py` ja estao em escala por-round. Apos a remediacao, `get_coach_baseline()` usa diretamente as taxas por-round.

Correcao V-2 — Normalizacao defensiva legacy: Os registros HLTV mais antigos no banco de dados poderiam conter valores percentuais (ex: `kast=72.0` ao inves de `kast=0.72`). A normalizacao condicional (`/100 se > 1.0`) gerencia ambos os formatos de forma transparente.

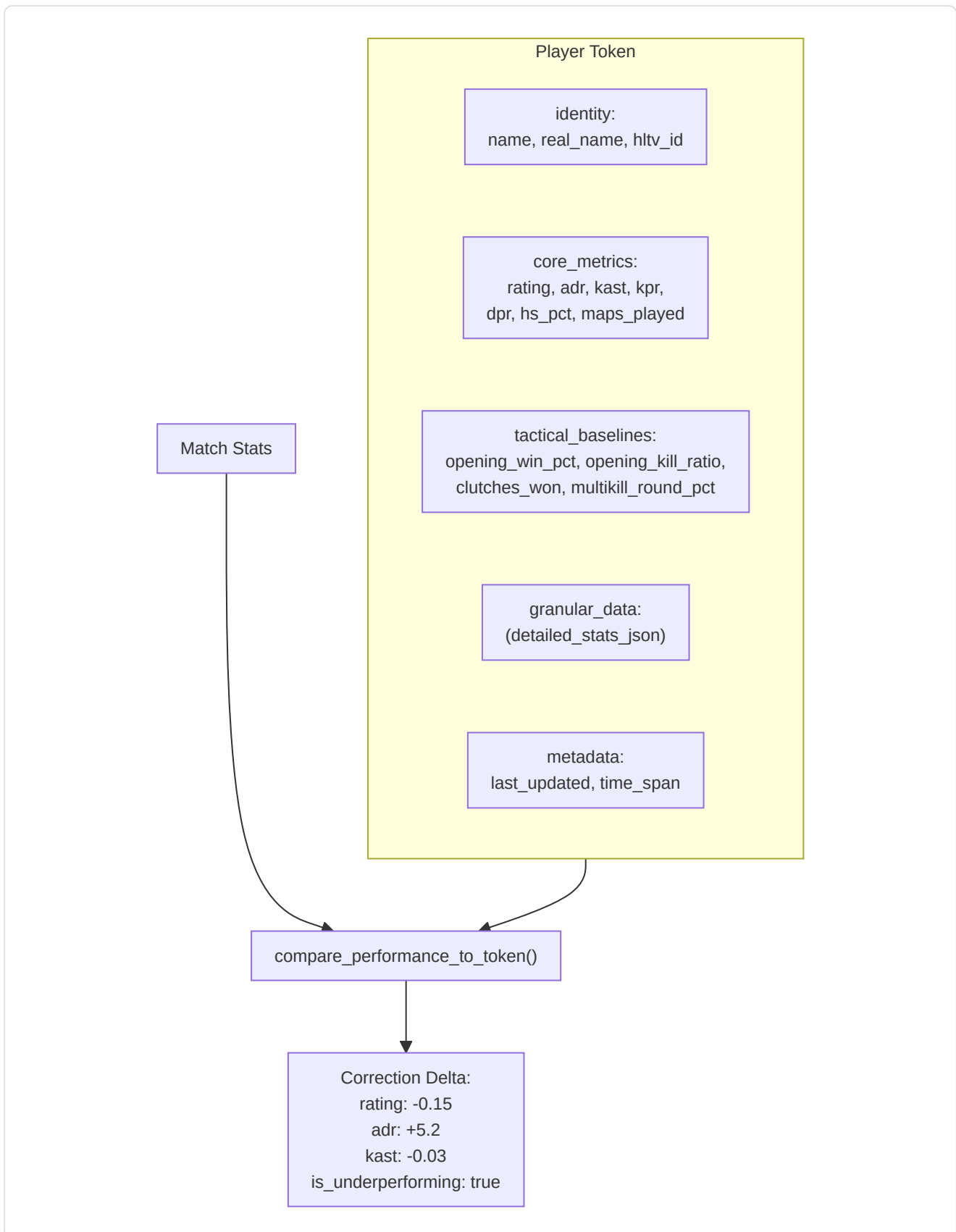
Classificacao arquétipo: `get_player_archetype()` -> Star Fragger (impact>1.3), Support Anchor (kast>0.75), Sniper Specialist (AWP kills>40%), All-Rounder (default).

-PlayerTokenResolver (`token_resolver.py`)

Resolve **tokens estaticos** (Card) para a comparacao dinamica:

- `get_player_token(player_name)` -> token dict com identity, core_metrics, tactical_baselines, granular_data, metadata
- `compare_performance_to_token(match_stats, token)` -> `Correction Delta` com deltas por rating, adr, kast, accuracy_vs_hs e flag `is_underperforming` (rating < 85% do pro)

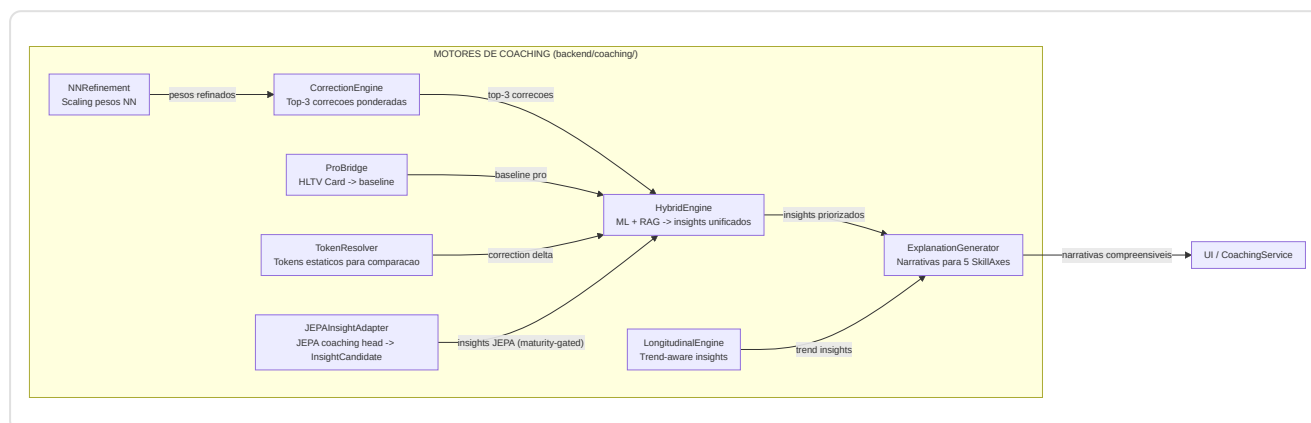
Estrutura Token:



-LongitudinalEngine (`longitudinal_engine.py`)

Gera insights trend-aware comparando trajetorias de desempenho no tempo com sinais de estabilidade NN:

- **Input:** `List[FeatureTrend]` (do modulo progress) + `nn_signals` (do training pipeline)
- **Filtro confianca:** Apenas trends com `confidence >= 0.6`
- **Output:** Max 3 insights, categorizados como:
 - **Regression:** slope < 0 -> severidade "Medium" (ou "High" se `nn_signals.stability_warning` ativo)
 - **Improvement:** slope > 0 -> severidade "Positive", focus "Reinforcement"



-JEPALnsightAdapter (`jepa_insight_adapter.py`)

Converte os vetores de saída brutos da **Coaching Head JEPA** em objetos `InsightCandidate` prontos para a pipeline de coaching (F1.1 / W5.1).

Contrato dimensional: Opera nos primeiros 10 elementos do contrato 25-dim (target features: health, armor, has_helmet, has_defuser, equipment_value, is_crouching, is_scoped, is_blinded, enemies_visible, pos_x) mapeados em cinco SkillAxes (`survival` , `economy` , `movement` , `utility` , `positioning`).

Escala de maturidade:

Estado	Multiplicador confianca	Max candidatos	Modo
<code>doubt</code> / <code>crisis</code>	0,5x	1	Hedged: "Observation: ..."
<code>learning</code>	0,8x	2	Direto
<code>conviction</code> / <code>mature</code>	1,0x	3	Direto

Flag de ativacao: `USE_JEPA_MODEL` (padrao `False`). Quando desligada, `generate_jepa_insights()` retorna lista vazia — nenhuma alteracao no comportamento existente (F1.2 byte-identical). Respeita a restricao **NO-WALLHACK** (F1.3): consome exclusivamente a janela de ticks do jogador observado (contrato POV 25-dim, P-X-01).

6. Subsistema 4 — Conhecimento e Recuperacao

pasta no repo: `backend/knowledge/` **Arquivos:** `rag_knowledge.py` , `experience_bank.py`

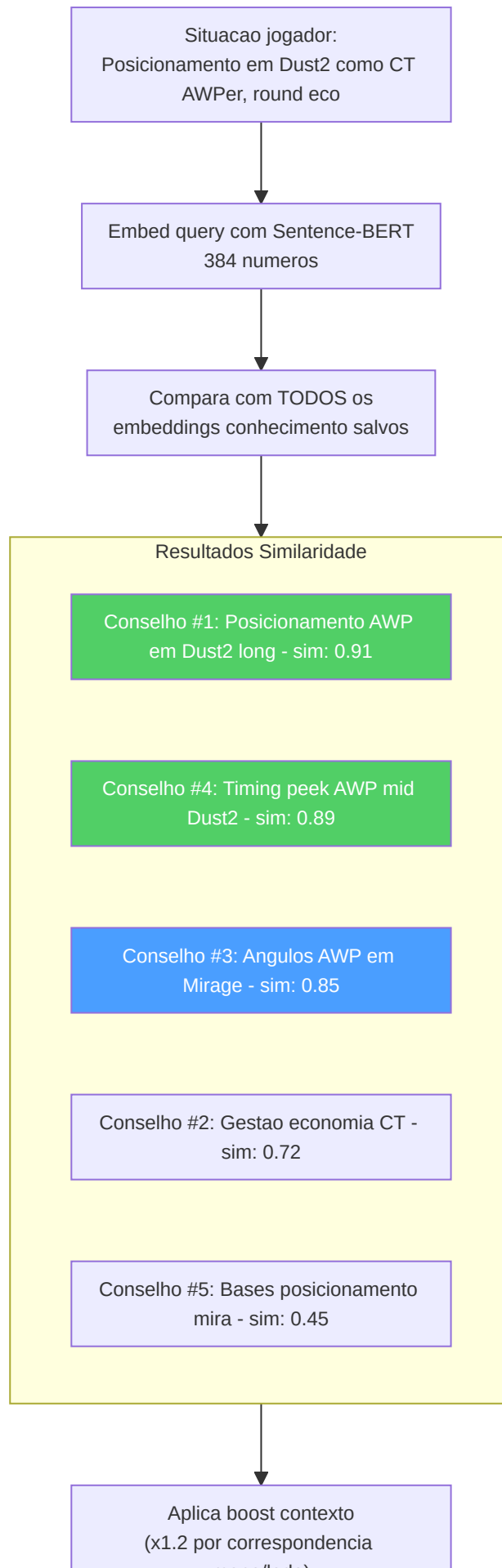
Este subsistema gerencia dois tipos de dados complementares: os **conhecimentos taticos** (estrategias, posicionamentos, uso de utility documentados e indexados para busca semantica) e as **experiencias de coaching** (registros de cada sessao de coaching com resultado, eficacia e contexto da partida, utilizados para aprendizado baseado em casos).

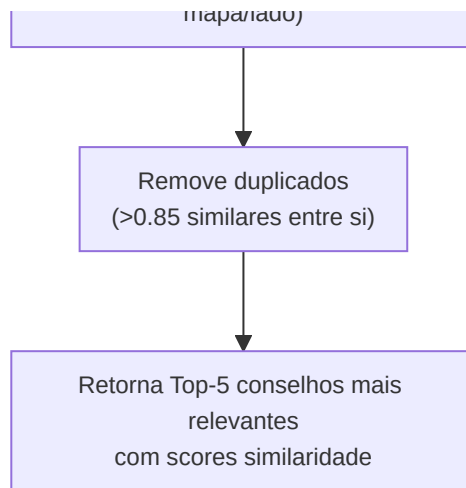
-Knowledge Base RAG (`rag_knowledge.py`)

Implementa uma pipeline de **retrieval augmented generation** usando a busca por similaridade vetorial densa:

Componente	Detalhe
Modelo de embedding	<code>sentence-transformers/all-MiniLM-L6-v2</code> (vetores de 384 dimensoes)
Fallback	Embeddings baseados em hash se Sentence-BERT nao esta disponivel
Armazenamento	Tabela SQLite <code>TacticalKnowledge</code> (embedding memorizado como array float codificado em JSON)
Retrieval	Similaridade do cosseno via <code>scipy.spatial.distance.cosine</code>
Top-k	Configuravel, padrao k=5
Versioning	<code>CURRENT_VERSION = "v3"</code> (2026-04, Coach Book refactor, Premier S4 active duty alignment); embeddings obsoletos v2 recalculados automaticamente via <code>trigger_reembedding()</code>
Categorias	14: objetivo, posicionamento, utility, movimento, economia, estrategia, posicionamento da mira, comunicacao, mental, senso do jogo, trading, <code>mid_round</code> , <code>retakes_post_plant</code> , <code>aim_and_duels</code>

Correcao M-07 — Rejeicao vetor norma-zero: `VectorIndex.search()` valida a norma do vetor query antes da busca. Se a norma e zero (tipicamente devido a um embedding fallback vazio ou a um input corrompido), o metodo retorna `None` com um warning no log ao inves de propagar um erro de divisao por zero na similaridade do cosseno. Isto protege o pipeline RAG de queries degeneradas sem interromper o fluxo de coaching.

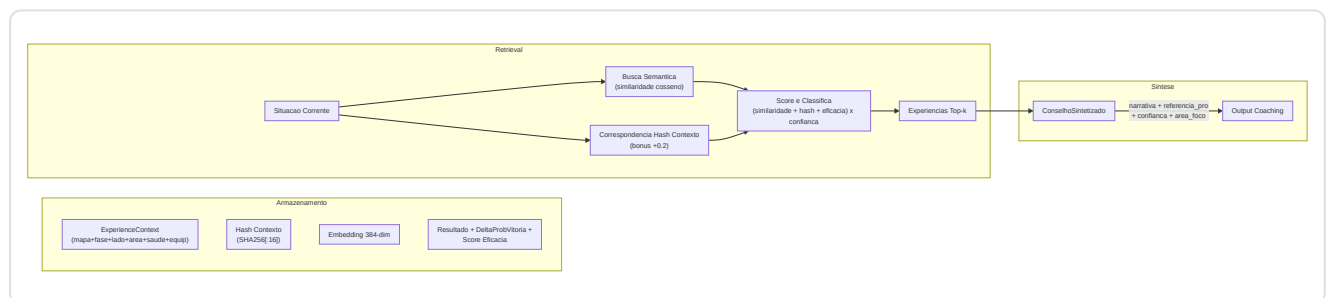




Construcao query: queries dinamicas em linguagem natural a partir de estatisticas jogador, mapa, lado, papel. Os elementos que correspondem ao contexto obtm um multiplicador de relevancia 1,2x. A deduplicacao filtra os elementos com uma similaridade >0,85 em relacao aos resultados ja seleccionados.

-Banca Experiencia (`experience_bank.py`) — Framework COPER (KT-01 Enhanced)

Implementa o framework **Observacao-Previsao-Experiencia-Recuperacao Contextual (COPER)** com semantica CRUD, replay priorizado e integracao TrueSkill:

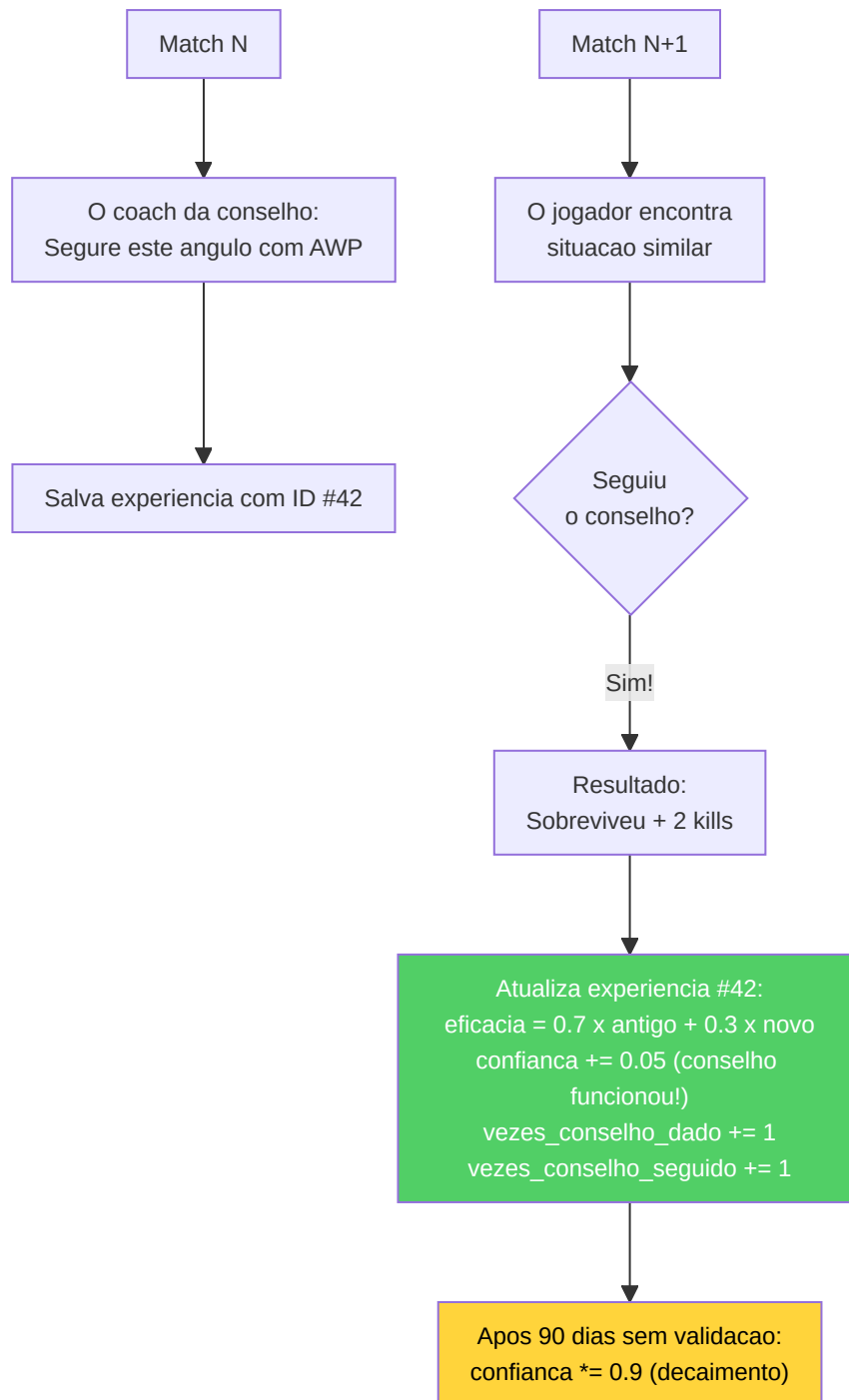


Estrategia de duplo retrieval:

1. **Experiencias usuario:** Situacoes passadas tiradas do historico de jogo do usuario.
2. **Experiencias profissionais:** Como os profissionais gerenciaram situacoes analogas.
3. **Analise dos padroes:** Identifica fraquezas recorrentes, tendencias de melhoria, correlacoes contextuais.

Ciclo de feedback (baseado em EMA):

- Cada experiencia rastreia `outcome_validated`, `effectiveness_score`, `times_advice_given`, `times_advice_followed`
- As correspondencias de follow-up atualizam a eficacia: $\text{new_score} = 0,7 \times \text{old_score} + 0,3 \times \text{outcome_value}$
- Experiencias obsoletas (>90 dias sem validacao): a confianca diminui em 10%
- O monitoramento do uso incrementa `usage_count` a cada retrieval

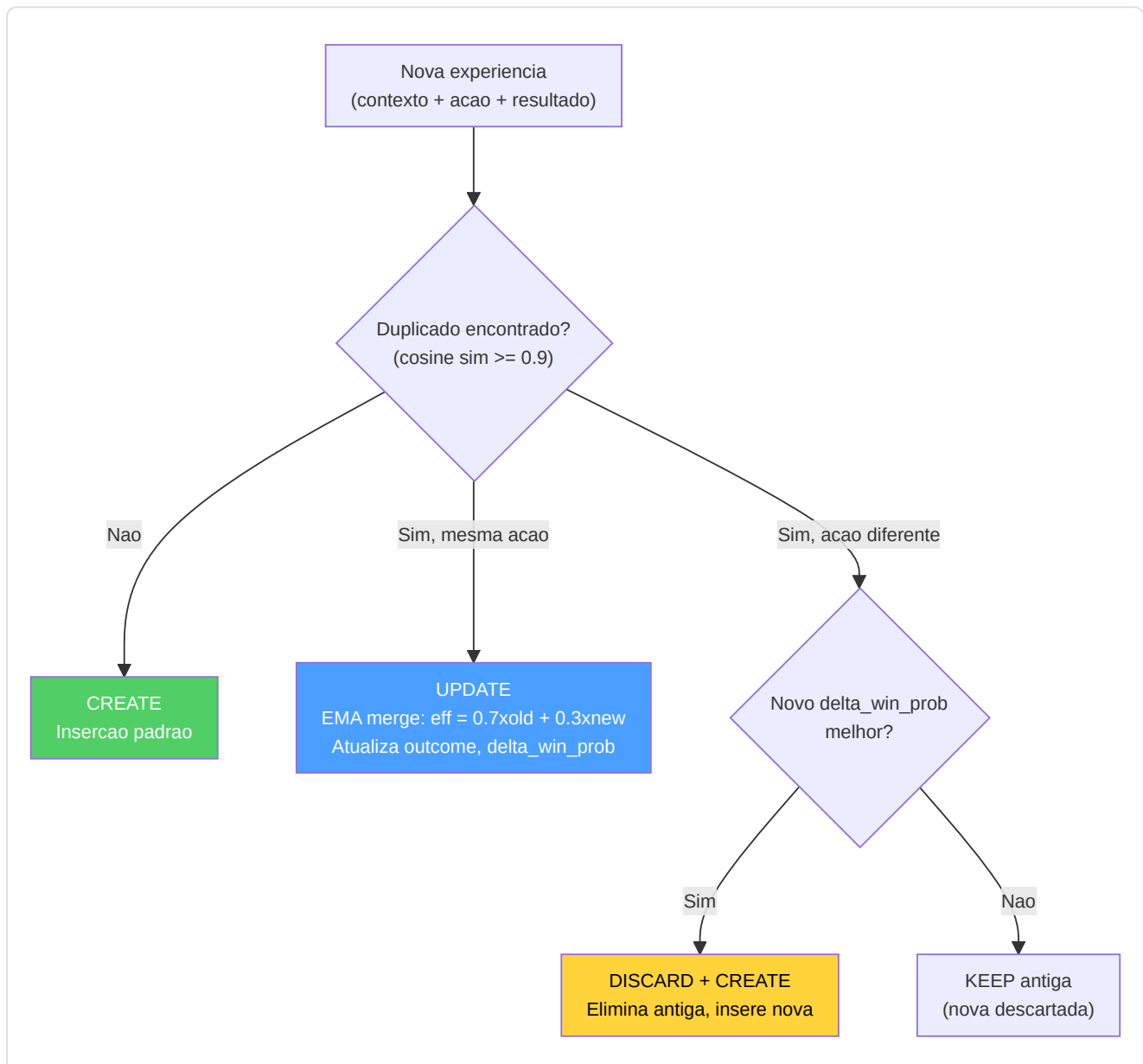


Extracao da experiencia das demos: Agrupa os eventos por tick, identifica as kills/mortes dos jogadores, cria o contexto a partir de uma snapshot do tick, infere a acao (scoped_hold, crouch_peek, pushed, held_angle), determina o resultado.

Melhorias KT-01 — Semantica CRUD e Replay Priorizado:

Constante	Valor	Proposito
<code>DUPLICATE_SIMILARITY_THRESHOLD</code>	0.9	Similaridade cosseno para deteccao duplicados
<code>CRUD_EMA_FACTOR</code>	0.3	Peso EMA para merge effectiveness em UPDATE
<code>REPLAY_ALPHA</code>	0.6	Expoente prioridade replay (mais baixo = mais uniforme)
<code>REPLAY_GATE</code>	0.4	Confianca minima para ser elegivel ao replay
<code>_MIN_EFFECTIVENESS_TRIALS</code>	5	Trials minimos antes que effectiveness influencie o retrieval

Decisao CRUD no momento da insercao:



Replay Priorizado (KT-01): As experiencias sao amostradas para o replay com probabilidade proporcional a $priority^{REPLAY_ALPHA}$, onde $priority = effectiveness_score \times confidence_score$. Apenas experiencias com $confidence_score \geq REPLAY_GATE$ sao elegiveis. Isto balanca exploitation (experiencias eficazes) com exploration (experiencias menos testadas).

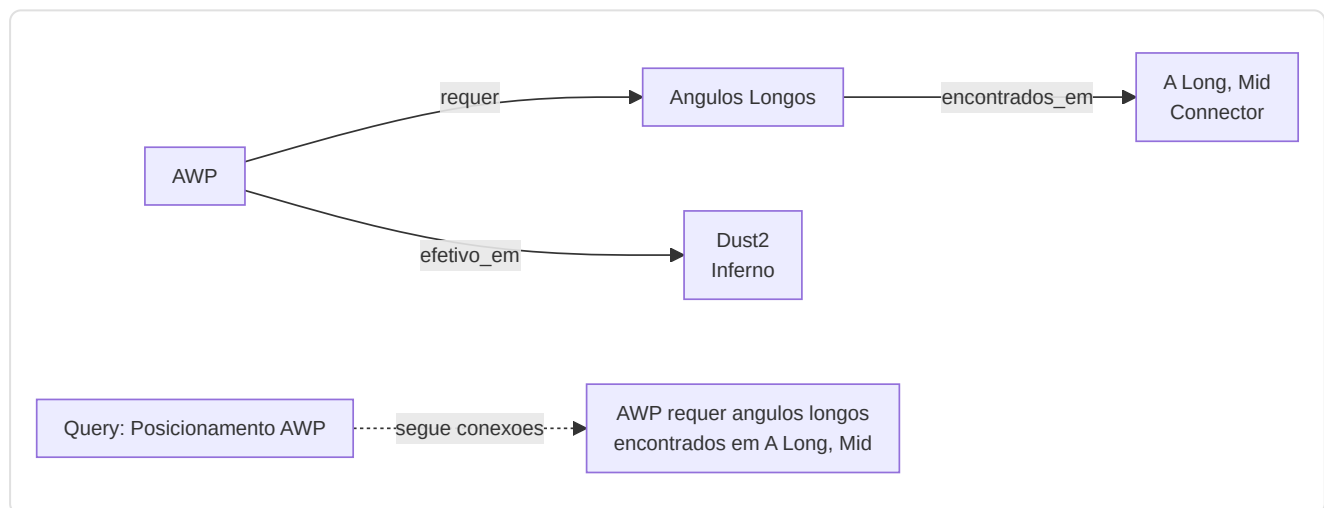
Integracao TrueSkill (KT-01): Campos `mu_skill` e `sigma_skill` para tracking bayesiano da competencia do jogador na situacao especifica. Os priors TrueSkill influenciam o peso da experiencia no retrieval: experiencias com alta incerteza (`sigma` alto) sao penalizadas em relacao aquelas com sinal estavel.

Compressao Embedding: Os embeddings 384-dim agora sao codificados como `base64(float32)` ao inves de JSON array, obtendo uma compressao 4x no espaco de armazenamento no banco de dados sem perda de precisao.

Linking Referencia Pro: Cada experiencia pode incluir `pro_player_name` , `pro_match_id` , `source_demo` para conectar diretamente a como um profissional especifico gerenciou uma situacao analoga.

-Knowledge Graph

Um **grafo entidade-relacao** leve memorizado em SQLite (tabelas `kg_entities` , `kg_relations`). Suporta `query_subgraph(entity_name)` a 1 hop para o raciocinio multi-hop, a fim de integrar a similaridade semantica.



-Inicializacao Knowledge Base (`init_knowledge_base.py`)

Script de orquestracao que popula o banco de dados RAG com conhecimento tatico de duas fontes:

- Conhecimento manual:** Carregamento de arquivo JSON (`data/tactical_knowledge.json`) com sugestoes curadas manualmente por categoria e mapa
- Mining automatico:** Invocacao de `ProDemoMiner.mine_all_pro_demos()` para extrair padroes taticos das demos profissionais

Apos o carregamento, gera um relatorio por categoria e mapa usando queries COUNT agregadas (sem carregar todos os records em memoria).

-ProDemoMiner (`pro_demo_miner.py`)

Extrai conhecimento tatico das demos profissionais usando pattern detection:

Classe	Linhas	Descrição
<code>ProDemoMiner</code>	~180	Extração padrões de metadados match (map, team, success rate)
<code>AdvancedProDemoMiner(ABC)</code>	~60	Base abstrata (F5-05) para parsing demo com demoparser2

Limites de qualidade:

Constante	Valor	Significado
<code>MIN_SUCCESS_RATE</code>	0.70	Padrão aceito apenas se sucesso $\geq 70\%$
<code>MIN_SAMPLE_SIZE</code>	5	Mínimo 5 ocorrências para padrão válido

Mapas suportados: mirage, dust2, inferno, nuke, overpass, vertigo, ancient, anubis.

Tipologias de conhecimento gerado:

- Conhecimento específico por mapa (posicionamento, utility, rotacões)
- Conhecimento específico por team (estratégias, tendências, estilos)
- Conhecimento por execuções de sucesso (padrões com taxa $\geq 70\%$)

-Round Utils (`round_utils.py`)

Utility compartilhada para classificação fase econômica do round, extraída para eliminar duplicação entre os layers knowledge e services:

Valor Equipamento	Fase	Constante
< \$1.500	<code>pistol</code>	<code>_PISTOL_MAX_EQUIP</code>
\$1.500 - \$2.999	<code>eco</code>	<code>_ECO_MAX_EQUIP</code>
\$3.000 - \$3.999	<code>force</code>	<code>_FORCE_MAX_EQUIP</code>
$\geq$ \$4.000	<code>full_buy</code>	—

7. Subsistema 5 — Motores de análise

pasta no repo: `backend/analysis/` **11 arquivos, ~2.600 linhas de código de produção**

Este subsistema contém **11 motores de análise especializados**, cada um projetado para investigar uma dimensão diferente do gameplay. Funcionam como análise de Fase 6, fornecendo aprofundamentos que vão além do que só as redes neurais podem oferecer.

OS 11 MOTORES DE ANÁLISE - SEU TIME DE ESPECIALISTAS

1. Classificador Papeis - Que posição você joga?

2. Probabilidade Vitória - Quais são as probabilidades agora?

3. Árvore de Jogo - Qual é a melhor jogada?

4. Morte Bayesiana - Quão provável é que você morra?

5. Índice Deception - Quão imprevisível você é?

6. Tracker Momentum - Você está em forma ou em tilt?

7. Analisador Entropia - Suas granadas são eficazes?

8. Detector Blind Spots - Que erro você repete?

9. Utility e Economia - Otimizador compras

10. Analisador Engajamento - A que distancia voce combate melhor?

11. Qualidade Movimento - Voce faz erros de posicionamento?

-Classificador de Papeis (`role_classifier.py`)

Atribui um dos 6 papeis usando **limiars estatisticos aprendidos**:

Papel	Sinal Primario	Sinal Secundario
AWPer	Ratio kills AWP vs limiar	—
Entry Fragger	Taxa de entry + bonus a primeira morte (0,3x)	—
Support	Taxa de assistencia + bonus dano de utility (max 0,3x)	—
IGL	Taxa de sobrevivencia + bonus balanceamento KD	—
Lurker	Ratio kills em solitario vs limiar	—
Flex	Fallback quando a confianca e baixa	—

Protecao para cold-start: `RoleThresholdStore` requer ≥ 10 amostras e ≥ 3 limiars validos para sair do cold-start. Retorna (`FLEX, 0.0`) se em cold-start. Os limiars sao **mantidos no banco de dados** via `persist_to_db()` e `load_from_db()` — completamente implementados, nao como stub.

Audit do balanceamento do team (`audit_team_balance()`): detecta multiplos AWPers (ALTA), Entry faltando (ALTA), Support faltando (MEDIA), nenhuma diversidade (CRITICA), multiplos Lurkers (MEDIA).

-Preditor de Probabilidade de Vitoria (`win_probability.py`)

Rede neural de 12 funcoes que estima $P(\text{round_win} \mid \text{game_state})$:

Arquitetura: `Linear(12, 64) -> ReLU -> Dropout(0,2) -> Linear(64, 32) -> ReLU -> Dropout(0,1) -> Linear(32, 1) -> Sigmoid .`

12 Features:

#	Feature	Normalizacao
1	economia_team	/16000
2	economia_inimigo	/16000
3	diferencial economia	(equipe-inimigo)/16000
4	jogadores_vivos	/5
5	inimigos_vivos	/5
6	diferencial contagem jogadores	(vivos-inimigo)/5
7	utility_restante	/5
8	percentual_controle_mapa	[0, 1]
9	tempo_restante	/115
10	bomba_plantada	binario
11	is_ct	binario
12	ratio valor equipamento	$\min(\text{equipe}/\text{inimigo}, 2)/2$

Overrides heurísticos: 3+ vantagem -> limite mínimo em 85%, 3+ desvantagem -> limite máximo em 15%, 0 vivos -> 0%, ajustes bomba plantada (T: +0,10, CT: -0,10) — aditivos sobre a probabilidade base, limites econômicos de $\pm 8000\$$.

Nota A-12 — Guard cross-load: Este preditor de 12 features (`WinProbabilityNN`) é um modelo separado e incompatível em relação ao `WinProbabilityTrainerNN` de 9 features descrito na Seção 12. Os checkpoints não são intercambiáveis: ao carregar e validar a dimensionalidade do `state_dict` e, em caso de mismatch, o modelo é reinicializado do zero com um warning no log.

Preditor Aumentado com Elo (KT-07):

O `EloAugmentedPredictor` envolve o `WinProbabilityNN` base com um sistema Elo opcional para aproveitar o histórico das partidas:

Constante	Valor	Descrição
<code>_ELO_INITIAL</code>	1500.0	Elo inicial para jogadores sem histórico
<code>_ELO_K_FACTOR</code>	32.0	Fator K base para a magnitude das atualizações
<code>_ELO_RECENCY_HALF_LIFE</code>	20 partidas	O peso de uma partida diminui pela metade a cada 20 partidas

Fórmula de atualização Elo com peso de recência:

```
new_elo = old_elo + K x w x (S - E)
```

onde:

S = score efetivo (1 para vitoria, 0 para derrota)

E = score esperado = $1 / (1 + 10^{((\text{opp_elo} - \text{elo}) / 400)})$

w = peso recencia = $2^{((\text{match_index} - N + 1) / \text{half_life})}$

O peso de recencia (adaptado de Glickman, 1999) garante que as partidas recentes contribuam mais ao rating final. Uma partida de 20 matches atras contribui metade do K-factor em relacao a ultima partida.

Blending NN + Elo:

```
final_prob = (1 - alpha) x nn_prob + alpha x elo_prob    (alpha = 0.15 default)
```

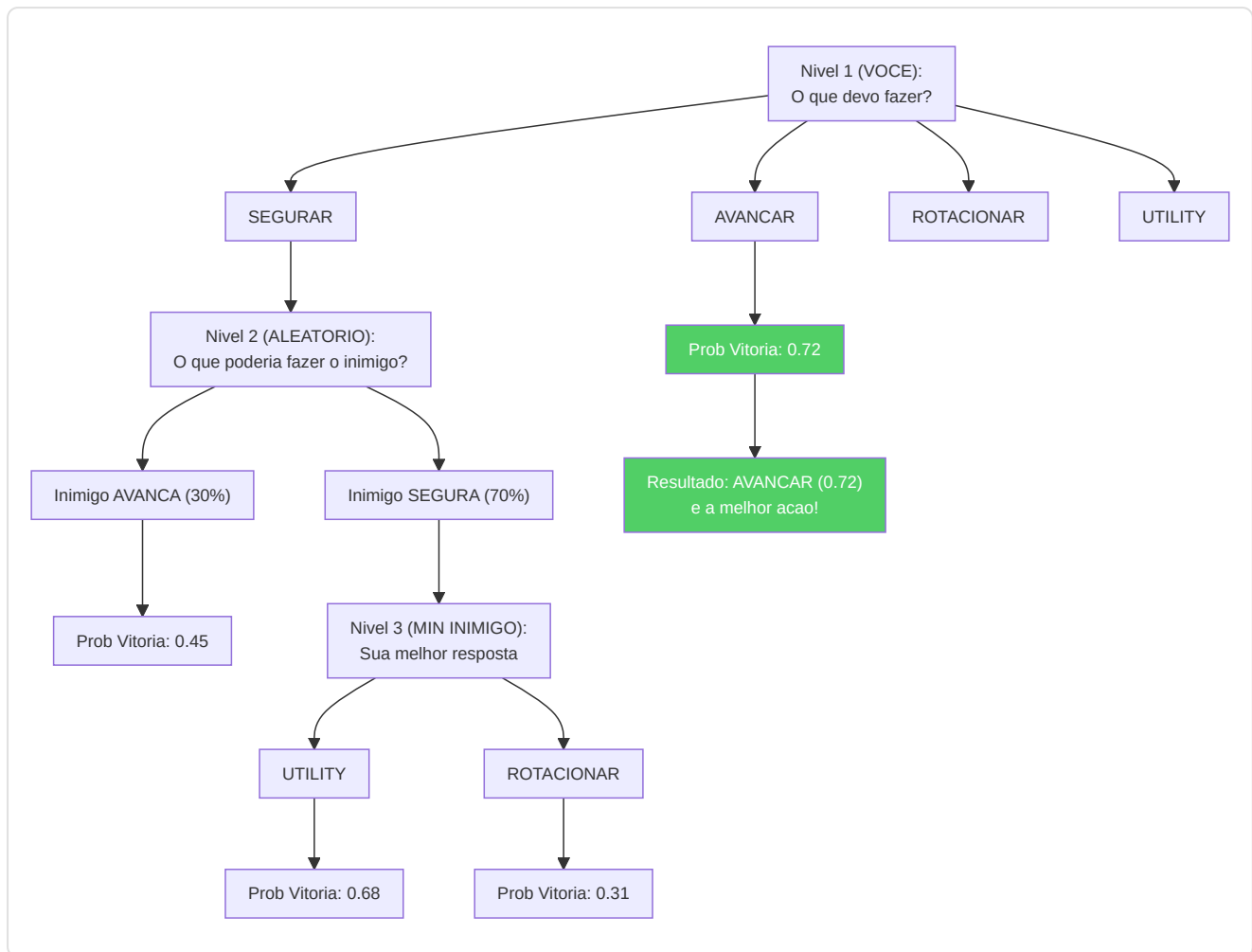
O `compute_elo_differential(team_histories, enemy_histories)` calcula o diferencial Elo medio entre as duas equipes, normalizado por 400 (uma "classe" Elo), e o converte em probabilidade atraves da formula logistica standard. O blending e conservativo (alpha = 0.15) porque o Elo captura apenas informacao historica, enquanto a NN ve o estado corrente do round.

Nota: O Elo e uma **augmentacao opcional** — a arquitetura base de 12 features do `WinProbabilityNN` permanece inalterada. Se o historico nao esta disponivel, o preditor volta a probabilidade NN pura.

-Arvore de jogo Expectiminimax (`game_tree.py`)

Implementa a **busca expectiminimax** com modelagem adaptiva do adversario:

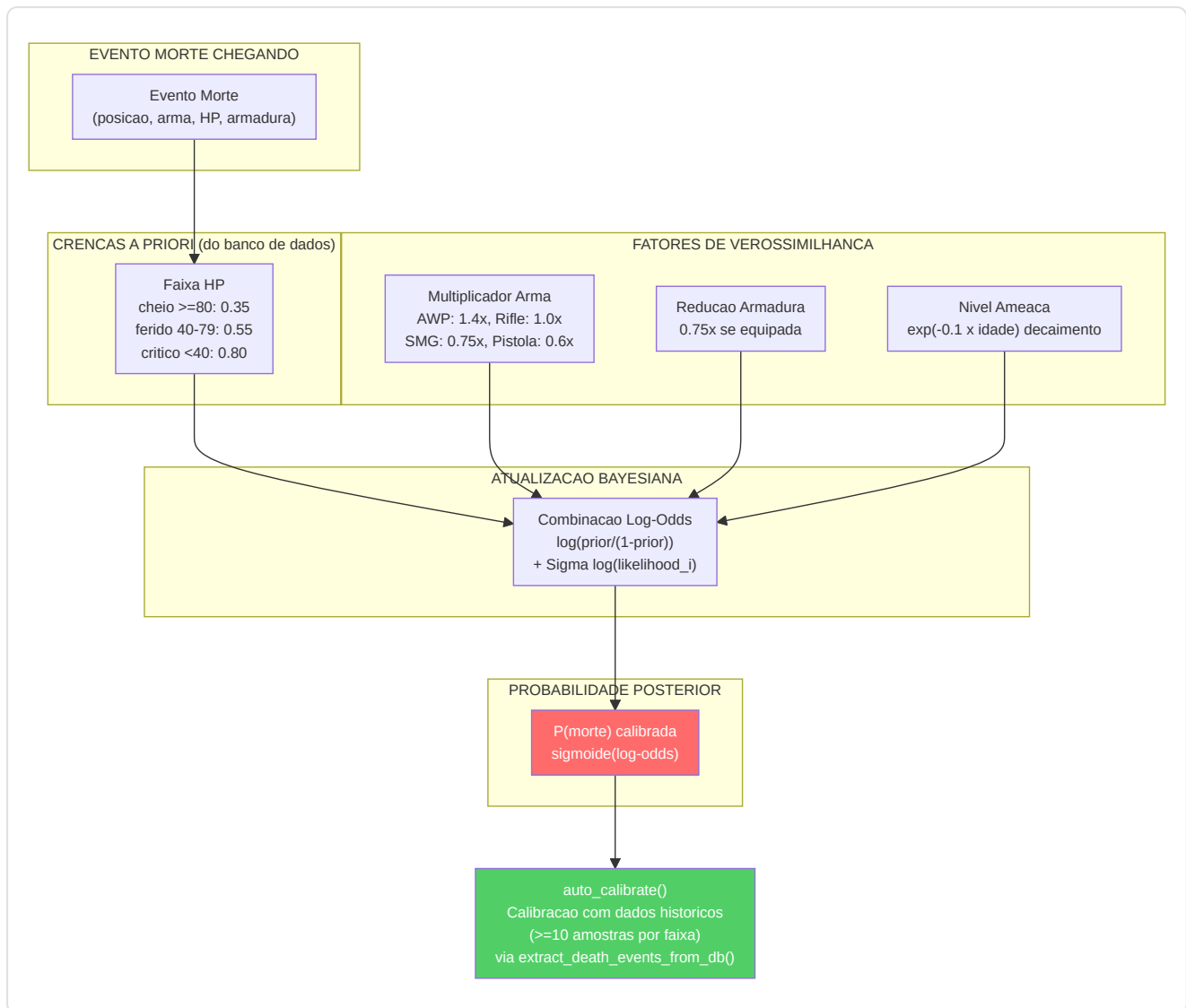
- **Acoes:** avancar, segurar, rotacionar, usar_utility
- **Modelo adversario:** Prioridades economicas (eco/force/full buy), ajustes laterais, ajustes da vantagem, pressao temporal
- **Profundidade:** 3 niveis (max -> probabilidade -> min)
- **Budget no:** 1000 (impede a explosao)
- **Avaliacao folha:** `WinProbabilityPredictor` (carregamento lazy)
- **Aprendizado adversario:** Atualizacao EMA incremental (alpha limitado a 0,5)



-Estimador Bayesiano de Morte (`belief_model.py`)

Modela $P(\text{morte} \mid \text{crenca}, \text{HP}, \text{armadura}, \text{classe_arma})$:

- **Antecedente:** Taxas de mortalidade por faixa HP (cheio ≥ 80 : 0,35, danificado 40-79: 0,55, critico < 40 : 0,80)
- **Fatores de verossimilhanca:** Nivel de ameaca (com decaimento exponencial $\exp(-0,1 \times \text{idade})$), reducao da armadura ($0,75x$), multiplicadores das armas (AWP: $1,4x$, Rifle: $1,0x$, Metralhadora: $0,75x$, Pistola: $0,6x$, Faca: $0,3x$)
- **Posterior:** Combinacao logistica no espaco log-odds
- **Calibracao:** `calibrate(historical_rounds)` aprende as prioridades empiricas (≥ 10 amostras por faixa)



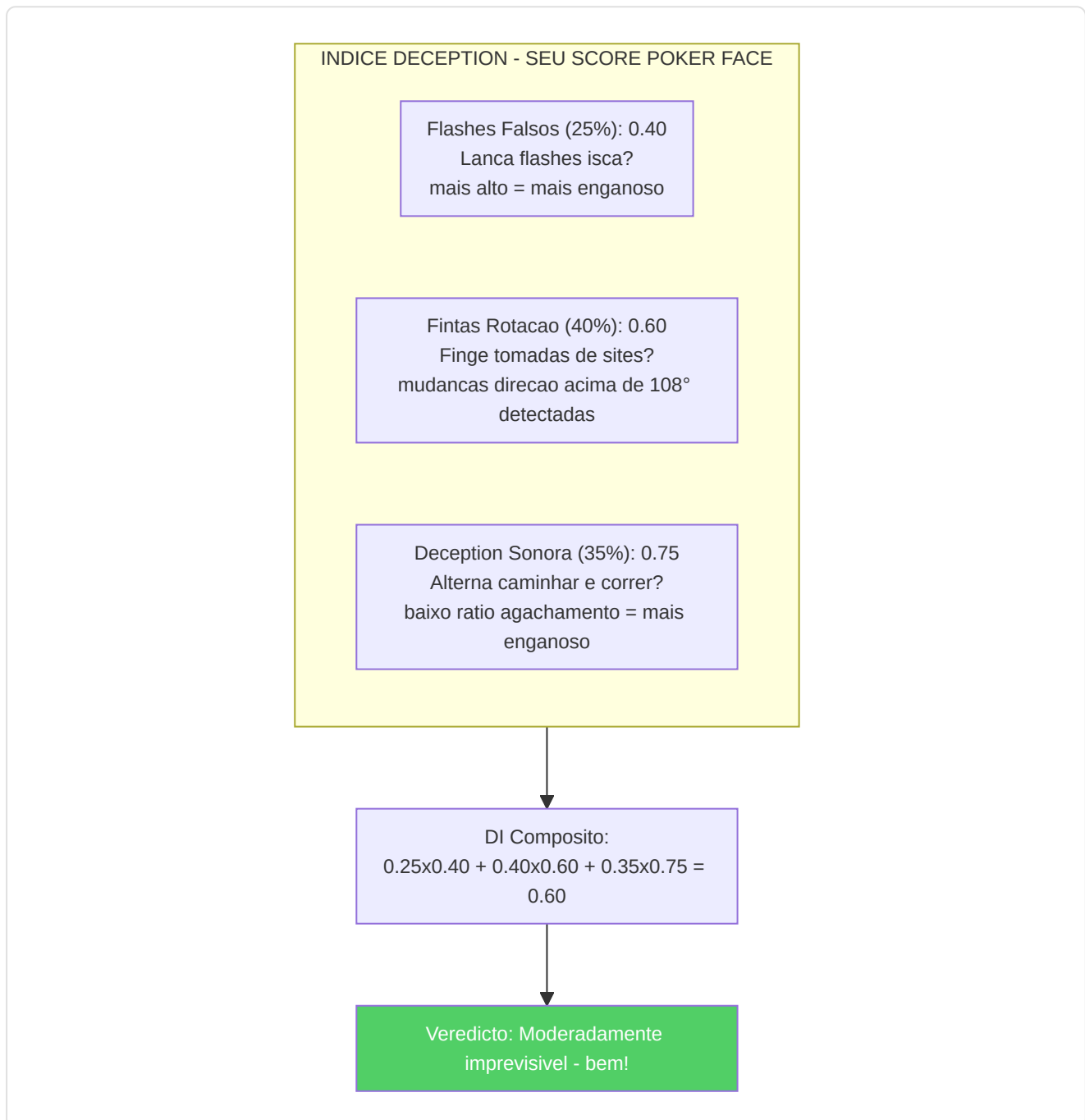
Nota sobre a calibracao (G-07): O Estimador Bayesiano de Morte e agora um **componente live** gracias ao wiring completado durante a remediacao. A funcao `extract_death_events_from_db()` no Session Engine extrai automaticamente os eventos de morte do banco de dados e os passa a `auto_calibrate()`, permitindo ao modelo refinar suas probabilidades a priori com base nos dados reais acumulados. Este ciclo de feedback transforma o estimador de um modelo estatico a um sistema que se auto-calibra com a experiencia.

-Indice de Deception (`deception_index.py`)

Quantifica a deception tatica atraves de tres sub-metricas:

Sub-metrica	Peso	Metodo de deteccao
Frequencia de falsos flash	0,25	Flashes que nao cegam inimigos — $\text{bait_rate} = 1 - \text{efetivo}/\text{total}$
Frequencia de finta rotacao	0,40	Mudancas de direcao >108° detectadas atraves de amostragem da velocidade angular (20 intervalos de posicao)
Score de deception sonora	0,35	Inverso do ratio de agachamento — $1,0 - \text{ratio_agachamento} \times 2,0$

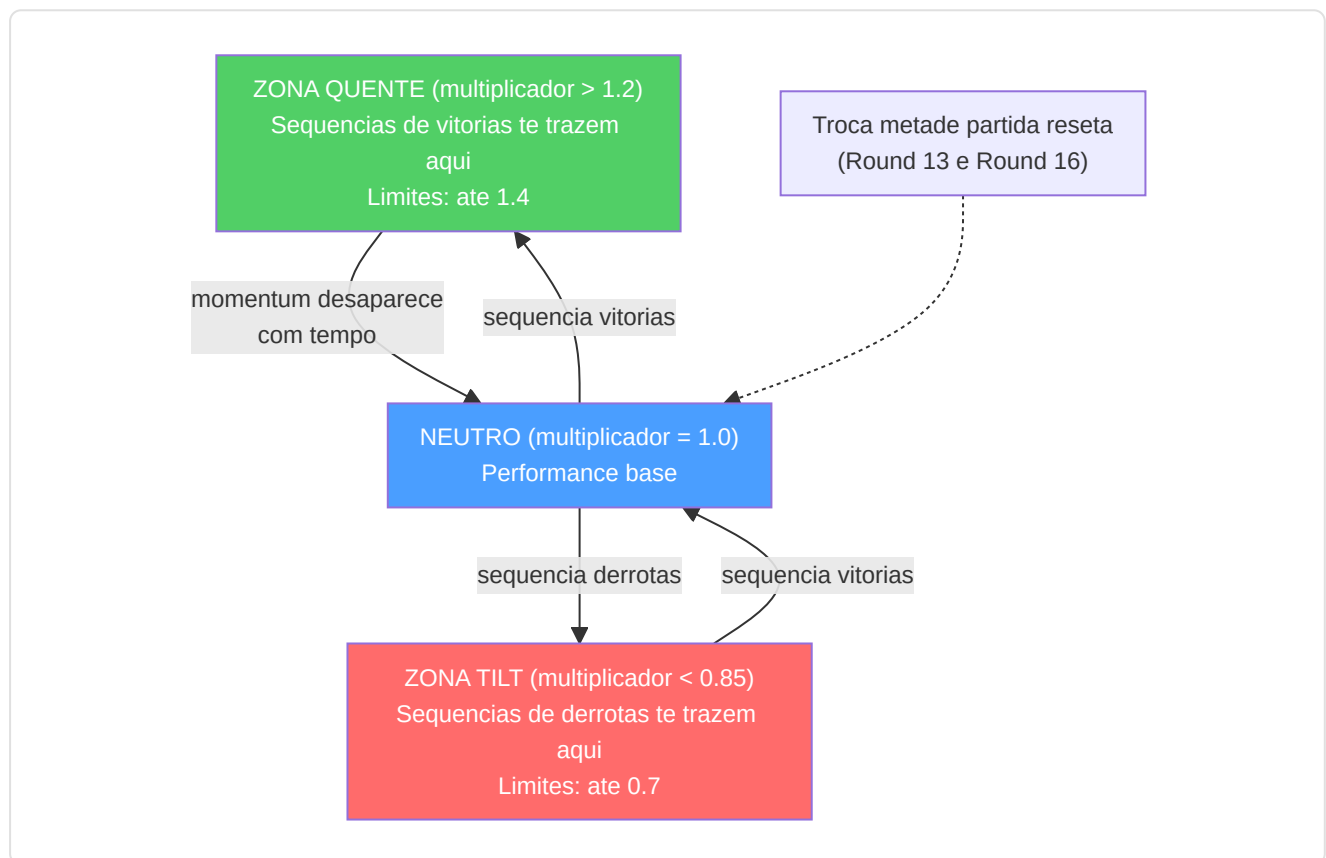
Composito: $\text{DI} = 0,25 \cdot \text{fake_flash} + 0,40 \cdot \text{rotation_feint} + 0,35 \cdot \text{sound_deception}$, fixado em [0, 1].



- Momentum Tracker (`momentum.py`)

Modela o momentum psicologico como um multiplicador de desempenho que decresce no tempo:

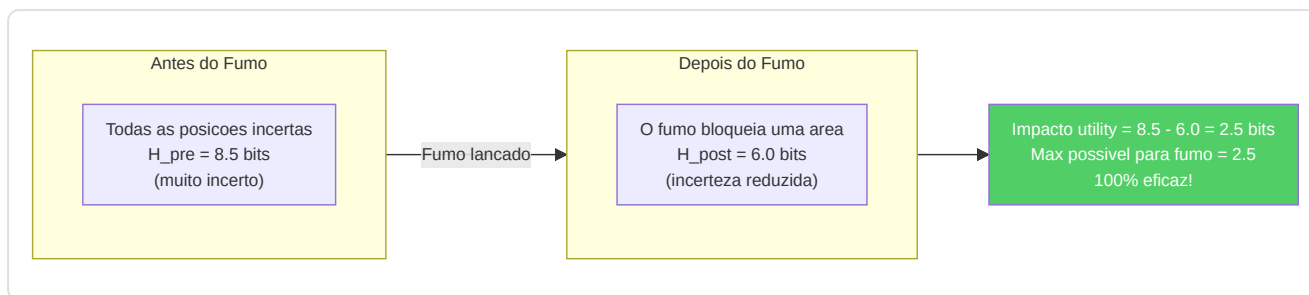
- Sequencias de vitorias: multiplicador = $1,0 + 0,05 \times \text{comprimento da sequencia} \times \text{decaimento}$
- Sequencias de derrotas: multiplicador = $1,0 - 0,04 \times \text{comprimento da sequencia} \times \text{decaimento}$
- Decaimento: $\exp(-0,15 \times \text{gap_rounds})$
- Limites: [0,7, 1,4]
- Deteccao tilt: multiplicador < 0,85
- Deteccao hot: multiplicador > 1,2
- Reset de metade: Round 13 (MR12) e 16 (MR13)



-Analizador de Entropia (`entropy_analysis.py`)

Mede a eficacia da utility atraves da **reducao de entropia de Shannon** das posicoes inimigas:

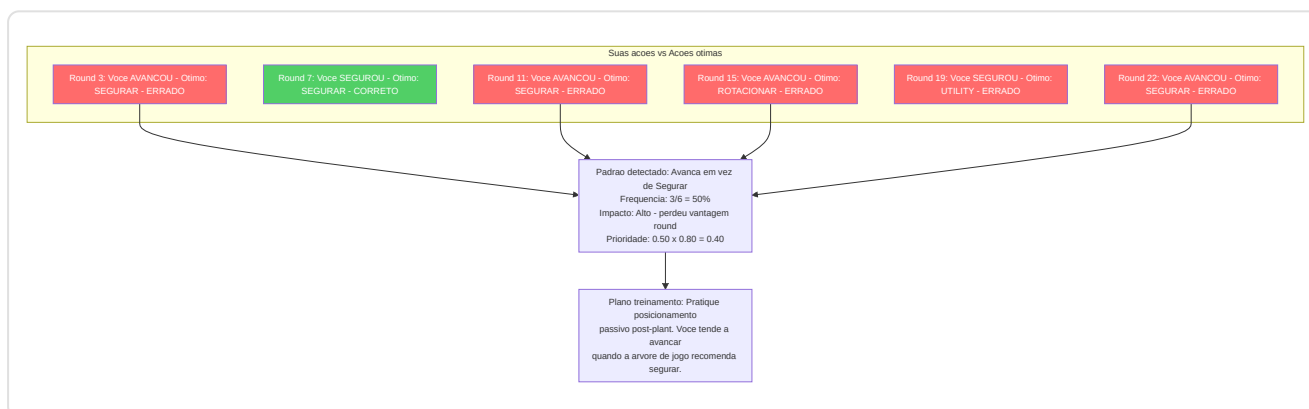
- Discretiza as posicoes em uma grade 32x32
- Calcula $H = -\text{Sigma } p(\text{cell}) \times \log_2(p(\text{cell}))$
- Impacto de utility = $H_{\text{pre}} - H_{\text{post}}$ (positivo = informacao obtida)
- Reducoes maximas de entropia: Smoke 2,5 bits, Molotov 2,0, Flash 1,8, HE 1,5



-Detector de Blind Spots (`blind_spots.py`)

Identifica decisoes recorrentes nao otimas em relacao as recomendacoes da arvore de jogo:

- Compara as acoes reais dos jogadores com as acoes otimas de `ExpectiminimaxSearch`
- Classifica as situacoes (post-plant, clutch, eco, round avancado, vantagem numerica)
- Prioridade = `frequencia x impact_rating`
- Gera planos de treinamento em linguagem natural para os blind spots principais



-Analizador distancia de engajamento (`engagement_range.py`)

Analisa as distancias de kill para construir **perfis de engajamento** especificos por papel e posicao:

Componentes principais:

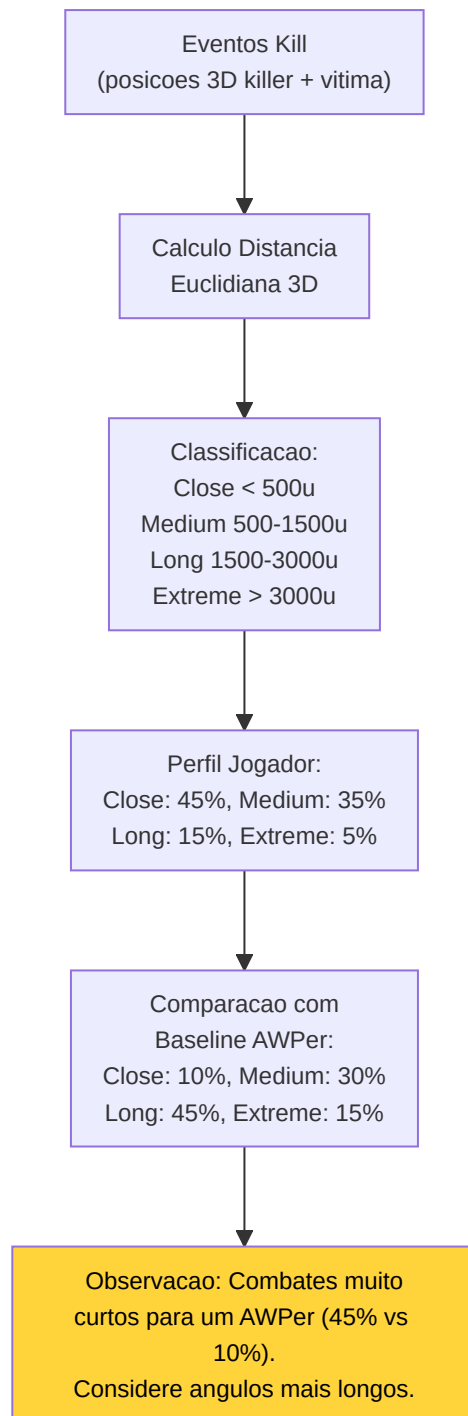
Componente	Proposito
<code>NamedPositionRegistry</code>	Registro de callouts por mapa (ex: "A Site", "Window", "Banana") com coordenadas 3D e raio
<code>EngagementRangeAnalyzer</code>	Calculo distancia euclidiana killer-vitima, classificacao e comparacao com baseline pro
<code>EngagementProfile</code>	Distribuicao % por faixa: close (<500u), medium (500-1500u), long (1500-3000u), extreme (>3000u)

Baseline pro por papel:

Papel	Close	Medium	Long	Extreme
AWPer	10%	30%	45%	15%
Entry Fragger	40%	40%	15%	5%
Support	25%	45%	25%	5%
Lurker	35%	35%	20%	10%
IGL/Flex	25%	40%	25%	10%

Limiar de desvio: Uma diferença >15% em relação a baseline do papel gera uma observação de coaching (ex: "Mais kills curtos do que o típico AWPer — considere ângulos mais longos").

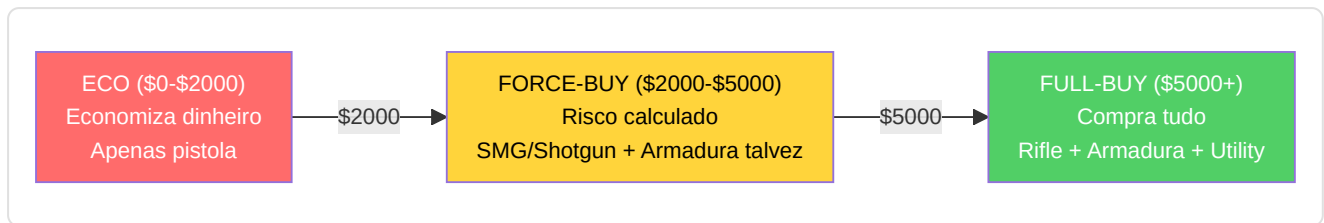
Mapas suportados: de_mirage, de_inferno, de_dust2, de_anubis, de_nuke, de_ancient, de_overpass, de_vertigo, de_train (não no Active Duty pool atual, suportada para demos históricas/workshop) — expansível via JSON.



-Analizador de utility e economia (`utility_economy.py`)

Analizador de utility: Score de eficacia por tipo em relacao as bases dos profissionais (Molotov: 35 dano/lancamento, Flash: 1,2 inimigos/flash, etc.)

Otimizador de economia: Conselhos de compra baseados em limiares economicos (\$5000 full buy, \$2000 force, <\$2000 economia), contexto do round, diferencial de score e bonus de derrota.



-Analizador Qualidade Movimento (`movement_quality.py`)

Detecta 4 erros comuns de posicionamento baseando-se no paper MLMove (SIGGRAPH 2024, Stanford/Activision/NVIDIA):

4 Tipologias de erro detectadas:

#	Tipo	Condicao de deteccao	Descricao
1	<code>high_ground_abandoned</code>	Descida ≥ 100 unidades sem contexto de combate	Abandono high ground sem necessidade
2	<code>position_abandoned</code>	Posicao mantida $\geq 3s$ deixada sem novas infos inimigas	Abandono posicao consolidada
3	<code>over_aggressive_trade</code>	Push solitario apos morte teammate com < 2 colegas restantes	Trading muito agressivo
4	<code>over_passive_support</code>	Imovel quando teammate cria abertura + vantagem numerica	Suporte muito passivo

Limiares chave:

Constante	Valor	Significado
<code>_ESTABLISHED_HOLD_TICKS</code>	384 (3s)	Tempo minimo para "posicao consolidada"
<code>_HIGH_GROUND_DROP</code>	100.0 unidades	Descida minima para flag high ground
<code>_TRADE_WINDOW_TICKS</code>	640 (5s)	Janela temporal para analise trade
<code>_AUDIO_RANGE_DISTANCE</code>	1500.0 unidades	Distancia maxima para "dentro do alcance audio"
<code>_MOVEMENT_THRESHOLD</code>	300.0 unidades	Deslocamento minimo para contar como "movido"
<code>_COMBAT_PROXIMITY_TICKS</code>	64 (~0.5s)	Ticks ao redor de um evento kill/death para contexto "em combate"

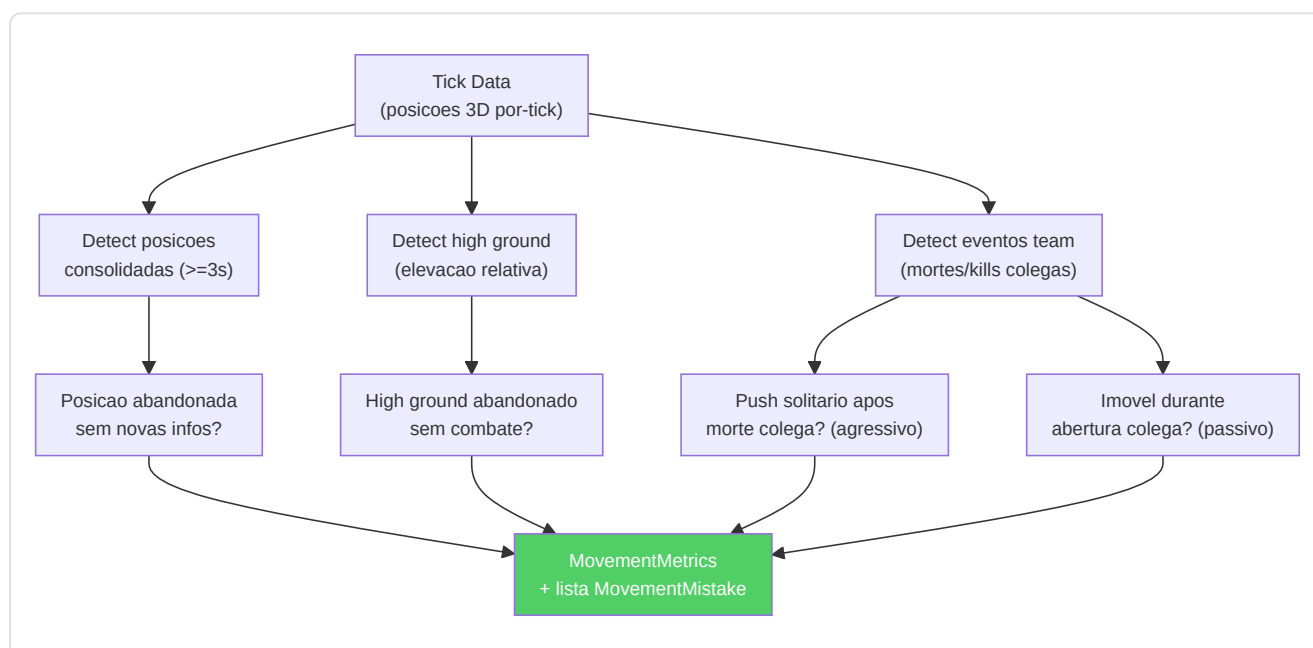
Dataclass de output:

- `MovementMistake` : tipo, round, tick, tempo no round, descricao, callout (posicao mapa), severidade [0-1]
- `MovementMetrics` : map_coverage_score, high_ground_utilization, position_stability, total_rounds_analyzed, lista mistakes + propriedade `mistakes_per_round`

API publica:

Metodo	Input	Output
<code>analyze_round_ticks(ticks, map_name, player, round)</code>	Ticks de um round	<code>List[MovementMistake]</code>
<code>analyze_match_ticks(all_ticks, map_name, player)</code>	Todos os ticks da partida	<code>MovementMetrics</code> (agregado)
<code>get_movement_quality_analyzer()</code>	—	Singleton <code>MovementQualityAnalyzer</code>

ADDITIVE: NAO modifica METADATA_DIM=25. As metricas de movimento sao calculadas como features derivadas dos dados tick existentes, memorizadas nos resultados de analise.



Resumo dos 11 Motores de Análise

#	Motor	Arquivo	Input	Output	Complexidade
1	Classificador Papeis	<code>role_classifier.py</code>	PlayerMatchStats	Papel (6 classes) + confiança	O(n) features
2	Probabilidade Vitória	<code>win_probability.py</code>	12 features estado round	P(CT win) in [0,1]	O(1) forward pass
3	Árvore de Jogo	<code>game_tree.py</code>	Estado round + ações	No ótimo (minimax)	O(b^d) branching
4	Morte Bayesiana	<code>belief_model.py</code>	Posição + tempo + round	P(morte) + fatores risco	O(n) prior update
5	Índice Deception	<code>deception_index.py</code>	Histórico round + posições	Score imprevisibilidade [0,1]	O(nxm) pattern match
6	Tracker Momentum	<code>momentum.py</code>	Sequência round	Estado hot/cold/neutral	O(n) sliding window
7	Analisador Entropia	<code>entropy_analysis.py</code>	Danos utility por tipo	Score eficácia vs pro	O(k) por tipo utility
8	Detector Blind Spots	<code>blind_spots.py</code>	Posições morte + ângulos	Padrões repetidos	O(n²) clustering
9	Utility e Economia	<code>utility_economy.py</code>	Economia round + utility	Conselho compra + rating	O(1) threshold check
10	Analisador Engajamento	<code>engagement_range.py</code>	Kill events 3D	Perfil distância (4 faixas)	O(n) euclidean dist
11	Qualidade Movimento	<code>movement_quality.py</code>	Tick data 3D por round	MovementMetrics (4 tipos erro)	O(n) por-tick scan

8. Subsistema 6 — Processamento e Feature Engineering

Diretório: `backend/processing/`

Este subsistema gerencia toda a **preparação dos dados**, transformando as gravações brutas do jogo nos formatos numéricos precisos de que as redes neurais precisam para o treinamento e a inferência.

-Extrator de Feature Unificado (`vectorizer.py`)

A **única fonte de verdade** para os vetores de features em nível de tick. Tanto o treinamento (`RAPStateReconstructor`) quanto a inferência (`GhostEngine`) DEVEM usar esta classe.

Contrato vetorial de features de 25 dimensões:

Indice	Feature	Normalizacao	Intervalo
0	health	/100	[0, 1]
1	armor	/100	[0, 1]
2	has_helmet	binario	{0, 1}
3	has_defuser	binario	{0, 1}
4	equipment_value	/10000	[0, 1]
5	is_crouching	binario	{0, 1}
6	is_scoped	binario	{0, 1}
7	is_blinded	binario	{0, 1}
8	enemies_visible	/5 (limitado)	[0, 1]
9	pos_x	/4096	[-1, 1]
10	pos_y	/4096	[-1, 1]
11	pos_z	/1024	[-1, 1]
12	view_yaw_sin	sin(yaw_rad)	[-1, 1]
13	view_yaw_cos	cos(yaw_rad)	[-1, 1]
14	view_pitch	/90	[-1, 1]
15	z_penalty	compute_z_penalty()	[0, 1]
16	kast_estimate	KAST das estatisticas ou 0,70 padrao	[0, 1]
17	map_id	Codificacao deterministica baseada em hash	[0, 1]
18	round_phase	0=pistola, 0,33=eco, 0,66=force, 1=full	[0, 1]
19	weapon_class	Mapeamento classes arma (0-1)	[0, 1]
20	time_in_round	/115 (segundos no round)	[0, 1]
21	bomb_planted	binario	{0, 1}
22	teammates_alive	/4 (colegas vivos)	[0, 1]
23	enemies_alive	/5 (inimigos vivos)	[0, 1]
24	team_economy	/16000 (media dinheiro team)	[0, 1]

Decisoes de design:

- **Codificacao ciclica do yaw** (sin/cos nos indices 12-13) elimina a descontinuidade de $\pm 180^\circ$
- **Penalidade Z** (indice 15) quantifica o risco de nivel errado para mapas multi-nivel
- **Integracao do contexto tatico** (indices 19-24) fornece ao modelo conhecimento da situacao de jogo
- **Cadeia de fallback da estimativa KAST:** valor explicito -> calculo das estatisticas -> valor padrao 0,70

- **Codificacao da identidade do mapa:** o hash deterministico permite o aprendizado especifico do mapa
- **HeuristicConfig** (`base_features.py`) permite sobrescrever todos os limites de normalizacao via JSON

Decisoões de design: A **codificacao ciclica do yaw** (sin/cos) resolve a descontinuidade angular: um angulo de -179° e $+179^\circ$ diferem apenas 2° na realidade mas 358° em representacao linear. O par $(\sin \theta, \cos \theta)$ mapeia o angulo em um circulo unitario onde a distancia euclidiana reflete a distancia angular real. A **penalidade Z** (`z_penalty` no vetor 25-dim) sinaliza explicitamente o risco de erro de andar vertical — critico em mapas multi-nivel (Nuke, Vertigo) onde confundir upper/lower leva a decisoes taticas completamente erradas. A **integracao tatica** (economia, vivos, tempo) fornece ao modelo o contexto necessario para avaliar se uma jogada agressiva e correta dado o vantagem numerica, o orcamento economico e o tempo restante no round.

-Rating HLTV 2.0 (`rating.py`)

O **modulo de rating unificado** que previne a inferencia-distorcao no treinamento:

$$R = (R_kill + R_survival + R_kast + R_impact + R_damage) / 5$$

onde:

$$R_kill = KPR / 0,679$$

$$R_survival = (1 - DPR) / 0,317$$

$$R_kast = KAST / 0,70$$

$$R_impact = (2,13 \cdot KPR + 0,42 \cdot ADR/100) / 1,0$$

$$R_damage = ADR / 73,3$$

Usado por: `demo_parser.py` (analise), `base_features.py` (agregacao), `coaching_service.py` (insights).

-Metricas PlusMinus e Rating Role-Adjusted (`rating.py`) — KT-06

Modulo complementar ao rating HLTV 2.0 que fornece duas metricas adicionais projetadas para capturar aspectos que o Rating 2.0 negligencia:

PlusMinus:

$$\text{PlusMinus} = (\text{kills} - \text{deaths}) / \max(\text{rounds_played}, 1) + \text{team_contribution_bonus}$$

Componente	Formula	Range tipico
Net frag differential	<code>(kills - deaths) / rounds</code>	[-1.0, +1.0]
Team contribution bonus	<code>_TEAM_CONTRIBUTION_SCALE x (team_win_rate - 0.5)</code>	[-0.05, +0.05]
<code>_TEAM_CONTRIBUTION_SCALE</code>	0.10	—

O bonus de contribuicao team premia os jogadores nas equipes vencedoras e penaliza aqueles nas equipes perdedoras, analogamente ao +/- no basquete/hoquei.

Rating Role-Adjusted (Bayesian):

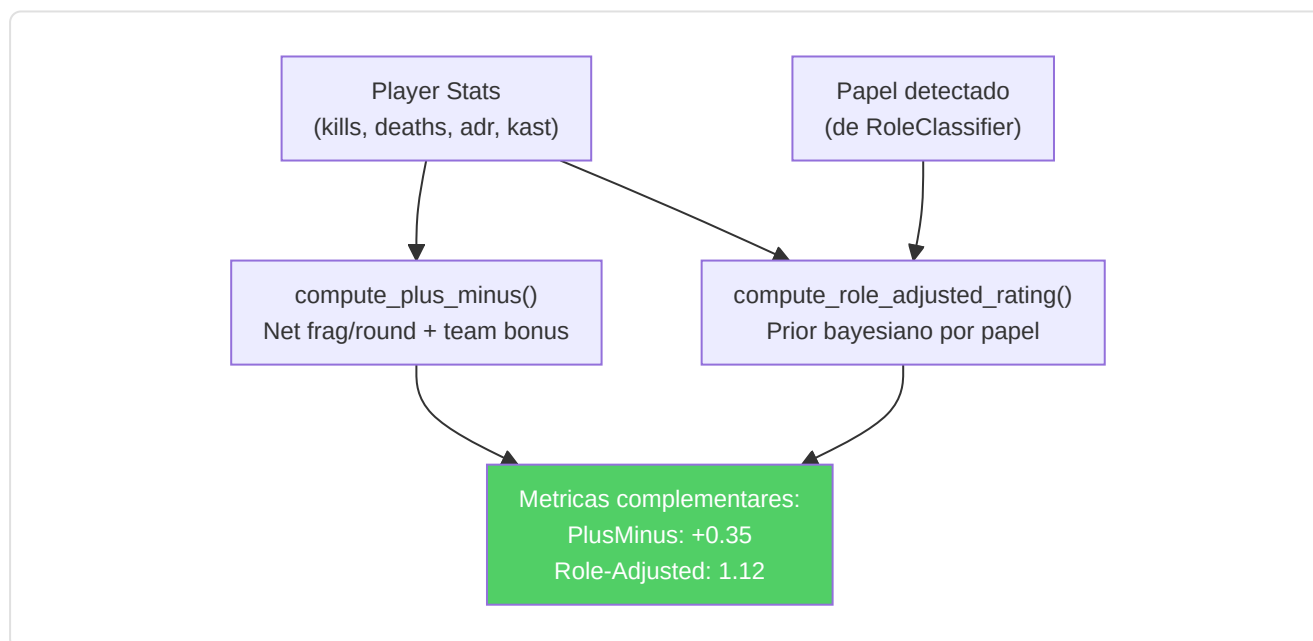
Aplica priors bayesianos especificos por papel para que AWPers nao sejam penalizados por KAST inferior e supports nao sejam penalizados por K/D inferior:

Papel	K/D Prior	KAST Prior	ADR Prior	Weight
AWPer	1.15	0.68	75.0	5.0
Entry	0.95	0.72	80.0	5.0
Support	0.90	0.78	65.0	5.0
Lurker	1.05	0.70	72.0	5.0
IGL	0.88	0.74	68.0	5.0

Priors calibrados de dados medios dos teams top-30 HLTV (temporada 2024-2025). Formula composta:

```
adj_metric = (n x observed + weight x prior) / (n + weight)
role_adjusted_rating = 0.40 x adj_kd + 0.35 x adj_kast + 0.25 x adj_adr_norm
```

Onde $\text{adj_adr_norm} = \text{adj_adr} / 120$ normaliza o ADR em [0, 1]. O framework e inspirado em TrueSkill (Herbrich et al., NeurIPS 2006) — quando a amostra e pequena (n baixo), o prior domina; quando a amostra e grande, os dados observados dominam.

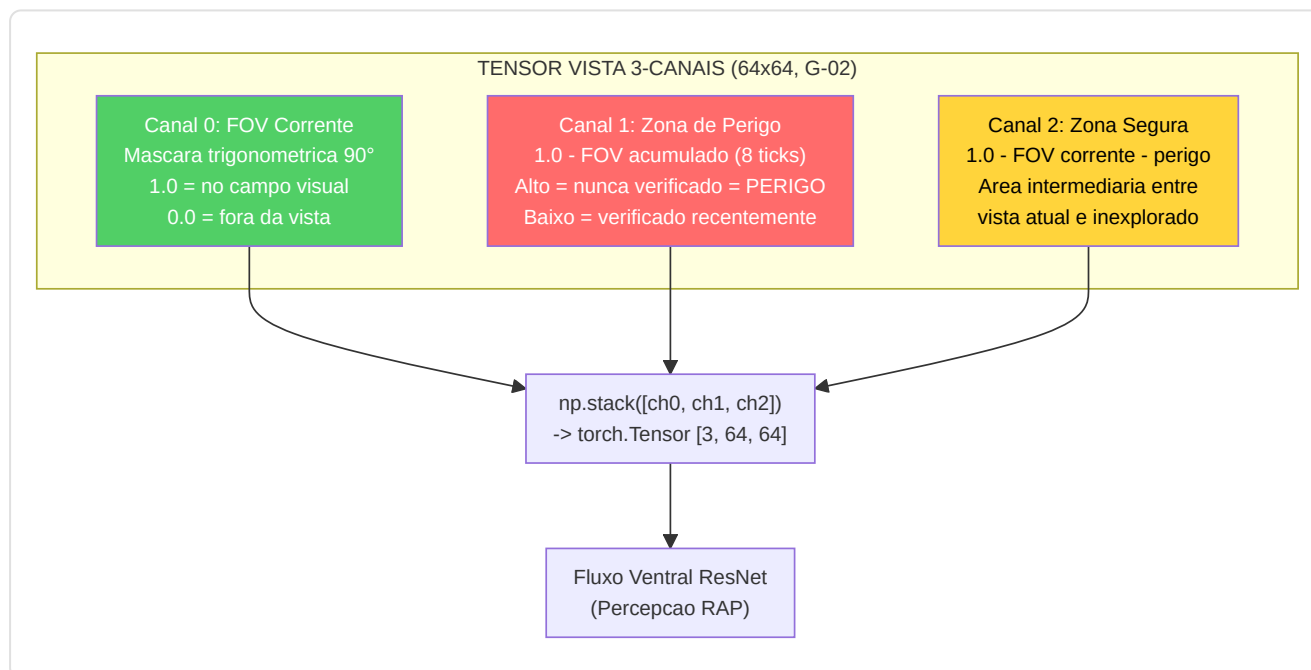


-Tensor Factory (`tensor_factory.py`)

Converte os dados tick brutos em tensores de imagens 64x64 para o nivel de percepcao RAP:

Tensor	Canais	Conteudo
Mapa	3 (R/G/B)	R: posicao do jogador, G: colegas de equipe (alpha-blended), B: inimigos (alpha-blended)
Vista	3	Ch0 : mascara FOV corrente (90° padrao, mascaramento trigonometrico); Ch1 : zona de perigo (= 1 - FOV acumulado nos ultimos 8 ticks), as areas nunca verificadas sao potenciais posicoes inimigas; Ch2 : zona segura (= 1 - FOV corrente - zona de perigo), a area entre a vista atual e as zonas inexploradas
Movimento	2 ou 3	Mapas de calor de velocidade/aceleracao (gotas 2D desfocadas gaussianas)

Correcao G-02 (Zona de Perigo): Anteriormente, o tensor vista tinha 3 canais identicos (placeholder). Apos a remediacao, os 3 canais codificam informacoes taticas distintas: o **FOV corrente** mostra o que o jogador ve agora, a **zona de perigo** acumula a historia FOV dos ultimos 8 ticks (~125ms a 64 Hz) e inverte o resultado para identificar as areas nunca verificadas — onde os inimigos poderiam estar escondidos — e a **zona segura** representa a area intermediaria entre a vista atual e as zonas inexploradas. O calculo da zona de perigo usa `np.maximum(accumulated_fov, tick_fov)` para cada tick historico, depois `danger_zone = 1.0 - accumulated_fov`. Isto fornece ao modelo RAP uma conscientizacao espacial da cobertura visual, transformando o tensor vista de um simples input estatico a um **indicador tatico temporal**.



-Heatmap Engine (`heatmap_engine.py`)

Mapas de ocupacao gaussianas de alto desempenho para a visualizacao tatica:

- Geracao de dados thread-safe (`generate_heatmap_data()`)
- Criacao de textura apenas no thread principal (OpenGL)
- Heatmaps diferenciais com deteccao de hotspots para o treinamento posicional

-Data Pipeline (`data_pipeline.py`)

`ProDataPipeline` gerencia a preparacao dos dados ML-ready:

1. **Recupera** todos os `PlayerMatchStats` do banco de dados
2. **Limpa** os valores anomalos (`avg_adr < 400` , `avg_kills < 3.0`)
3. **Redimensiona** via `StandardScaler`
4. **Divide** temporalmente (70/15/15) com ordenamento cronologico por grupo (pro/usuario)
5. **Mantenha** a coluna `dataset_split` no lugar

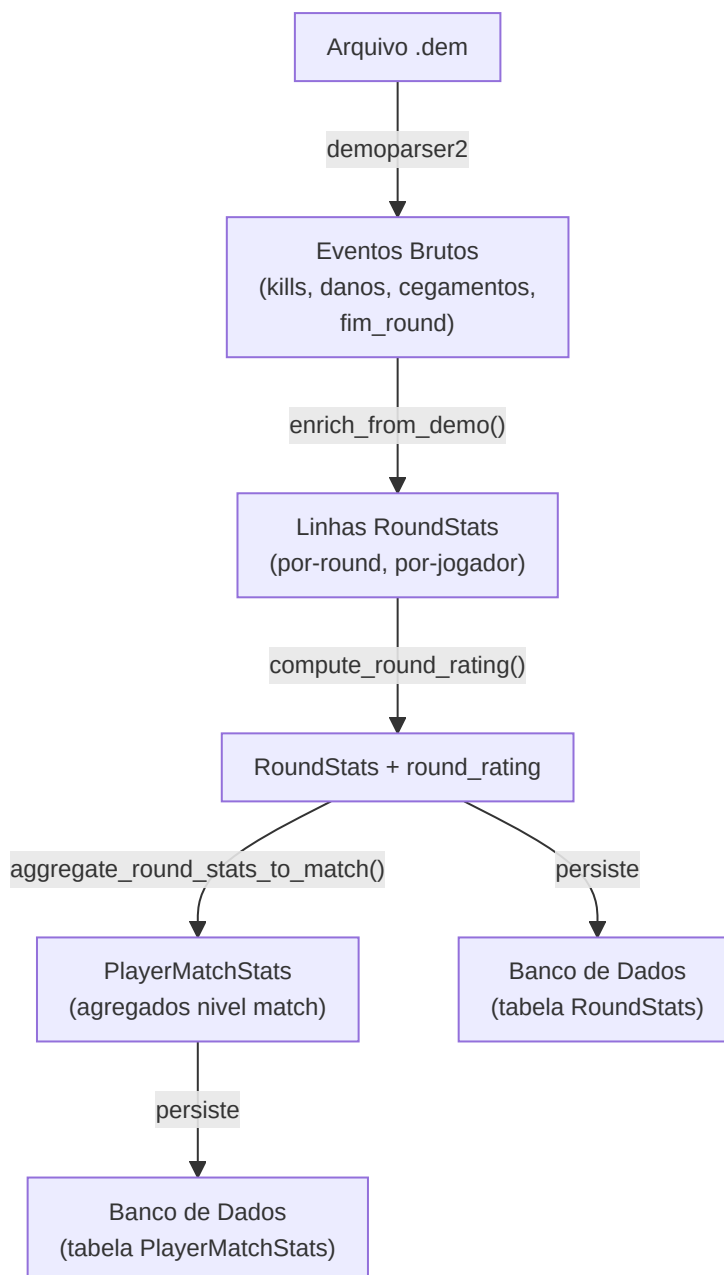
-Gerador de estatisticas por round (`round_stats_builder.py`)

Conecta os eventos demo brutos ao **nivel de isolamento por round** (`RoundStats` em `db_models.py`), impedindo a contaminacao estatistica entre rounds e permitindo analises detalhadas do coaching por round.

Funcoes chave:

Funcao	Proposito
<code>build_round_stats(parser, demo_name)</code>	Analisa os eventos round_end, player_death, player_hurt, player_blind e constroi as estatisticas por round
<code>enrich_from_demo(demo_path, demo_name)</code>	Enriquecimento completo: kills noscope/cegos, contagem flash/smoke, assist flash, kills com troca, analise das utilities
<code>aggregate_round_stats_to_match(round_stats_list)</code>	Processa as estatisticas em nivel de round -> PlayerMatchStats em nivel de partida
<code>compute_round_rating(round_stats)</code>	Rating HLTV 2.0 por round usando o modulo de rating unificado

Pipeline de enriquecimento:



Campos de enriquecimento kills adicionados por `enrich_from_demo()` :

Campo	Metodo de deteccao
<code>noscope_kills</code>	Kills onde AWP/Scout/SSG esta equipada mas <code>is_scoped=False</code>
<code>blind_kills</code>	Kills onde <code>attacker_blinded=True</code> no momento da morte
<code>flash_assists</code>	Kills dentro de 128 ticks (2 s) de um inimigo atingido por um flash da mesma equipe
<code>thrusmoke_kills</code>	Kills atraves de granada fumogena (dos flags do evento demo)
<code>wallbang_kills</code>	Kills atraves da penetracao do muro (dos flags do evento demo)
<code>trade_kills</code>	Kills dentro de 5 s da morte de um colega de equipe pelas maos do mesmo inimigo

Integracao com as pipelines de ingestao: Tanto `user_ingest.py` quanto `pro_ingest.py` agora chamam `enrich_from_demo()` apos a analise da demo e persistem os objetos `RoundStats` resultantes no banco de dados. Isto garante que cada demo ingerida, usuario ou profissional, produza dados granulares em nivel de round.

-Decaimento da baseline temporal (`pro_base_line.py`)

A classe `TemporalBaselineDecay` integra (nao substitui) a funcao `get_pro_baseline()` existente com uma media ponderada no tempo, garantindo que as comparacoes entre os coaches reflitam a **atual meta CS2** ao inves das medias historicas obsoletas.

Formula de decaimento:

```
weight(age_days) = max(MIN_WEIGHT, exp(-ln(2) x age_days / HALF_LIFE))
```

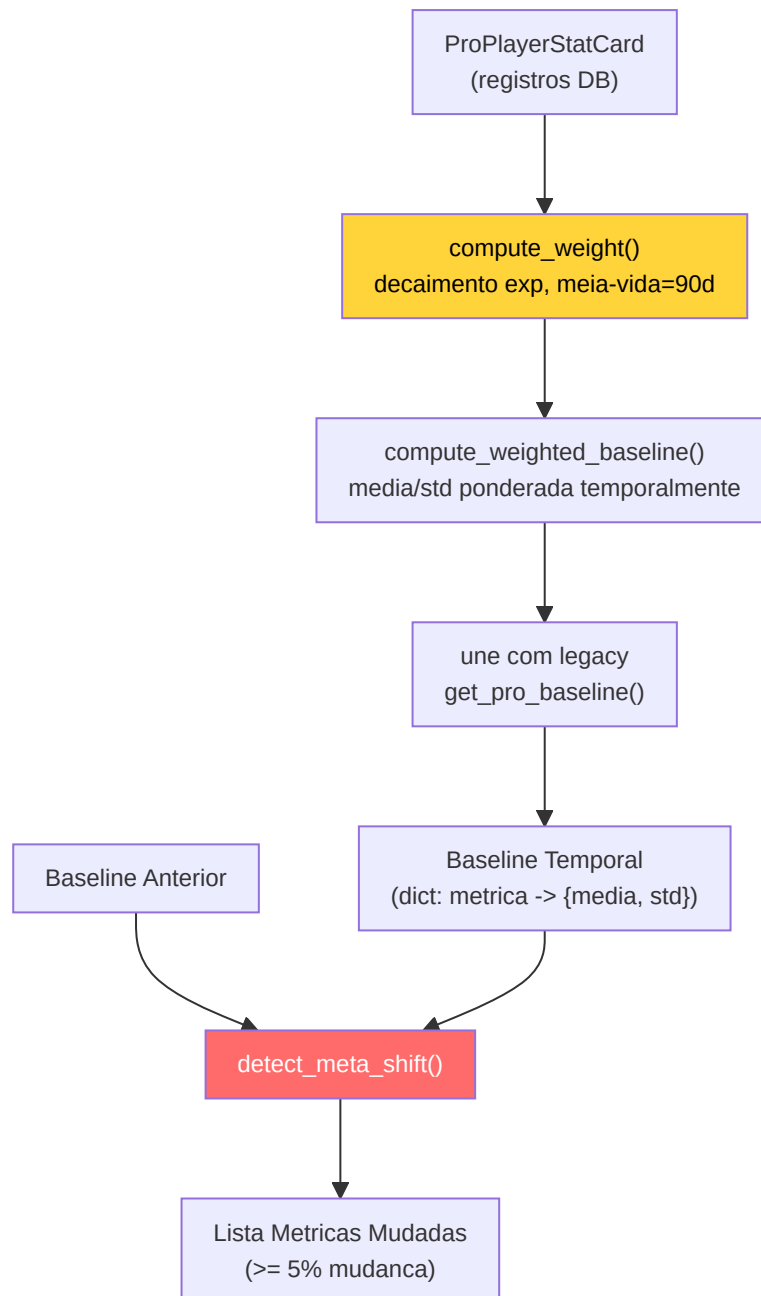
Parametro	Valor	Significado
<code>HALF_LIFE_DAYS</code>	90	O peso se divide pela metade a cada 90 dias
<code>MIN_WEIGHT</code>	0,1	Limiar minimo: dados muito antigos contribuem ainda 10%
<code>META_SHIFT_THRESHOLD</code>	0,05	Uma variacao de 5% entre as epocas sinaliza uma mudanca da meta

Metodos chave:

Metodo	Proposito
<code>compute_weight(stat_date)</code>	Peso de decaimento exponencial para uma unica stat card
<code>compute_weighted_baseline(stat_cards)</code>	Media/std ponderada no tempo em todos os registros ProPlayerStatCard
<code>get_temporal_baseline(map_name)</code>	Pipeline completa: recuperacao cards -> peso -> uniao com os valores padrao legacy
<code>detect_meta_shift(old, new)</code>	Sinaliza as metricas que sofreram um desvio >= 5% entre as epocas de referencia

Pontos de integracao:

- **CoachingService:** `_get_temporal_baseline()` o utiliza para o enriquecimento COPER
- **Teacher Daemon:** `_check_meta_shift()` e executado apos cada ciclo de retreinamento, registrando as metricas que sofreram um desvio



-Validacao ([validation/](#))

- **drift.py**: Deteccao do desvio na distribuicao dos dados com objetos DriftReport
- **schema.py**: Validacao do schema para os registros do banco de dados
- **sanity.py**** / dem_ **validator.py**:** Controles de integridade dos dados e dos arquivos demo

Cobertura quantitativa: O projeto compreende **1.515+ testes** distribuidos em 94 arquivos de teste e **319+ controles headless validator** articulados em 24+ fases de validacao. Esta cobertura vai da integridade do schema DB a coerencia dos vetores de embedding, da correcao das pipelines de treinamento a validacao end-to-end dos fluxos de coaching.

-PlayerKnowledge — Sistema Perceptivo NO-WALLHACK

(`player_knowledge.py`)

Modelo de percepcao **Player-POV** que reconstrói o que um jogador legitimamente sabe em cada tick, sem informacoes de wallhack. Este é o fundamento para coaching eticamente correto: o coach vê apenas o que o jogador via.

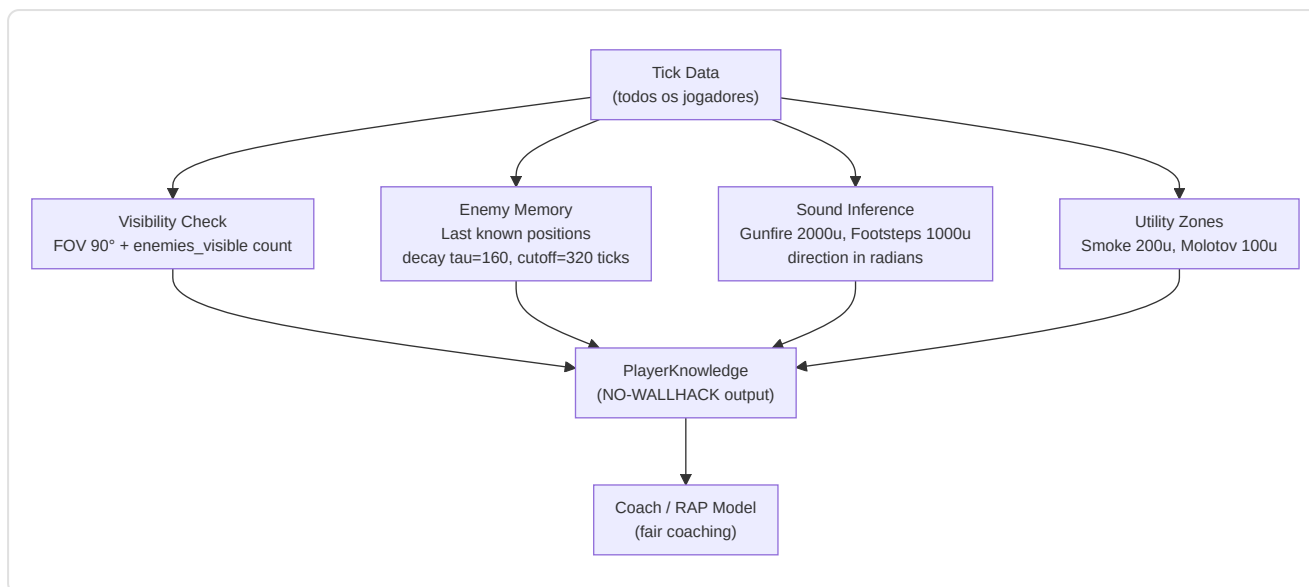
Dataclass de percepcao:

Dataclass	Campos	Descricao
<code>VisibleEntity</code>	position, distance, is_teammate, health, weapon	Entidade visivel no FOV
<code>LastKnownEnemy</code>	position, tick_seen, confidence	Posicao inimiga da memoria (decaimento)
<code>HeardEvent</code>	position, distance, direction_rad, event_type	Evento sonoro percebido
<code>UtilityZone</code>	position, radius, utility_type, team	Zona utility ativa
<code>PlayerKnowledge</code>	own_state, visible_entities, last_known, heard_events, utility_zones	Output completo

Constantes sensoriais:

Constante	Valor	Significado CS2
<code>FOV_DEGREES</code>	90.0	Campo visual horizontal CS2
<code>HEARING_RANGE_GUNFIRE</code>	2000.0	Distancia maxima percepcao tiros (world units)
<code>HEARING_RANGE_FOOTSTEP</code>	1000.0	Distancia maxima percepcao passos
<code>MEMORY_DECAY_TAU</code>	160	Meia-vida memoria: 2.5s a 64 tick/s
<code>MEMORY_CUTOFF_TICKS</code>	320	Cutoff memoria: 5 segundos max
<code>SMOKE_RADIUS</code>	200.0	Raio zona fumo
<code>MOLOTOV_RADIUS</code>	100.0	Raio zona molotov

Pipeline perceptiva:



Verificacao FOV: `_is_in_fov()` usa trigonometria ($\text{atan2} + \text{angle_diff}$) para determinar se um alvo esta dentro do campo visual horizontal do jogador. Apenas os inimigos confirmados visiveis (do contador `enemies_visible` da demo) sao incluidos.

-RAPStateReconstructor (`state_reconstructor.py`)

Vision Bridge Phase 1: Converte sequencias de `PlayerTickState` do DB em tensores de percepcao para o modelo RAP-Coach:

- **Metadados:** Vetor METADATA_DIM (25) do `FeatureExtractor` unificado
- **Percepcao:** Tensores view/map/motion da `TensorFactory`
- **Player-POV:** Modo opcional via parametro `knowledge` (PlayerKnowledge)
- **Windowing:** Janelas temporais de `sequence_length=32` ticks com overlap 50%

-ConnectMapContext (`connect_map_context.py`)

Features espaciais Z-aware para mapas multi-nivel (Task 2.17.1):

Constante	Valor	Uso
<code>Z_LEVEL_THRESHOLD</code>	200	Separacao entre niveis (world units)
<code>Z_PENALTY_FACTOR</code>	2.0x	Multiplicador distancia cross-level

Output: Vetor de 6 features espaciais normalizadas [0,1]: distancia de bombsite A/B, distancia de spawn T/CT, distancia de mid, penalidade Z. Distancias calculadas com `distance_with_z_penalty()` para penalizar percursos cross-level em Nuke/Vertigo.

-CVFrameBuffer (`cv_framebuffer.py`)

Ring buffer thread-safe para frames RGB (Task 2.24.2) com extracao regioes HUD:

Regiao	Coordenadas (1920x1080)	Conteudo
MINIMAP_REGION	(0, 0, 320, 320)	Minimapa (top-left)
KILL_FEED_REGION	(1520, 0, 1920, 300)	Kill feed (top-right)
SCOREBOARD_REGION	(760, 0, 1160, 60)	Scoreboard (top-center)

Dynamic resolution scaling: Todas as regioes escalam automaticamente em relacao a `REFERENCE_RESOLUTION = (1920, 1080)` para suportar resolucoes diferentes. Conversao BGR->RGB integrada no metodo `capture_frame()`.

-EliteAnalytics (`external_analytics.py`)

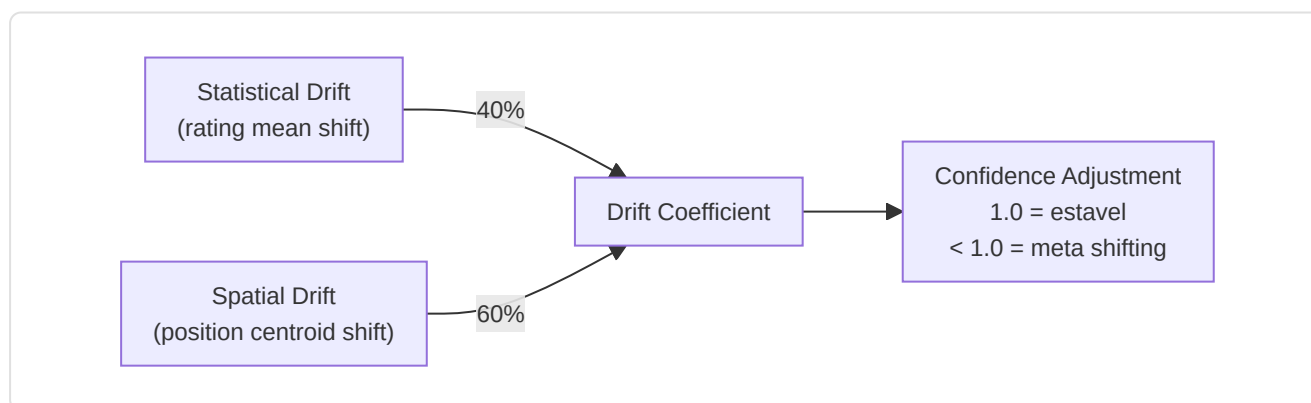
Carrega e analisa datasets CSV de referencia (top 100 jogadores, estatisticas match, dados torneios):

- **7 datasets:** top100, top100_core, match_stats, player_stats, kills_processed, historical_stats, tournament_stats
- **Calculo Z-score:** `_calc_z_scores()` compara usuario vs media historica para ADR, deaths, kills, rating, HS%
- **Z-score torneios:** `_calc_tournament_z()` compara vs baseline tournament para accuracy, econ_rating, utility_value
- **Graceful degradation:** `is_healthy()` = True se >=1 dataset carregado com sucesso

-MetaDriftEngine (`meta_drift.py`)

Pillar 2 Phase 3 — Meta-Drift Surveillance:

Compara posicoes pro dos ultimos 30 dias vs historico para detectar mudancas na meta:



Integracao: `HybridCoachingEngine._calculate_confidence()` chama `MetaDriftEngine.get_meta_confidence_adjustment()` para penalizar a confianca dos insights quando a meta e instavel.

-NicknameResolver (`nickname_resolver.py`)

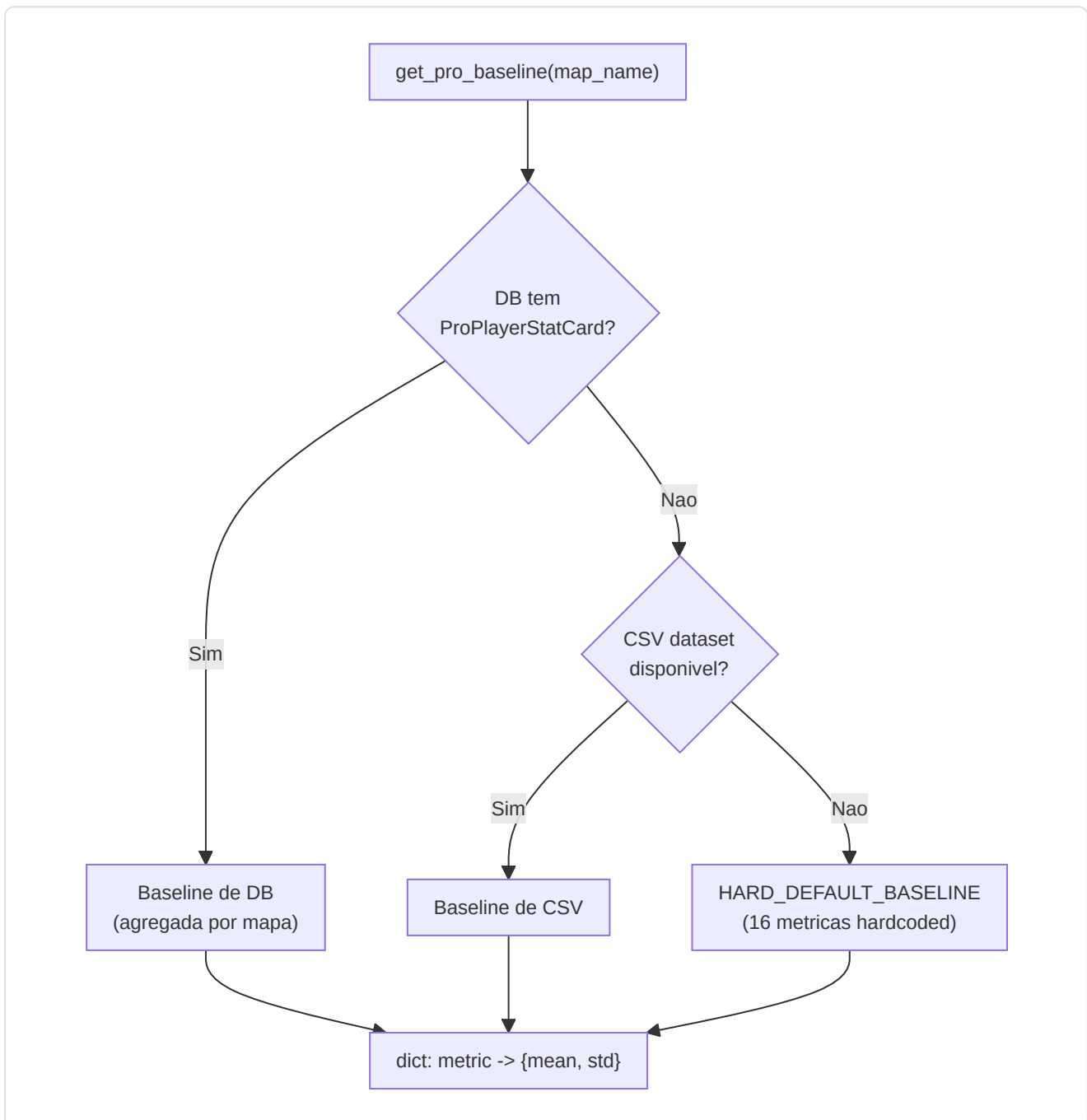
Task 2.18.2: Resolve nicknames in-game das demos em HLTV ProPlayer ID com matching a 3 niveis:

Nivel	Metodo	Exemplo
1. Exato	Case-insensitive match	"s1mple" -> ProPlayer.hltv_id
2. Substring	Nickname contido no nome demo	"FaZe_ropz" contem "ropz"
3. Fuzzy	SequenceMatcher >= 0.8 threshold	"s1mpl3" ~ "s1mple"

Gerencia tags de team e prefixos clan. Complexidade $O(n)$ por query, aceitavel para <1.000 pros no DB.

-ProBaseline (`pro_baseline.py`)

Task 2.18.1: Sistema de baseline pro com suporte map-specific e cadeia de fallback a 3 niveis:



HARD_DEFAULT_BASELINE (16 métricas com mean + std):

Métrica	Mean	Std
rating	1.06	0.15
kpr	0.68	0.12
dpr	0.62	0.10
adr	77.8	12.0
kast	0.70	0.06
hs_pct	0.45	0.10
impact	1.05	0.20
opening_kill_ratio	1.05	0.30
clutch_win_pct	0.15	0.08
...	...	...

`calculate_deviations(player_stats, baseline)` — Calcula Z-score para cada feature: $z = \frac{(\text{player_value} - \text{mean})}{\text{std}}$. Gerencia tanto baseline flat (legacy) quanto estruturadas (dict com mean/std).

-RoleThresholdStore (`role_thresholds.py`)

Princípio anti-mock: Todos os limiares de papel aprendem exclusivamente de dados pro reais (HLTV, demos, ML):

Estatística	Descrição	Cold-Start Default
<code>awp_kill_ratio</code>	% kills com AWP	—
<code>entry_rate</code>	Taxa de entry frag	—
<code>assist_rate</code>	Taxa de assistências	—
<code>survival_rate</code>	Taxa de sobrevivência	—
<code>solo_kill_rate</code>	Taxa kills solitárias	—
<code>first_death_rate</code>	Taxa primeira morte	—
<code>utility_damage_rate</code>	Dano utility por round	—
<code>clutch_rate</code>	Taxa de clutches vencidos	—
<code>trade_rate</code>	Taxa de trade kill	—

Cold-start protection:

Requisito	Valor	Significado
<code>MIN_SAMPLES_FOR_VALIDITY</code>	10	Minimo amostras para limiar valido
Limiares validos minimos	3	Para sair do cold-start
Cold-start output	<code>(FLEX, 0.0)</code>	Papel generico, 0% confianca

Persistencia: `persist_to_db()` / `load_from_db()` — os limiares aprendidos sobrevivem aos reinicios. `validate_consistency()` verifica a coerencia interna (ex: `entry_rate` nao pode ser negativo).

9. Subsistema 7 — Modulo de Controle

Pasta no repo: `backend/control/` **Arquivos:** 4 modulos

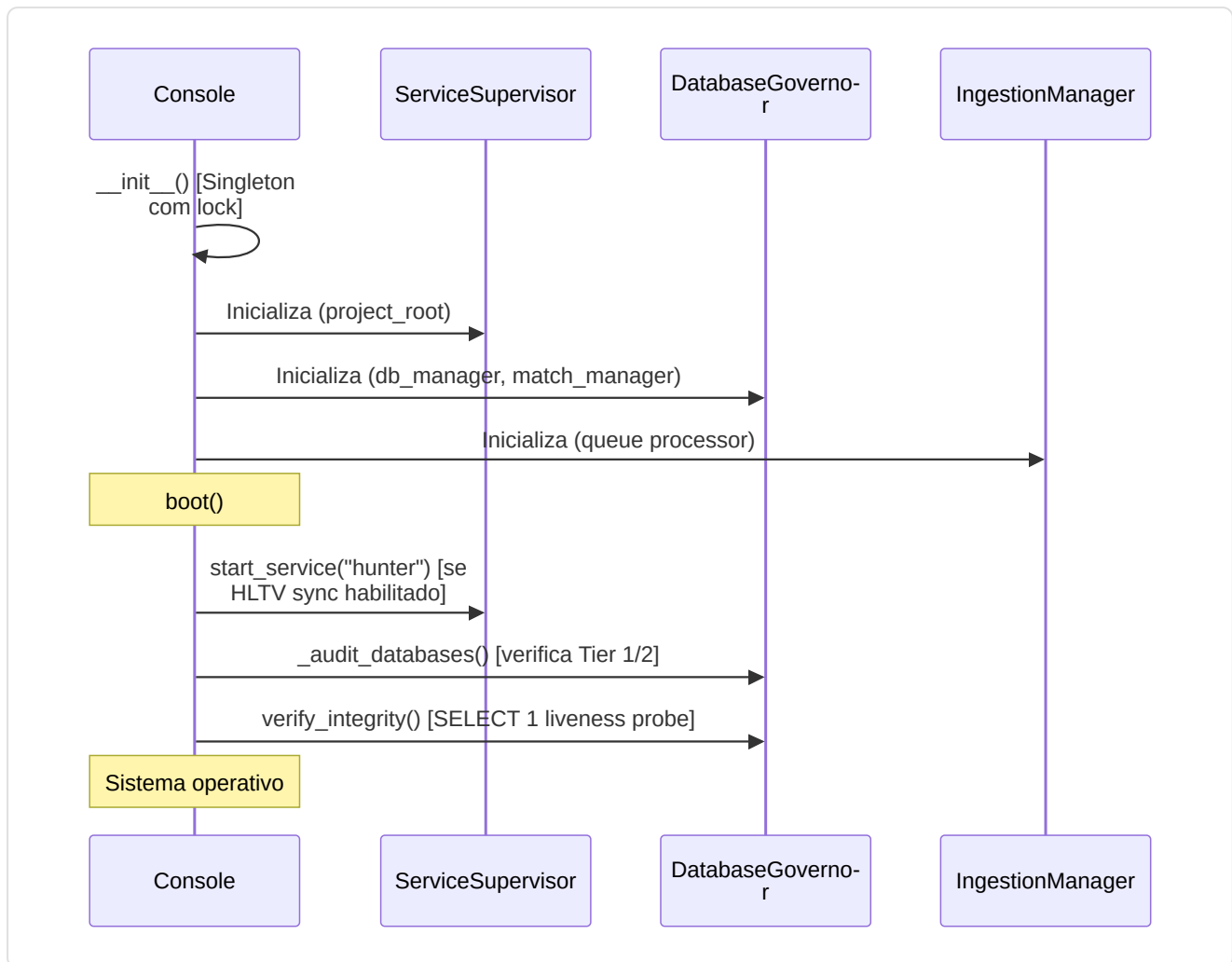
Este subsistema e a **torre de controle** do sistema: supervisiona servicos background, governa os bancos de dados, gerencia a ingestao das demos e controla o ciclo de vida do training ML.

-Console (`console.py`) — Singleton

A **Console de Controle Unificada**: autoridade central para ML, Ingestion e System State.

Componente	Papel
<code>ServiceSupervisor</code>	Supervisor background daemons (PID, liveness, auto-restart)
<code>IngestionManager</code>	Controller ingestao demo governado pelo operador
<code>DatabaseGovernor</code>	Controller integridade banco Tier 1/2/3
<code>MLController</code>	Supervisor ciclo de vida ML training

Sequencia de boot:



Cache de estado:

Cache	TTL	Conteudo
<code>_baseline_cache</code>	60s	Estado baseline temporal (stat_cards count, mode)
<code>_training_data_cache</code>	120s	Progresso demo (pro/user processed, on disk, trained_on)

Shutdown graceful: `shutdown()` -> para hunter -> para ingestion -> para ML training -> aguarda max 5s por confirmacao -> log warning se timeout.

Correcao M-08 — Validacao arquivo settings: `load_user_settings()` valida a estrutura JSON no carregamento: verifica que o payload e um `dict` (nao uma lista ou um tipo primitivo), loga um erro detalhado se a estrutura e invalida, e aplica um fallback graceful aos valores padrao. Isto impede que um arquivo `settings.json` corrompido ou adulterado cause crashes em cascata no modulo de configuracao.

Correcao C-09 — Bootstrap antecipado: As fases de bootstrap antes da inicializacao do logger usam `print()` para as mensagens diagnosticas. Isto evita o `NameError` que se verificava quando o codigo tentava chamar `logger.info()` antes que o logger fosse configurado.

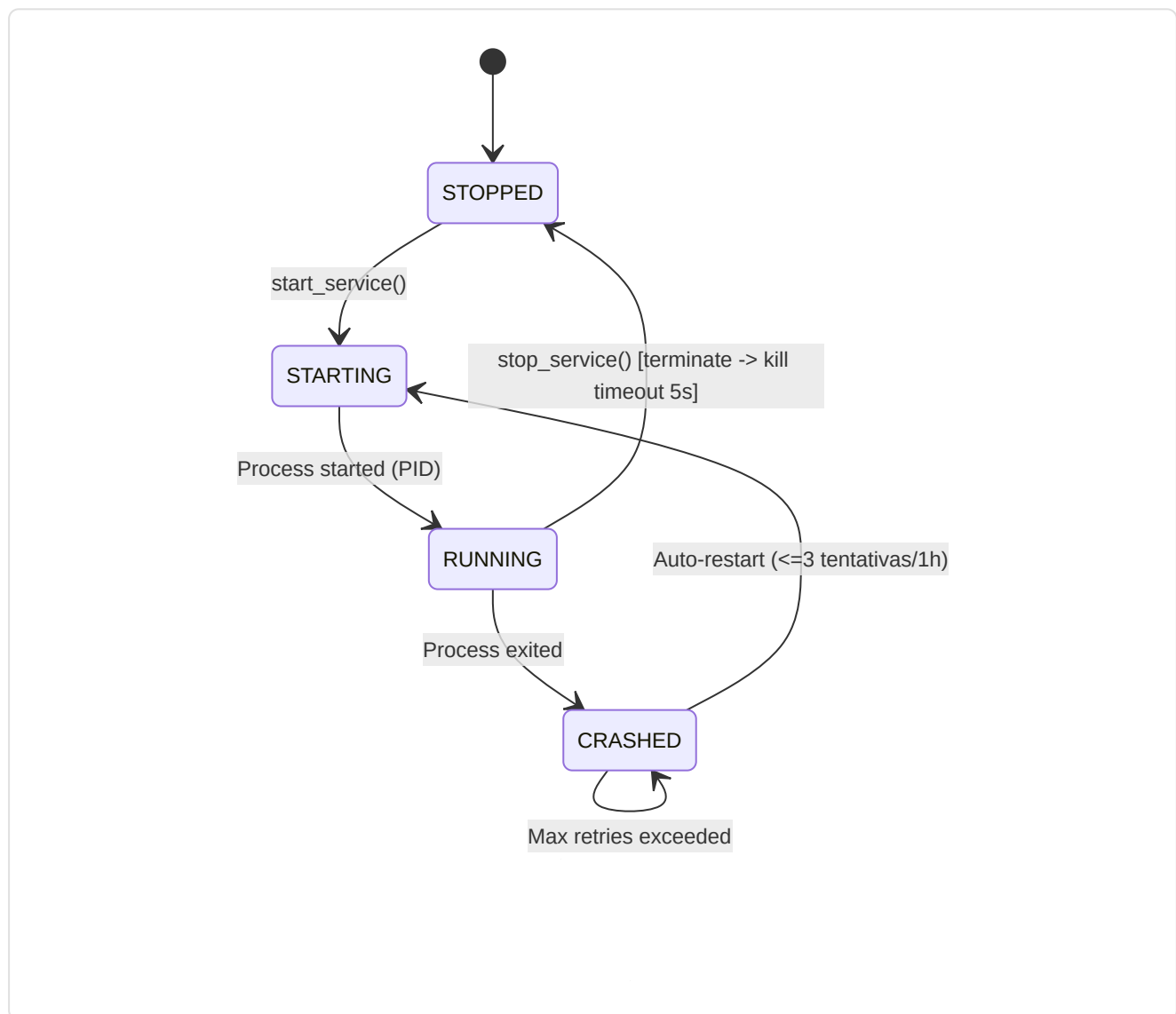
Agregacao health report: `get_system_status()` -> timestamp, state, services, teacher, ml_controller, ingestion, storage, baseline, training_data. Cada subsistema e isolado com `_safe_call()` — um erro em um subsistema nao impede o report dos outros.

-ServiceSupervisor (`console.py` , classe interna)

Gerenciador autoritativo para daemons de background com auto-restart:

Constante	Valor	Descricao
<code>_MAX_RETRIES</code>	3	Max auto-restart antes de desistir
<code>_RETRY_RESET_WINDOW_S</code>	3600 (1h)	Janela reset contador retry
<code>_RESTART_DELAY_S</code>	5.0	Delay entre tentativas de restart

Ciclo de vida servico:



Servicos gerenciados: Atualmente apenas `hunter` (HLTV sync daemon: `hltv_sync_service.py`).
Arquitetura extensivel para daemons adicionais.

-DatabaseGovernor (`db_governor.py`)

Controller autoritativo para Database Tier 1, 2 e 3:

Metodo	Proposito
<code>audit_storage()</code>	Calcula tamanho monolith + WAL + SHM, conta matches Tier 3, detecta anomalias
<code>verify_integrity(full=False)</code>	Lightweight: <code>SELECT 1</code> ; Full: <code>PRAGMA quick_check</code> (lento em DB grandes)
<code>prune_match_data(match_id)</code>	Cancelamento privilegiado dados telemetria match
<code>rebuild_indexes()</code>	Manutencao: <code>REINDEX</code> via ORM engine

Anomalias detectadas:

- `CRITICAL: Monolith database.db not found!`
- `CRITICAL: hltv_metadata.db missing and no backup found.`
- `WARNING: hltv_metadata.db missing, but .bak exists. Restore required.`

Tier Architecture:

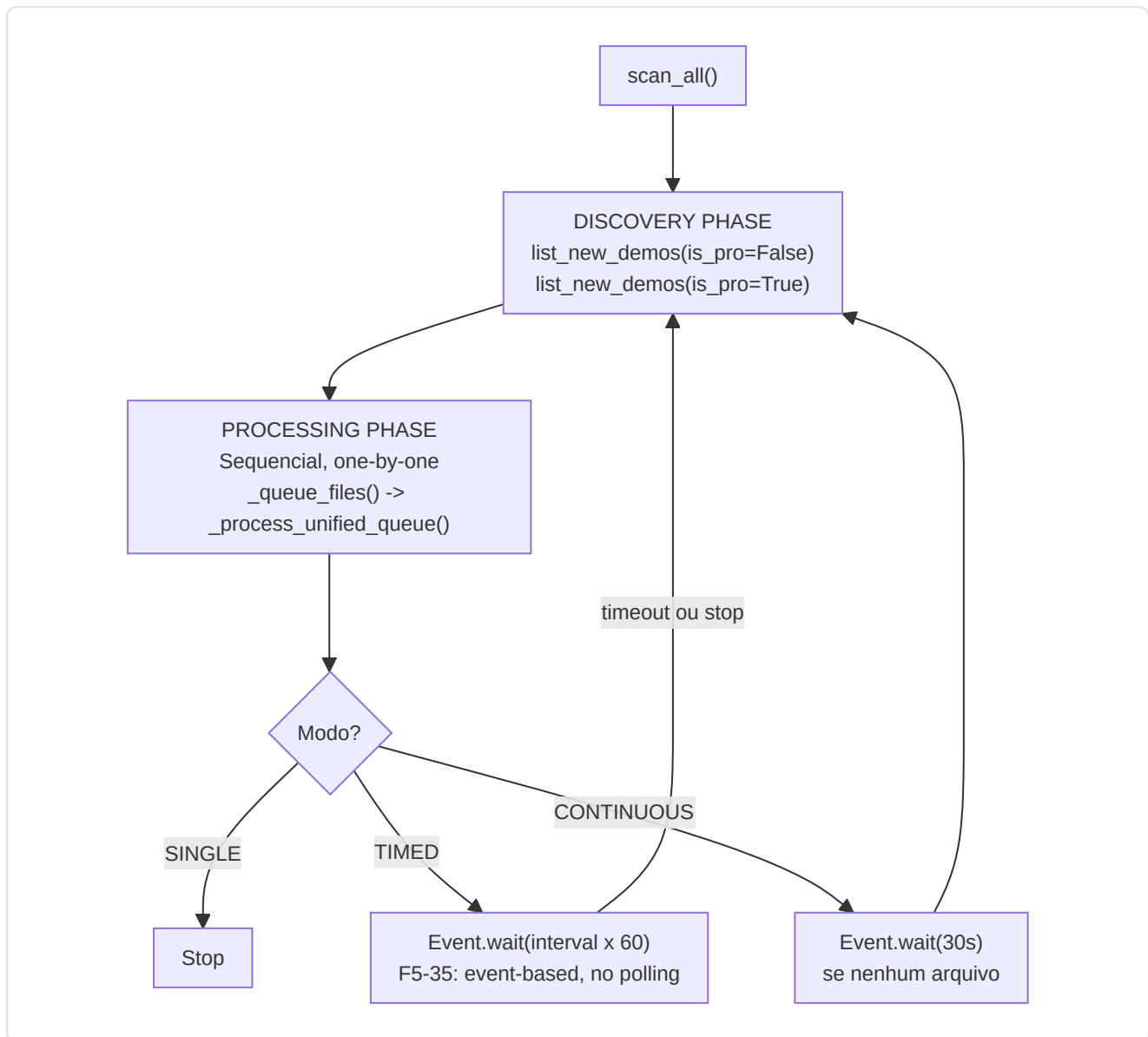
Tier	Database	Conteudo
Tier 1	<code>database.db</code> (monolith)	Jogadores, insights, perfis, modelos, knowledge base
Tier 2	<code>hltv_metadata.db</code>	Metadados HLTV, ProPlayer, ProPlayerStatCard
Tier 3	<code>match_*.db</code> (per-match)	Dados telemetria tick-by-tick por partida

-IngestionManager (`ingest_manager.py`)

Controller de ingestao demo com 3 modos operativos:

Modo	Comportamento
<code>SINGLE</code>	Processa uma demo e para
<code>CONTINUOUS</code>	Re-scan imediato, pausa 30s se nenhum arquivo encontrado
<code>TIMED</code>	Re-scan a cada N minutos (default 30)

Ciclo unificado:



Sinal de stop (F5-35): `threading.Event()` para stop nao-bloqueante — elimina o polling a 1 segundo nos wait loops.

Integracao StateManager: Progresso parsing em tempo real (0-100%) exposto via `state_manager.update_parsing_progress()`.

-MLController (`ml_controller.py`)

Supervisor do ciclo de vida ML com intervencao em tempo real:

Classe	Responsabilidade
<code>MLControlContext</code>	Token de controle passado aos loops ML
<code>MLController</code>	Supervisor thread-safe para training

MLControlContext — Comandos operador:

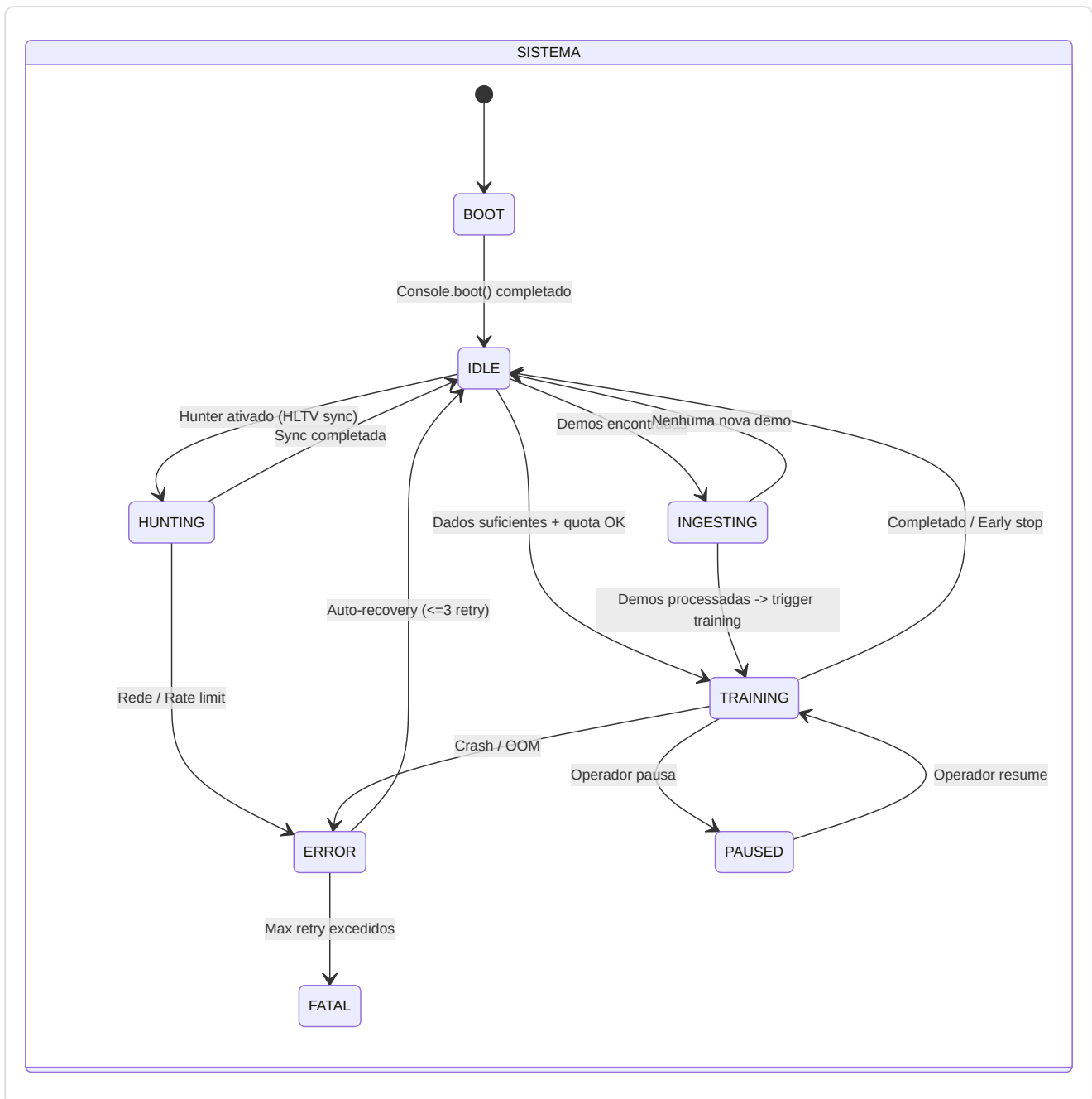
Metodo	Efeito
<code>check_state()</code>	Bloqueia se pause ativo (F5-15 event-based), lanca <code>TrainingStopRequested</code> se stop
<code>request_stop()</code>	Soft-stop no proximo checkpoint seguro
<code>request_pause()</code>	Bloqueia <code>check_state()</code> via <code>_resume_event.clear()</code>
<code>request_resume()</code>	Desbloqueia via <code>_resume_event.set()</code>
<code>set_throttle(value)</code>	0.0 = full speed, 1.0 = max delay

Pipeline: `start_training()` -> thread daemon ->

`CoachTrainingManager.run_full_cycle(context=self.context)` -> `TrainingStopRequested` capturada para stop graceful -> `set_error()` para crash.

Coordenacao Inter-Daemon

O Modulo de Controle orquestra os 4 daemons do sistema (Hunter, Digester, Teacher, Pulse) atraves de canais de comunicacao baseados em estado compartilhado (`CoachState`) e sinais event-based:



10. Subsistema 8 — Progresso e Tendencias

Pasta no repo: `backend/progress/` Arquivos: 2 modulos

Subsistema leve dedicado ao rastreamento das **trajetorias de desempenho** no tempo.

-FeatureTrend (`longitudinal.py`)

Dataclass minimal que representa o trend de uma unica feature:

Campo	Tipo	Descricao
<code>feature</code>	str	Nome da feature (ex: "avg_adr")
<code>slope</code>	float	Inclinacao da regressao linear (positivo = melhoria)
<code>volatility</code>	float	Desvio padrao dos valores (estabilidade)
<code>confidence</code>	float	$\min(1.0, \text{num_samples} / 30)$ — requer 30+ partidas para plena confianca

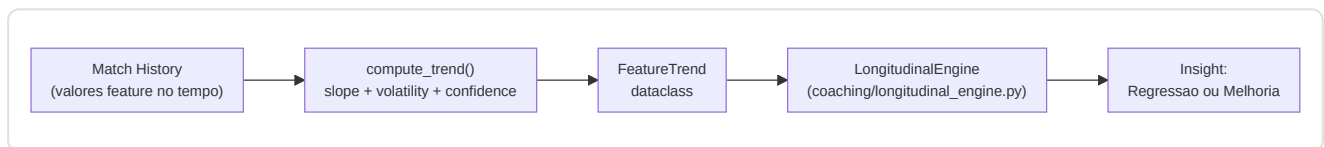
-TrendAnalysis (`trend_analysis.py`)

`compute_trend(values)` — Calcula slope, volatility e confidence de uma serie de valores:

- **Slope:** Coeficiente linear via `np.polyfit(x, y, 1)[0]`
- **Volatility:** `np.std(values)`
- **Confidence:** Escala linearmente com o numero de amostras, plena confianca em 30+

Integracao: Os resultados de `compute_trend()` alimentam `FeatureTrend` ->

`LongitudinalEngine.generate_longitudinal_coaching()` -> insights trend-aware no coaching.

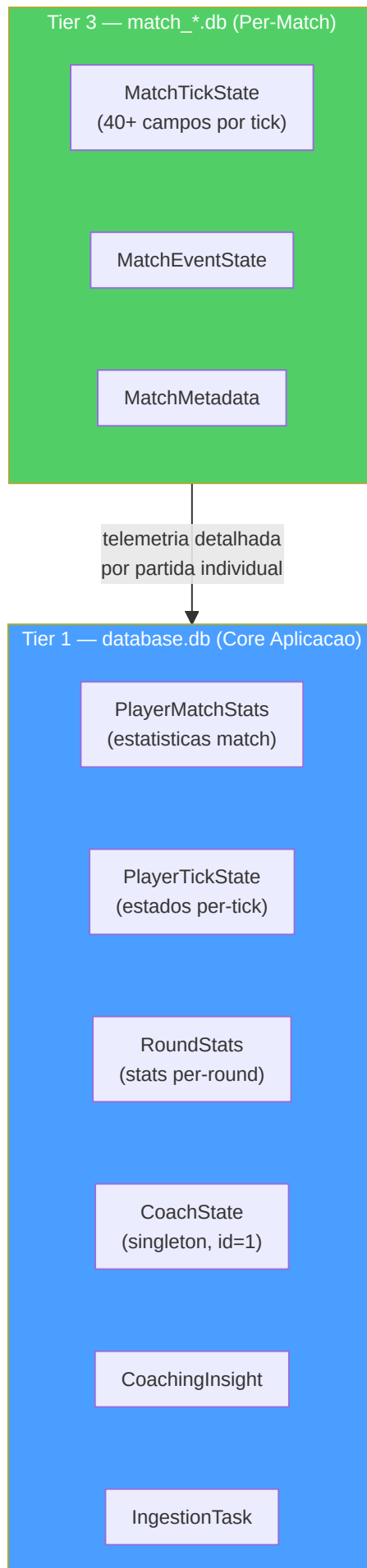


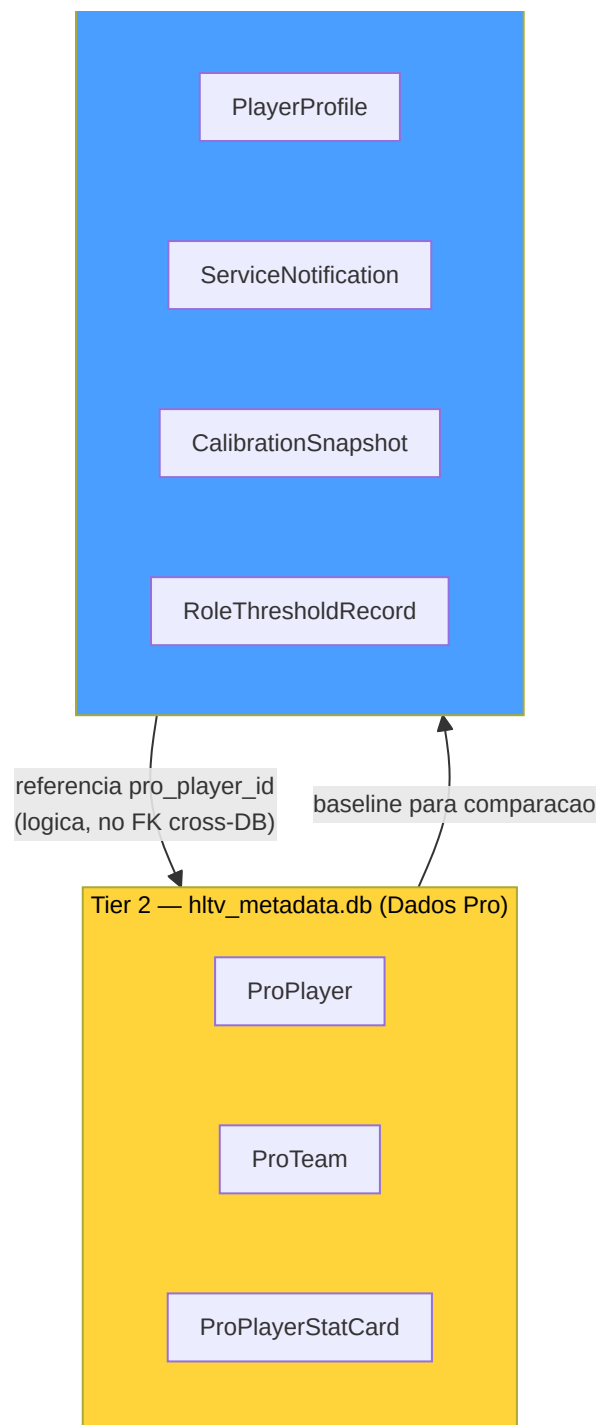
11. Subsistema 9 — Banco de Dados e Storage

Pasta no repo: `backend/storage/` **Arquivos:** 10 modulos

Este subsistema e o **sistema de arquivamento permanente** do projeto inteiro: cada dado — das estatisticas de uma partida aos modelos treinados, das experiencias de coaching aos perfis dos jogadores profissionais — e salvo, protegido, versionado e disponibilizado atraves desta infraestrutura.

Arquitetura Tri-Database:





Explicacao Diagrama: Os tres bancos de dados sao fisicamente separados: Tier 1 contem os dados core da aplicacao (17 tabelas), Tier 2 os dados profissionais baixados de HLTV (3 tabelas), e Tier 3 os dados de telemetria per-match em arquivos SQLite individuais. A separacao evita contencao WAL: as escritas de ingestao (Tier 1) nao bloqueiam as leituras HLTV (Tier 2) nem o registro de telemetria (Tier 3). As referencias cross-database sao **logicas** (nao FK reais) pois SQLite nao suporta FK cross-file.

-Modelos de Dados (`db_models.py`)

O **esquema canonico** de todo o sistema: define cada tabela, constraint, indice e validador via SQLAlchemy (Pydantic + SQLAlchemy). Dois enums e 20+ modelos organizados por tier.

Enum de integridade:

Enum	Valores	Uso
<code>DatasetSplit</code>	<code>TRAIN</code> , <code>VAL</code> , <code>TEST</code> , <code>UNASSIGNED</code>	Split ML em <code>PlayerMatchStats</code>
<code>CoachStatus</code>	<code>Paused</code> , <code>Training</code> , <code>Idle</code> , <code>Error</code>	Estado pipeline ML em <code>CoachState</code>

Constante de seguranca: `MAX_GAME_STATE_JSON_BYTES = 16.384` (16 KB) — cap para `game_state_json` em `CoachingExperience` , validado por `@field_validator` Pydantic. Previne crescimento ilimitado do DB de snapshots de estado grandes demais.

Catalogo dos modelos (Tier 1 — `database.db`):

Modelo	Campos Chave	Constraints / Indices	Proposito
PlayerMatchStats	40+ campos: kills, ADR, rating, HLTV 2.0 components, trade metrics, utility breakdown	UNIQUE(demo_name, player_name) , CHECK(avg_kills>=0) , CHECK(0<=rating<=5.0)	Estatisticas agregadas por partida por jogador
PlayerTickState	pos_x/y/z, view_x/y, health, armor, weapon, enemies_visible, round context	INDEX(demo_name, tick) , INDEX(player_name, demo_name) (P2-05)	Estado jogador per-tick para treinamento ML
PlayerProfile	player_name (unique), bio, role, profile_pic_path	UNIQUE(player_name)	Perfil usuario com papel e bio
Ext_TeamRoundStats	equipment_value, damage, kills, deaths, accuracy	INDEX(match_id) , INDEX(map_name)	Estatisticas round de CSV externos (torneios)
Ext_PlayerPlaystyle	6 probabilidades papel, metricas agregadas, config usuario (social, specs, cfg)	UNIQUE(steam_id)	Dados playstyle + perfil usuario (tabela conflada, candidata a split futuro)
CoachingInsight	title, severity, message, focus_area, user_id	INDEX(player_name) , INDEX(demo_name)	Feedback de coaching gerados pelo sistema
IngestionTask	demo_path (unique), status, retry_count, error_message	UNIQUE(demo_path) , INDEX(status) , R2-05: auto-refresh updated_at	Fila de ingestao demo com retry tracking
TacticalKnowledge	title, description, category, situation, embedding (384-dim JSON)	INDEX(title) , INDEX(category) , INDEX(map_name)	Base conhecimento RAG para coaching tatico
CoachState	status, daemon tracking (3 daemons), epoch/loss, heartbeat, resource limits	CHECK(id=1) singleton (P2-01)	Estado pipeline ML para a GUI — exatamente 1 linha
ServiceNotification	daemon, severity, message, is_read	INDEX(daemon) , INDEX(is_read)	Fila notificacoes erros/eventos para a UI
RoundStats	kills, deaths, assists, damage, utility, economy, round_won, round_rating	INDEX(demo_name, player_name) , INDEX(demo_name, round_number) (P2-05)	Isolamento estatisticas per-round per-jogador
CalibrationSnapshot	calibration_type, parameters_json, sample_count, source	INDEX(calibration_type)	Historico calibracao modelo bayesiano
RoleThresholdRecord	stat_name (unique), value, sample_count, source	UNIQUE(stat_name)	Limiaries papel aprendidos de dados pro

Modelos de match e pro (cross-tier):

Modelo	Tier	Campos Chave	Proposito
MatchResult	Tier 1	match_id (PK), team_a/b, winner, event_name	Metadados match
MapVeto	Tier 1	match_id (FK), map_name, action (pick/ban)	Vetos mapas
CoachingExperience	Tier 1	context_hash, map/phase/side, game_state_json (16KB cap), action, outcome, embedding, effectiveness_score, feedback loop fields	Banco experiencias COPER com semantic search e feedback loop
ProTeam	Tier 2	hltv_id (unique), name, world_rank	Teams profissionais
ProPlayer	Tier 2	hltv_id (unique), nickname, team_id (FK->ProTeam)	Jogadores profissionais
ProPlayerStatCard	Tier 2	player_id (FK), rating_2_0, kpr, adr, kast, detailed_stats_json	Stat cards HLTV com dados granulares

Nota arquitetural: `PlayerMatchStats.pro_player_id` e uma referencia **logica** (nao uma FK real) a `ProPlayer.hltv_id` porque residem em bancos de dados separados. SQLite nao suporta FK cross-file — o join e efetuado em nivel aplicativo.

Nota (P2-01): A constraint `CHECK(id=1)` em `CoachState` garante que exista exatamente uma linha no banco de dados, transformando-o em um singleton persistente. Qualquer tentativa de inserir uma segunda linha falha em nivel de banco de dados, nao apenas em nivel aplicativo.

-DatabaseManager (`database.py`)

O **guardiao do arquivo**: gerencia as conexoes SQLite com WAL mode obrigatorio, separando fisicamente o banco monolito (Tier 1) do banco HLTV (Tier 2).

Duas classes, dois bancos de dados:

Classe	Banco de dados	Tabelas	Engine
DatabaseManager	<code>database.db</code>	17 tabelas (<code>_MONOLITH_TABLES</code>)	<code>pool_size=1</code> , <code>max_overflow=4</code>
HLTVDatabaseManager	<code>hltv_metadata.db</code>	3 tabelas (<code>_HLTV_TABLES</code>)	<code>pool_size=1</code> , <code>max_overflow=4</code>

PRAGMA SQLite (aplicados em cada conexao via `@event.listen_for`):

PRAGMA	Valor	Proposito
<code>journal_mode</code>	<code>WAL</code>	Leituras concorrentes + unica escrita
<code>synchronous</code>	<code>NORMAL</code>	Balanceamento performance/durabilidade
<code>busy_timeout</code>	<code>30000</code> (30s)	Timeout contencao WAL lock

API publica:

Metodo	Descricao
<code>create_db_and_tables()</code>	Cria schema no monolito (apenas <code>_MONOLITH_TABLES</code> , nao todas as tabelas SQLAlchemy)
<code>get_session()</code>	Context manager transacional: auto-commit em sucesso, auto-rollback em execucao
<code>upsert(model_instance)</code>	Upsert atomico; gerenciamento especial para <code>PlayerMatchStats</code> (lookup por <code>demo_name</code> + <code>player_name</code>)
<code>get(model_class, pk)</code>	Leitura por chave primaria

Singleton: `get_db_manager()` e `get_hltv_db_manager()` com double-checked locking (pattern identico). Evita a criacao de engines duplicados durante o import de threads multiplos.

Isolamento tabelas (critico): A lista explicita `_MONOLITH_TABLES` impede que os modelos per-match (`MatchTickState`, `MatchEventState`, `MatchMetadata`) sejam criados no banco monolito se seus modulos sao importados antes de `create_db_and_tables()`. Sem esta guarda, cada modelo SQLAlchemy globalmente registrado seria criado em `database.db`.

Nota sobre a reconciliacao HLTV: `HLTVDatabaseManager._reconcile_stale_schema()` detecta colunas faltantes em tabelas HLTV existentes (schema atualizado mas DB antigo) e recria as tabelas interessadas via `DROP TABLE + CREATE TABLE`, operando como uma migracao destrutiva de emergencia — aceitavel porque os dados HLTV sao re-baixaveis.

-MatchDataManager (`match_data_manager.py`)

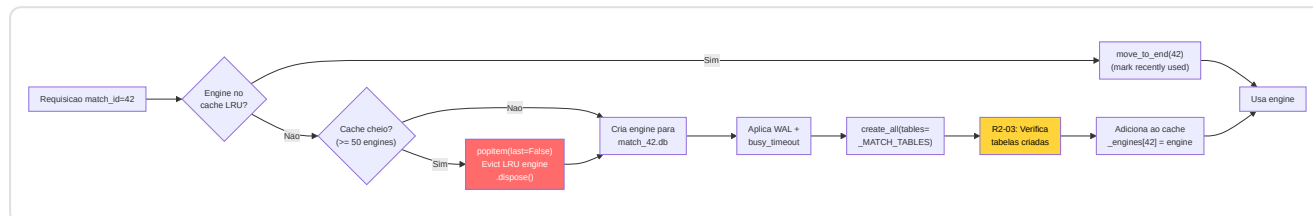
O **sistema de cameras de seguranca** do projeto: cada partida recebe seu arquivo SQLite dedicado (`match_{id}.db`) contendo a telemetria tick-by-tick — ate **1,7 milhoes de linhas por partida**.

3 modelos per-match (Tier 3):

Modelo	Campos	Proposito
<code>MatchTickState</code>	40+ campos: posicao, estado, equipamento, visibility, round context, cumulativos match	Estado jogador a cada tick (128 Hz)
<code>MatchEventState</code>	tick, event_type, player/victim info, position, weapon, damage, entity_id	Eventos de jogo para reconstrucao Player-POV
<code>MatchMetadata</code>	match_id, demo_name, map_name, tick/round/player count, team names/scores	Metadados para acesso sem DB monolito

Nota (C-08): `MatchEventState.entity_id` usa `-1` como sentinel (nao `0`) para distinguir "ID nao populado do parser" de "entity ID real = 0", evitando acoplamentos errados fumo/molotov.

Cache LRU (M-18):



Nota (R2-03): O filtro `tables=_MATCH_TABLES` no `create_all()` e **critico**. Sem ele, `SQLModel` criaria todas as ~20 tabelas globais em cada arquivo per-match. Um controle defensivo post-criacao verifica que apenas as 3 tabelas esperadas existam no DB, logando um warning se aparecem tabelas inesperadas.

API publica:

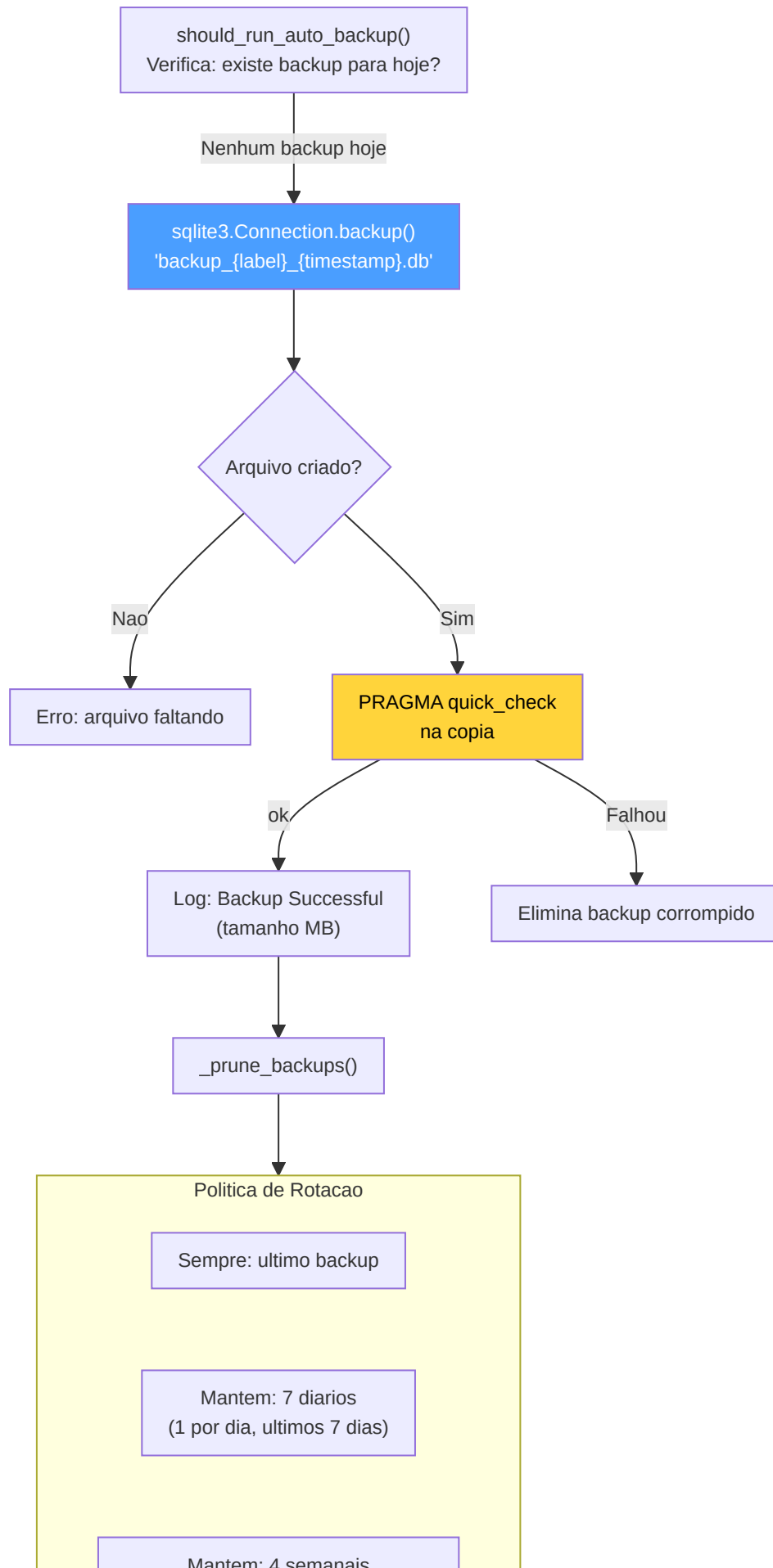
Metodo	Descricao
<code>get_match_session(match_id)</code>	Context manager transacional para um match especifico
<code>store_tick_batch(match_id, ticks)</code>	Insercao batch de <code>MatchTickState</code> — retorna contagem
<code>store_metadata(match_id, metadata)</code>	Upsert metadados match
<code>get_ticks_for_round(match_id, round)</code>	Query ticks por round especifico
<code>get_player_ticks(match_id, player, start, end)</code>	Janela ticks por jogador + intervalo
<code>get_all_players_at_tick(match_id, tick)</code>	Snapshot todos os jogadores em um tick
<code>get_all_players_tick_window(match_id, start, end)</code>	Janela ticks para todos os jogadores (treinamento RAP)

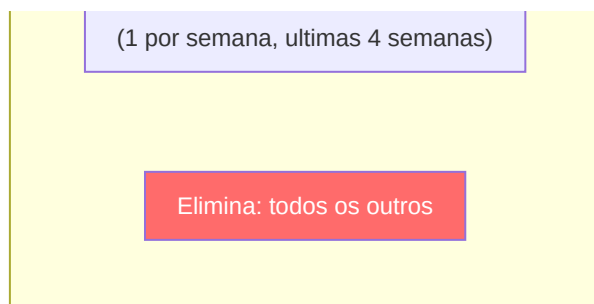
-BackupManager (`backup_manager.py`)

O **responsavel pela seguranca dos dados**: cria copias de backup nao-bloqueantes atraves da **SQLite Online Backup API** (`sqlite3.Connection.backup()`) e as gerencia com uma politica de rotacao. A API de backup online (nao `VACUUM INTO`) e a escolha correta porque opera de forma segura com WAL-mode e acesso concorrente — copia as paginas de forma incremental sem exigir lock exclusivo no banco de dados inteiro.

Correcao H-02 — DB handle leak: `_verify_integrity()` utiliza um bloco `try/finally` para garantir o fechamento da conexao `sqlite3` mesmo em caso de erro. O PRAGMA utilizado e `quick_check` (nao `integrity_check`) para reduzir o tempo de verificacao em bancos de dados grandes — `quick_check` omite a validacao dos indices, suficiente para verificar a integridade estrutural das paginas do backup.

Pipeline de backup:





Seguranca path: O label do backup e pre-validado com regex `_SAFE_BACKUP_LABEL_RE = ^[a-zA-Z0-9_\-]{1,64}$` (BE-01, DB-03). O path de destino e verificado via `Path.resolve().relative_to()` para garantir que reside dentro do diretorio de backup — defesa em profundidade (BE-06) contra path traversal. A API de backup online (`sqlite3.Connection.backup()`) nao envolve queries SQL para o path, eliminando o risco de injection neste nivel.

-StorageManager (`storage_manager.py`)

Gerenciador do **storage local** para demos CS2: pastas de ingestao, arquivo post-elaboracao, e quotas disco.

Funcionalidade	Detalhe
Pasta ingestao	Path configuravel (<code>DEFAULT_DEMO_PATH</code>), fallback a <code>data/</code> in-project
Arquivo	<code>datasets/user_archive/</code> e <code>datasets/pro_archive/</code> no "Brain Root"
Quotas	<code>MAX_DEMOS_PER_MONTH</code> e <code>MAX_TOTAL_DEMOS_PER_USER</code> de config
Quota disco	<code>LOCAL_QUOTA_GB</code> (default 10 GB), verificada antes do arquivamento
Discovery	<code>list_new_demos(is_pro)</code> — busca <code>.dem</code> nao ainda processados em <code>PRO_DEMO_PATH</code> ou <code>DEFAULT_DEMO_PATH</code>

-StateManager (`state_manager.py`)

DAO centralizado para o singleton `CoachState` : interface thread-safe entre os daemons e a GUI.

Metodo	Descricao
<code>get_state()</code>	Recupera o singleton (o cria se nao existe, lock P0-04)
<code>update_status(daemon, status, detail)</code>	Atualiza estado de um daemon especifico (hunter/digester/teacher/global)
<code>update_training_progress(epoch, total, train_loss, val_loss, eta)</code>	Progresso real-time treinamento para a GUI
<code>update_parsing_progress(progress)</code>	Barra de progresso parsing demo (0.0-100.0)
<code>push_notification(daemon, severity, message)</code>	Insere uma <code>ServiceNotification</code> na fila para a UI
<code>get_unread_notifications(limit)</code>	Recupera notificacoes nao lidas para visualizacao

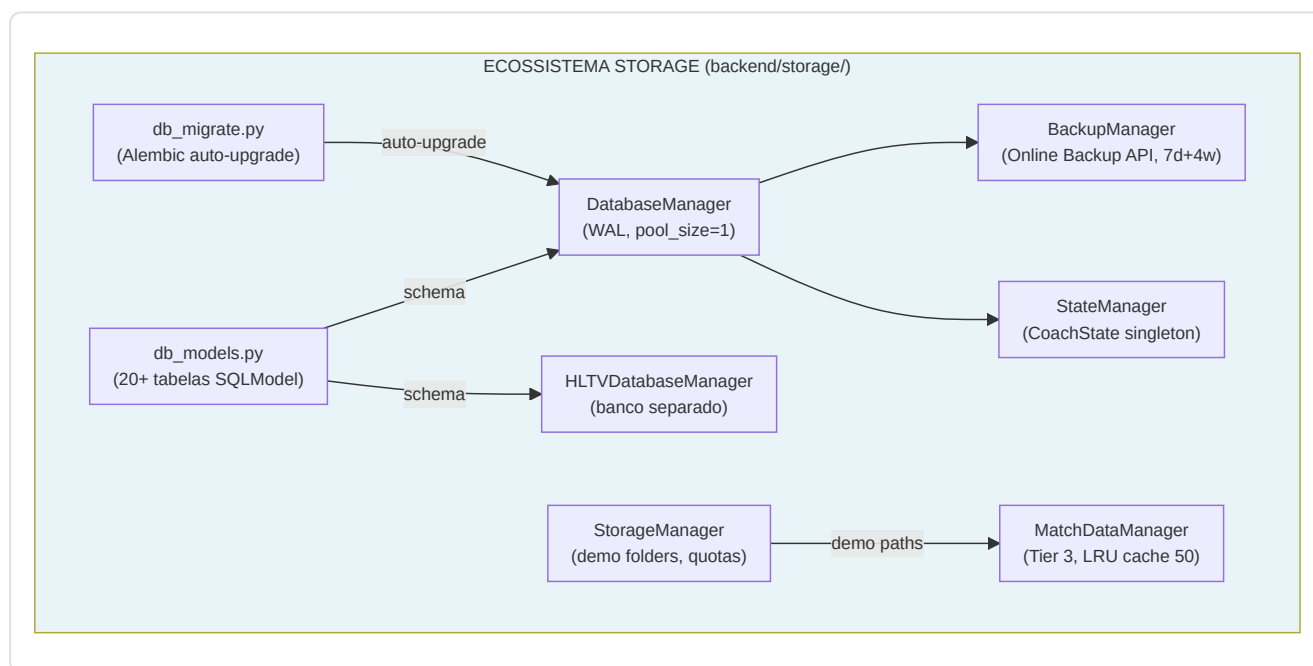
-Servicos Storage Adicionais

stat_aggregator.py — Persistencia dados spider HLTV: `persist_player_card(card_data, player_hltv_id)` converte os dados extraídos do spider em um `ProPlayerStatCard` e o salva no banco de dados HLTV. `persist_team(team_data)` salva/atualiza um `ProTeam`.

db_migrate.py — Orquestracao migracoes Alembic: `ensure_database_current()` e chamada no inicio da aplicacao para auto-atualizar o schema. Compara `current_rev` vs `head_rev` e se diferentes executa `alembic.command.upgrade(cfg, "head")`. Se `alembic.ini` nao existe (desenvolvimento sem Alembic), retorna `True` silenciosamente.

maintenance.py — Manutencao banco de dados: `prune_old_metadata(days=90)` elimina registros antigos em batches de 500 (`SQLITE_MAX_VARIABLE_NUMBER` safety), evitando queries DELETE com milhares de parametros que SQLite rejeitaria.

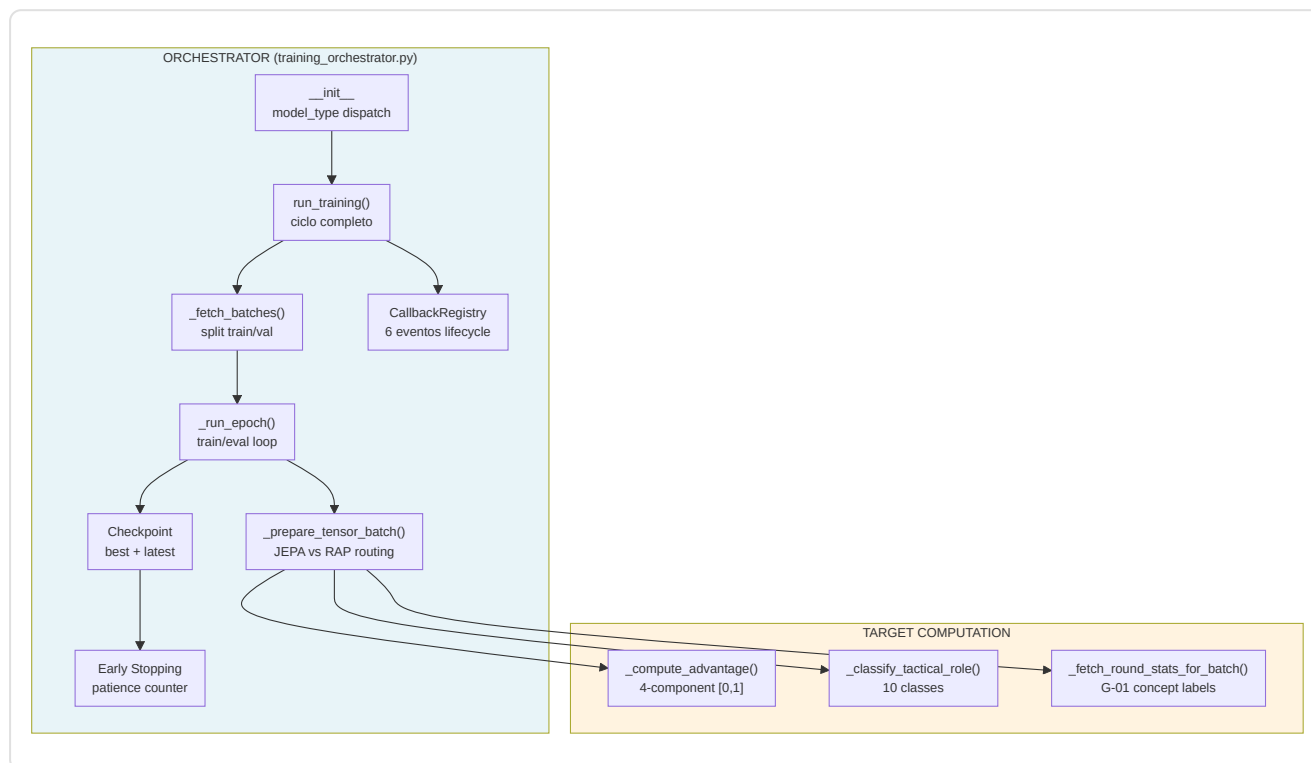
remote_file_server.py — Servidor de cloud storage pessoal construido em FastAPI para acesso remoto aos dados da aplicacao. Implementa autenticacao via API key com comparacao HMAC-safe (`hmac.compare_digest`) para prevenir timing attacks, rate limiting com janela deslizante (10 req/min por IP) via middleware ASGI, e protecao contra path traversal em todos os endpoints de acesso a arquivos. **Seguranca BE-07:** recusa binding em enderecos non-localhost sem TLS — se certificado e chave sao fornecidos, serve via HTTPS pelo uvicorn; caso contrario, restringe-se a `localhost`. Endpoints: `GET /list` (lista de arquivos com auth), `GET /download/{filename}` (download com auth e protecao path traversal), `GET /health` (health check sem auth).



12. Pipeline de Treinamento e Orquestracao

O subsistema de treinamento tem tres responsabilidades distintas:

1. **Orquestracao** — Ciclo de vida completo: carregamento dados -> epocas -> checkpoint -> parada antecipada
2. **Preparacao Tensores** — Conversao ticks brutos do banco em batches tensoriais prontos para o modelo
3. **Calculo Target** — Geracao de labels de treinamento: funcao vantagem (continua) e papel tatico (10 classes)



-TrainingOrchestrator (training_orchestrator.py)

A classe central da pipeline de treinamento. Gerencia JEPA, VL-JEPA e RAP de um unico ponto de entrada unificado, com dispatch baseado em `model_type`.

Construtor e Configuracao:

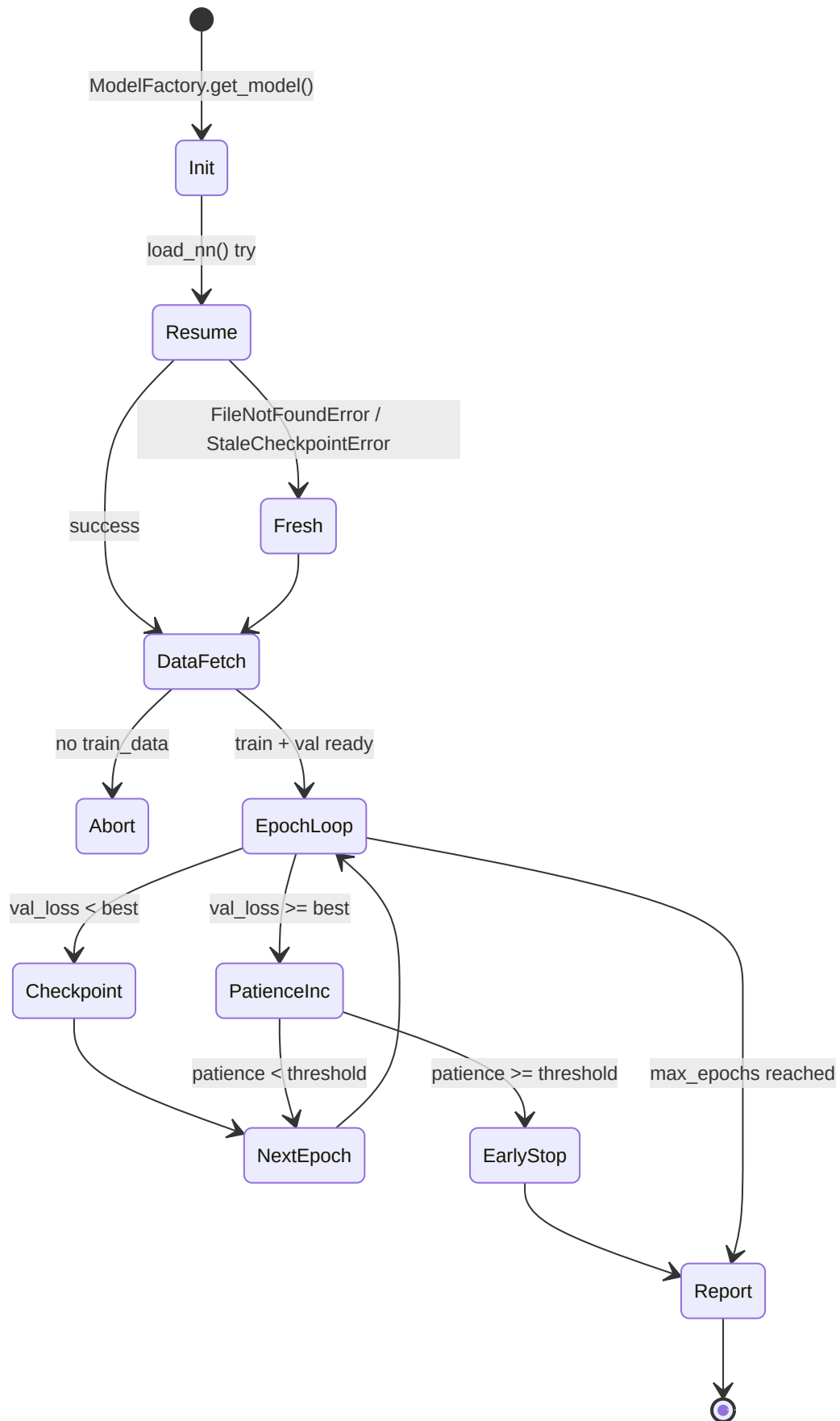
Parametro	Default	Descricao
<code>model_type</code>	<code>"jepa"</code>	Routing: <code>"jepa"</code> -> JEPATrainer, <code>"vl-jepa"</code> -> JEPATrainer (VL mode), <code>"rap"</code> -> RAPTrainer
<code>max_epochs</code>	100	Limite superior epocas
<code>patience</code>	10	Epocas sem melhoria antes de early stopping
<code>batch_size</code>	32	Amostras por batch
<code>callbacks</code>	<code>CallbackRegistry()</code>	Sistema plugin para TensorBoard, logging, etc.

Detalhes notaveis do construtor:

- **RNG deterministico** (F3-02): `np.random.default_rng(seed=42)` para a amostragem negativos JEPA — garante reprodutibilidade
- **Contadores fallback** (F3-11): `_total_samples` e `_total_fallbacks` rastreiam a taxa agregada de tensores-zero no ciclo inteiro
- **RAP gated**: O modelo RAP requer `USE_RAP_MODEL=True` nas configuracoes — se desabilitado, `ValueError` imediato
- **Learning rate**: JEPA 1e-4, RAP 5e-5 (mais conservativo para o multi-task)

Ciclo de Vida: `run_training()`

O metodo principal executa 6 fases sequenciais:



1. **Init:** `ModelFactory.get_model(type)` instancia o modelo, depois tenta `load_nn()` para retomar de checkpoint existente. Gerencia 3 excecoes: `FileNotFoundError` (primeiro treinamento), `StaleCheckpointError` (arquitetura mudada), e generico (corrupcao)
2. **Data Fetch:** `_fetch_batches(is_train=True/False)` com split train/val
3. **Epoch Loop:** Para cada epoca, `_run_epoch()` em modo train e depois eval. O `context.check_state()` permite interrupcao cooperativa (TrainingStopRequested)
4. **Checkpoint:** Duplo salvamento — `model_name` (best) e `model_name_latest` (sempre). Apenas o best e salvo quando `val_loss` melhora
5. **Early Stopping:** Contador `patience_counter` incrementado a cada epoca sem melhoria
6. **Report:** Callback `on_train_end` com metricas finais + log da taxa de fallback F3-11

Preparacao Dados: `_fetch_batches()`

Recupera ticks do banco de dados atraves do manager, respeitando o split train/val:

- Chama `manager._fetch_jepa_ticks(is_pro, split)` para todos os tipos de modelo (RAP incluido — pipeline dados RAP-especifica nao ainda implementada, com warning explicito)
- **Ordenamento temporal preservado** — nenhum shuffle, porque os modelos sao sequenciais
- Subdivisao em batches de tamanho `self.batch_size`

Routing Tensores: `_prepare_tensor_batch()`

O metodo mais critico: converte objetos `PlayerTickState` do banco de dados em dicionarios de tensores PyTorch.

Fluxo JEPA (context/target/negatives):

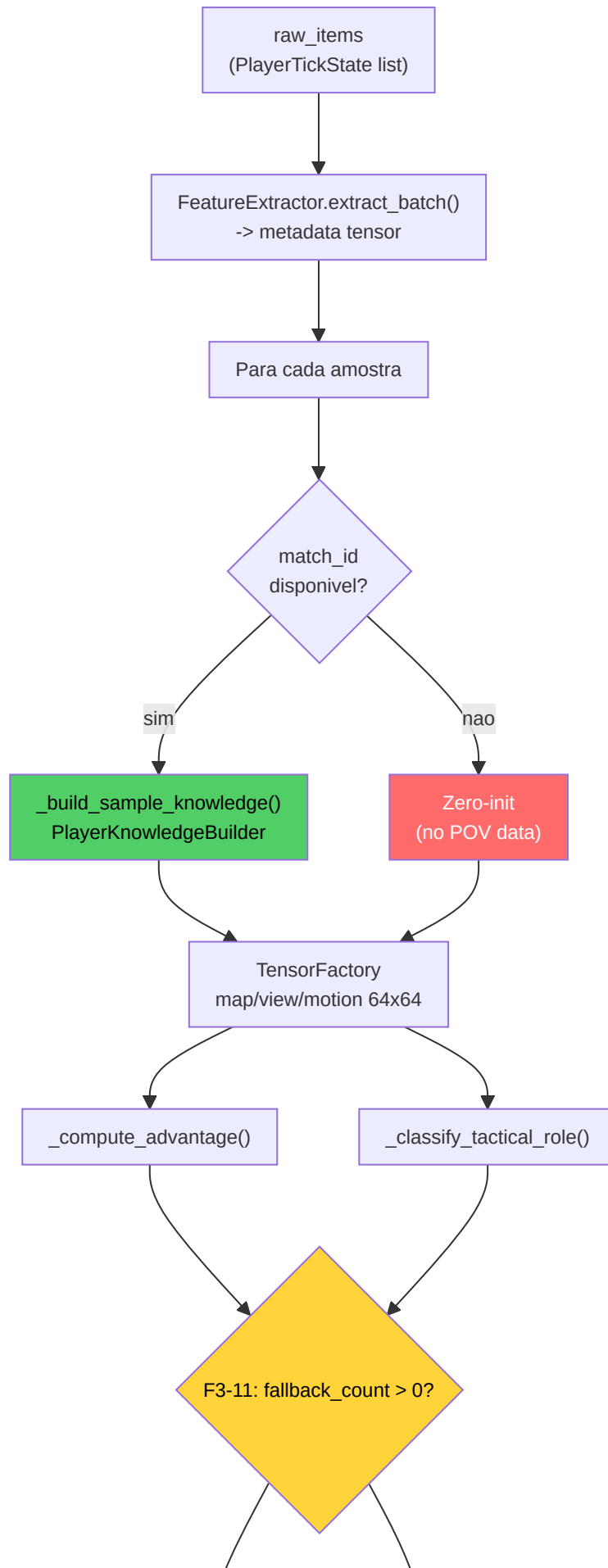
Tensor	Shape	Construcao
<code>context</code>	<code>(1, 10, METADATA_DIM)</code>	Primeiros 10 ticks do batch, padding se < 10
<code>target</code>	<code>(1, METADATA_DIM)</code>	Media do ultimo tick (next-item prediction)
<code>negatives</code>	<code>(1, 5, METADATA_DIM)</code>	5 amostras aleatorias do batch (RNG deterministico)

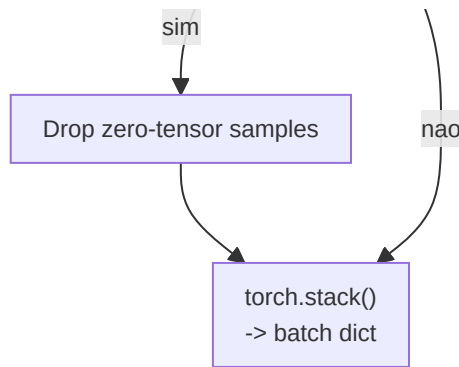
Requer pelo menos 5 amostras reais para os negativos contrastivos — se o batch e pequeno demais, retorna `None` (skip).

Correcao G-01: Para VL-JEPA, e tambem chamado `_fetch_round_stats_for_batch()` para fornecer labels conceituais baseadas nos resultados reais dos rounds (elimina label leakage de labeling heuristico).

Fluxo RAP (Player-POV completo):

O caminho RAP e significativamente mais complexo — delegado a `_prepare_rap_batch()`:





Para cada amostra no batch:

1. **Resolucao match_id** -> query ao `MatchDataManager` para todos os jogadores no tick
2. **PlayerKnowledgeBuilder** -> modelo sensorial NO-WALLHACK (visibilidade, som, utility)
3. **TensorFactory** -> tensores map/view/motion a 64x64 (resolucao training)
4. **Target computation** -> advantage [0,1] + papel tatico (10 classes)

Cache per-batch (4 dicionarios) evitam queries redundantes no mesmo match/tick:

Cache	Chave	Valor
<code>_all_players_cache</code>	<code>(match_id, tick)</code>	Lista jogadores no tick
<code>_window_cache</code>	<code>(match_id, tick)</code>	Historico 320 ticks para enemy memory
<code>_event_cache</code>	<code>(match_id, tick)</code>	Eventos no intervalo [-320, +64]
<code>_metadata_cache</code>	<code>match_id</code>	Metadados match (mapa, etc.)

Correcao F3-11: As amostras com fallback a zero-tensor sao **descartadas** (nao usadas para training). A taxa agregada e rastreada e se supera 10%, um warning e emitido. Se o batch inteiro e fallback, e pulado completamente.

Correcao H-03 — Remocao batch swallowing silencioso: Os loops de training e validacao do RAP nao contem mais o bloco `except (KeyError, TypeError): continue` que mascarava silenciosamente erros nos dados. Agora `train_step()` deve retornar um `dict` com chave `'loss'` ou e levantado um `ValueError` explicito. Isto garante que erros estruturais nos batches sejam detectados imediatamente ao inves de ignorados, melhorando a diagnosticabilidade do pipeline de treinamento.

Funcao Vantagem: `_compute_advantage()`

Formula continua [0, 1] que substitui o target binario vencido/perdido (G-04):

```

advantage = 0.4 x alive_diff + 0.2 x hp_ratio + 0.2 x equip_ratio + 0.2 x bomb_factor

```

Componente	Peso	Calculo	Range
<code>alive_diff</code>	0.4	<code>(team_alive - enemy_alive + 5) / 10</code>	[0, 1]
<code>hp_ratio</code>	0.2	<code>team_hp / (team_hp + enemy_hp)</code>	[0, 1]
<code>equip_ratio</code>	0.2	<code>team_equip / (team_equip + enemy_equip)</code>	[0, 1]
<code>bomb_factor</code>	0.2	Planted: T=0.7, CT=0.3 / Not planted: 0.5	{0.3, 0.5, 0.7}

O `alive_diff` tem o peso maior (0.4) porque a superioridade numerica e o unico fator mais preditivo no CS2. A normalizacao `(diff + 5) / 10` mapeia o range [-5, +5] (5v0 -> 0v5) em [0, 1].

Classificacao Papel Tatico: `_classify_tactical_role()`

Atribui um de 10 papeis taticos a cada tick baseando-se em heurísticas informadas do `PlayerKnowledge`:

Indice	Papel	Condicao
0	Site Take	T default (nenhuma condicao especial)
1	Rotation	— (reservado)
2	Entry Frag	Inimigos visiveis + distancia < 800u
3	Support	T + distancia media team < 500u + >=2 colegas
4	Anchor	CT + agachado
5	Lurk	Distancia media do team > 1500u + nenhum inimigo visivel
6	Retake	CT + bomba plantada
7	Save	Equipamento < \$1500 (prioridade maxima)
8	Aggressive Push	Inimigos visiveis + distancia >= 800u
9	Passive Hold	CT default (sem knowledge)

A cadeia de prioridade e: Save -> Retake -> Lurk -> Entry Frag -> Aggressive Push -> Anchor/Passive Hold -> Support -> Site Take. Sem `PlayerKnowledge`, recai em defaults baseados no team (CT -> Passive Hold, T -> Site Take).

Resolucao Mapa: `_resolve_map_name()`

Cadeia de fallback a 3 niveis para determinar o mapa de cada amostra:

1. **Match metadata DB** -> `MatchDataManager.get_metadata(match_id).map_name`
2. **Regex em demo_name** -> busca nomes mapa conhecidos (mirage, inferno, dust2, etc.)
3. **Fallback estatico** -> `"de_mirage"` (mapa mais jogado)

Validacao Contrastiva (Evaluation)

Durante a validacao, o calculo da loss JEPA requer uma passagem extra para os negativos:

1. Raw negatives `[batch, n_neg, feat_dim]` -> flatten a `[batch*n_neg, feat_dim]`
2. Encode atraves `target_encoder` -> `[batch*n_neg, latent_dim]`
3. Reshape a `[batch, n_neg, latent_dim]`
4. Calculo `jepa_contrastive_loss(pred, target, neg_latent)`

Para RAP, a validacao usa `trainer.criterion_val` (MSE em `value_estimate` vs `target_val`).

-JEPATrainer (`jepa_trainer.py`)

O trainer especializado para a arquitetura JEPA. Gerencia pre-training self-supervised, monitoramento drift, e o protocolo VL-JEPA a tripla loss.

Construtor:

Parametro	Default	Detalhe
<code>lr</code>	1e-4	Learning rate para AdamW
<code>weight_decay</code>	1e-4	Regularizacao L2
<code>drift_threshold</code>	2.5	Limiar z-score para DriftMonitor

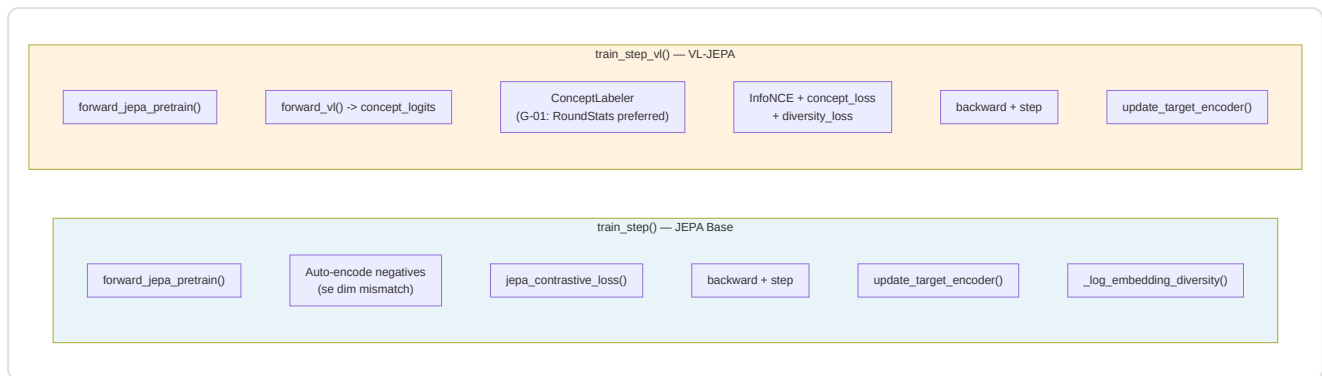
- **NN-36:** Os parametros do `target_encoder` sao **excluidos** do otimizador — sao atualizados apenas via EMA (Exponential Moving Average), nunca com gradientes diretos
- **KT-05:** Os parametros dos layers `concept` recebem um **multiplicador LR 0.05x** (5% do learning rate base) atraves de grupos de parametros separados no otimizador. Isto previne o collapse das embeddings conceituais durante o training VL-JEPA (por VL-JEPA paper, Section 4.6, Assran et al.)
- **Scheduler:** `CosineAnnealingLR` com `T_max=100`, step uma vez por epoca

Schedule EMA Momentum (J-6):

$$\text{tau}(t) = 1 - (1 - \text{tau_base}) \cdot (\cos(\pi \cdot t / T) + 1) / 2$$

Com `tau_base = 0.996`, o momentum parte de 0.996 (tracking rapido do target encoder) e converge a 1.0 (target encoder congelado) durante o training. Este schedule cosseno, adaptado de Assran et al. (CVPR 2023, Section 3.2), permite ao target encoder tracejar rapidamente o online encoder nas primeiras fases, depois estabilizar-se para fornecer targets consistentes na fase avancada do treinamento.

Duas modalidades de training step:

**train_step() (JEPA base):**

1. Forward -> `pred_embedding` , `target_embedding`
2. Auto-detect negativos raw (dim mismatch com `pred_embedding`) -> encode via `target_encoder`
3. `jepa_contrastive_loss(pred, target, negatives)` — InfoNCE
4. Backward + optimizer step
5. EMA update do target encoder
6. **P9-02**: Monitoramento variancia embeddings — warning se < 0.01 (risco collapse)

train_step_vl() (VL-JEPA):

Tres componentes de loss:

Loss	Peso	Funcao
InfoNCE	1.0	Contrastive self-supervised
Concept alignment	alpha = 0.5	Alinhamento conceitos a labels
Diversity regularization	beta = 0.1	Previne concept collapse

Correcao G-01: As labels conceituais sao preferencialmente geradas por `ConceptLabeler.label_from_round_stats(rs)` usando os dados reais de resultado round. Apenas como fallback e usado o labeling heuristico (com warning logged uma vez sozinho).

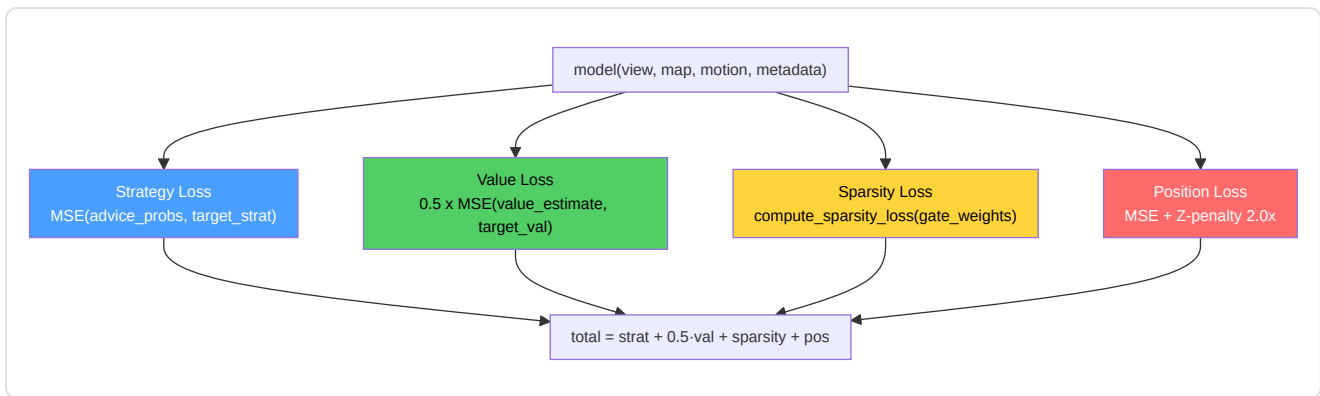
Sistema Drift (Task 2.19.3):

- `check_val_drift(val_df, reference_stats)` -> `DriftMonitor.check_drift()` com z-threshold 2.5
- `should_retrain(history, window=5)` -> True se drift persistente nas ultimas 5 epocas
- `retrain_if_needed()` -> ciclo completo de retreinamento com scheduler resetado

-RAPTrainer (rap_coach/trainer.py)

O trainer multi-task para o modelo RAP Coach. Gerencia 4 componentes de loss simultaneamente.

Loss Composition:



Loss	Peso	Criterio	Target
Strategy	1.0	MSE	<code>target_strat</code> (papel tatico one-hot, 10 classes)
Value	0.5	MSE	<code>target_val</code> (advantage [0,1])
Sparsity	1.0	L1 em gate weights	Interpretabilidade: pesos gate -> ~0
Position	1.0	MSE + Zx2.0	<code>target_pos</code> delta (opcional)

Task 2.17.1 — Penalidade Z-axis: A `compute_position_loss()` aplica um peso 2x no erro eixo Z (verticalidade), porque no CS2 os erros de elevacao (ex: inimigo em uma escada vs andar terreo) sao taticamente criticos.

F3-07: Os `gate_weights` sao passados explicitamente a `compute_sparsity_loss()` ao inves de lidos da instancia do modelo — garante thread-safety.

-Ponto de Entrada Training (train.py)

Modulo entry-point que fornece a funcao `train_nn()` com dispatch automatico e funcoes standalone.

`train_nn(X, y, X_val, y_val, model, config_name, context) :`

config_name	Percurso	Pipeline
"jepa"	<code>_train_jepa_self_supervised()</code>	Self-supervised, InfoNCE, 5 epocas prototype
outro	<code>_execute_validated_loop()</code>	Supervised, MSE, DataLoader standard

Funcoes helper:

Funcao	Responsabilidade
<code>_prepare_splits(X, y)</code>	Train/val split 80/20 com <code>random_state=42</code> . Rejeita se < 20 amostras (P1-04)
<code>_train_jepa_self_supervised()</code>	Prototipo JEPa: <code>SelfSupervisedDataset</code> , <code>context_len=10</code> , <code>prediction_len=5</code> , <code>clip_grad=1.0</code>
<code>_execute_validated_loop()</code>	Loop supervisionado com <code>EarlyStopping(patience=10)</code> , throttling configuravel
<code>run_training()</code>	Standalone entry: usa <code>CoachTrainingManager</code> para fetch dados pro

Nota P1-02: Todos os percursos chamam `set_global_seed()` antes do treinamento para garantir reprodutibilidade. O seed global e configurado no modulo `config.py`.

-WinProbabilityTrainer (win_probability_trainer.py)

Modulo dedicado ao modelo de probabilidade vitoria — a rede mais simples do ecossistema.

Arquitetura `WinProbabilityTrainerNN`:

```
Input(9) -> Linear(32) -> ReLU -> Linear(16) -> ReLU -> Linear(1) -> Sigmoid
```

9 Features de Input:

#	Feature	Tipo
1	<code>ct_alive</code>	Jogadores CT vivos
2	<code>t_alive</code>	Jogadores T vivos
3	<code>ct_health</code>	HP total CT
4	<code>t_health</code>	HP total T
5	<code>ct_armor</code>	Armadura total CT
6	<code>t_armor</code>	Armadura total T
7	<code>ct_eqp</code>	Valor equipamento CT
8	<code>t_eqp</code>	Valor equipamento T
9	<code>bomb_planted</code>	Bomba plantada (0/1)

Training:

- Loss: **BCELoss** (Binary Cross-Entropy — output e probabilidade [0,1])
- Optimizer: Adam (lr=0.001) — nao AdamW, mais simples para esta rede
- Early stopping: patience=10, min_delta=1e-4
- Target: `did_ct_win` (binario)
- Split: 80/20 com `random_state=42`
- Minimo amostras: 20 (AR-6)
- Salvamento: `torch.save(state_dict, model_path)` — persistencia direta, nao atraves de `save_nn()`

`predict_win_prob(model, state_dict)` — Inferencia singular: constroi o tensor das 9 features e retorna a probabilidade CT-win como float.

Nota A-12 — Arquiteturas distintas: Este `WinProbabilityTrainerNN` (9 features, treinamento offline) e um modelo *separado e incompativel* em relacao ao `WinProbabilityNN` de 12 features descrito na Secao 7. O primeiro e treinado nos dados agregados das demos; o segundo opera em tempo real durante a analise live. Os checkpoints nao sao intercambiaveis.

-Utilidades de Treinamento

Os seguintes modulos fornecem a infraestrutura de suporte ao ciclo de treinamento — configuracao, controle, monitoramento, dataset, e parada antecipada.

`training_config.py` — Dataclass centralizadas para hiperparametros:

Dataclass	Parametros Chave	Uso
<code>TrainingConfig</code>	<code>base_lr=1e-4</code> , <code>max_epochs=100</code> , <code>patience=10</code> , <code>batch_size=1</code> , <code>gradient_clip=1.0</code> , <code>ema_decay=0.999</code>	Configuracao base para todos os modelos
<code>JEPATrainingConfig</code>	Estende <code>TrainingConfig</code> + <code>latent_dim=256</code> , <code>contrastive_temperature=0.07</code> , <code>momentum_target=0.996</code> , <code>pretraining_epochs=50</code>	Especificas para JEPA self-supervised

`training_controller.py` — Gatekeeper que decide se uma nova demo deve ser usada para treinamento:

- **Quota mensal:** `MAX_DEMOS_PER_MONTH = 10` — conta os `PlayerMatchStats` processados nos ultimos 30 dias
- **Score de diversidade:** Compara a nova demo com os ultimos 5 matches via similaridade cosseno em 6 features normalizadas (kills, deaths, ADR, HS%, utility, opening duels). Se `diversity < 0.3`, a demo e rejeitada
- Retorna um `TrainingDecision(should_train, reason, diversity_score)` — o chamador decide se proceder

`training_monitor.py` — Persistencia metricas em JSON para monitoramento real-time e analise post-training. Rastreia: `epochs`, `train_loss`, `val_loss`, `learning_rate`, `best_val_loss`, estado (in progress / early stopped / completed).

`early_stopping.py` — Implementacao callable da parada antecipada:

```
stopper = EarlyStopping(patience=10, min_delta=1e-4)
if stopper(val_loss): # True = para o treinamento
    break
```

Logica: se `val_loss < best_loss - min_delta`, reset do contador. Senao incrementa. Quando `counter >= patience`, retorna `True`. Usado por todos os trainers (JEPA, RAP, Legacy, WinProb).

`dataset.py` — Dois Datasets PyTorch:

Dataset	Input	Output	Uso
<code>ProPerformanceDataset</code>	<code>(X, y)</code> tensors	<code>(x[i], y[i])</code> tuple	Training supervisionado Legacy
<code>SelfSupervisedDataset</code>	X sequencial	<code>(context, target)</code> janelas deslizantes	JEPA self-supervised

O `SelfSupervisedDataset` usa uma **janela deslizante**: `context_len=10` ticks como input, `prediction_len=5` ticks como target. O numero total de amostras e `len(X) - context_len - prediction_len`. Levanta `ValueError` se os dados sao insuficientes (F3-34).

-Sistema de Callback (`training_callbacks.py` + `tensorboard_callback.py`)

Arquitetura plugin a dois niveis para a observabilidade do training, sem modificar o loop principal.

Layer 1 — `training_callbacks.py`:

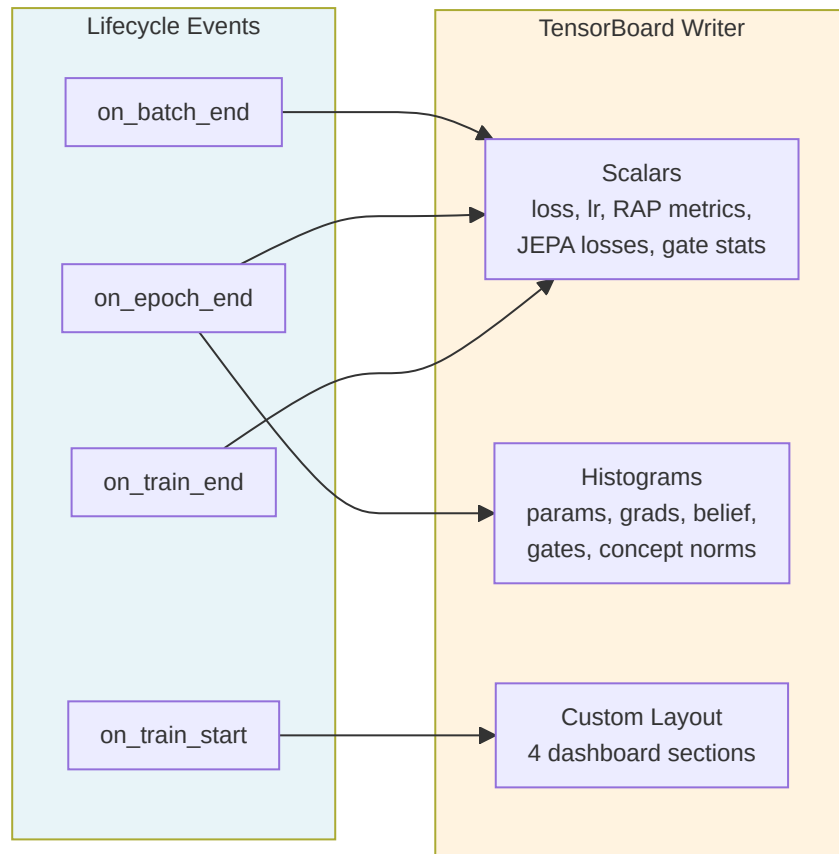
`TrainingCallback` (ABC) define 7 hooks de lifecycle:

Hook	Quando	Parametros
<code>on_train_start</code>	Antes da primeira epoca	model, config dict
<code>on_epoch_start</code>	Inicio de cada epoca	epoch
<code>on_batch_end</code>	Apos cada batch de training	batch_idx, loss, outputs dict
<code>on_epoch_end</code>	Fim de cada epoca	epoch, train_loss, val_loss, model, optimizer
<code>on_validation_end</code>	Apos a passagem de validacao	epoch, val_loss, model
<code>on_train_end</code>	Apos o completamento	model, final_metrics dict
<code>close</code>	Liberacao recursos	—

Nota F3-31: Nenhum metodo e `@abstractmethod` — pattern opt-in onde as subclasses implementam apenas os hooks necessarios. Todos os metodos base sao no-op.

`CallbackRegistry` e o dispatcher: `fire(event, **kwargs)` chama o hook correspondente em cada callback registrado. Erros nos callbacks sao capturados e logados — **nunca travam o training**.

Layer 2 — `tensorboard_callback.py`:



Sinais registrados por tipo de modelo:

Categoria	Métricas	Tipo
Core	<code>loss/train</code> , <code>loss/val</code> , <code>loss/gap</code> , <code>loss/batch</code>	Scalars per-epoch/batch
RAP	<code>rap/sparsity_ratio</code> , <code>rap/z_axis_error</code> , <code>rap/loss_position</code>	Scalars per-batch
JEPA	<code>jepa/infonce_loss</code> , <code>jepa/concept_loss</code> , <code>jepa/diversity_loss</code>	Scalars per-batch
Gates	<code>gates/mean_activation</code> , <code>gates/sparsity</code> , <code>gates/active_ratio</code>	Scalars per-batch
Diagnostica	<code>params/*</code> , <code>grads/*</code> , <code>belief/vector</code> , <code>concepts/embedding_norms</code> , <code>gates/activations</code>	Histogramas per-epoch

4 dashboards custom: **Coach Vital Signs**, **RAP Coach Internals**, **JEPA Self-Supervised**, **Superposition Gates**.

Import graceful: se `tensorboard` não está instalado, o callback se torna um no-op completo (`_TB_AVAILABLE = False`).

13. Funcoes de Perda — Detalhe Implementativo

Catalogo das Loss Functions



-jepa_contrastive_loss() — InfoNCE

Arquivo: `jepa_model.py:326` | **Signature:** `(pred, target, negatives, temperature=0.07) -> scalar`

Formula InfoNCE (Noise Contrastive Estimation):

$$L = -\log\left(\frac{\exp(\text{sim}(\text{pred}, \text{target}))}{\tau} / \left(\frac{\exp(\text{sim}(\text{pred}, \text{target}))}{\tau} + \sum \exp(\text{sim}(\text{pred}, \text{neg}))\right)\right)$$

Passagens:

1. **Normalizacao L2** de pred, target, negatives (cosine similarity space)
2. **Similitude positiva:** `pos_sim = (pred · target) / tau` com `tau = 0.07`
3. **Similitudes negativas:** `neg_sim = bmm(negatives, pred) / tau`
4. **Cross-entropy:** concat `[pos_sim, neg_sim]` -> `F.cross_entropy` com labels=0 (o positivo e sempre o indice 0)

A temperatura `tau = 0.07` e baixa, o que torna a loss **sensivel a pequenas diferencas** — apropriado para os embeddings CS2 onde os ticks consecutivos sao muito similares.

-vl_jepa_concept_loss() — Alinhamento Conceitual

Arquivo: `jepa_model.py:913` | **Signature:** `(concept_logits, concept_labels, concept_embeddings, alpha=0.5, beta=0.1) -> (total, concept, diversity)`

Dois componentes:

Componente	Formula	Peso	Proposito
Concept alignment	<code>BCE_with_logits(logits, labels)</code>	alpha = 0.5	Multi-label: cada conceito ativado independentemente
Diversity (VICReg)	<code>-mean(std(emb_norm, dim=0))</code>	beta = 0.1	Previne collapse: penaliza baixa variancia das embeddings conceituais

Os 16 conceitos de coaching (`COACHING_CONCEPTS`) cobrem: positioning (agressivo/passivo/exposto), economy (eco/force/full), timing (trade/rotate/patience), utility (flash/smoke/molotov), e communication (info/callout).

-compute_sparsity_loss() — Sparsidade Gate RAP

Arquivo: `rap_coach/model.py:105` | **Signature:** `(gate_weights) -> scalar`

```
L_sparsity = lambda x: mean(|gate_weights|)
```

L1 norm nos pesos do `ContextGate` (SuperpositionLayer). O objetivo e que a maioria dos gates esteja proxima de 0, com poucas ativacoes fortes — isto torna o modelo **interpretavel** (quais inputs influenciam realmente a decisao?).

Nota F3-07: Os `gate_weights` sao passados como parametro explicito ao inves de lidos de `self._last_gate_weights` — previne race condition em ambientes multi-thread.

-Position Loss RAP — Penalidade Z-axis

Arquivo: `rap_coach/trainer.py:89` | **Signature:** `(pred_delta, target_delta) -> (loss, z_error)`

```
L_pos = MSE(delta_x) + MSE(delta_y) + 2.0 * MSE(delta_z)
```

O peso 2x no eixo Z reflete a importancia critica da verticalidade no CS2: confundir o andar terreo com o primeiro andar (ex: Nuke, Vertigo) leva a decisoes taticas completamente erradas.

-Loss Legacy e Win Probability

- **TeacherRefinementNN:** `MSELoss` standard em predicoes supervised (train.py)
- **WinProbabilityNN:** `BCELoss` para output sigmoid [0,1] -> probabilidade CT-win

